Hochschule München
University of Applied Sciences

Design and Implementation of a Real-Time Impedance Controller for a 7-DOF Collaborative Robot

Entwurf und Implementierung eines Echtzeit-Impedanzreglers für einen 7-achsigen kollaborativen Roboter

Master Thesis

submitted for the degree of Master of Science (M.Sc.)

submitted at

Hochschule München

Faculty of Mechanical, Automotive and Aeronautical Engineering

submitted by

Heykel Khadhraoui

Study programme: Computational Engineering (M.Sc.)

Matriculation number: 41192924

Main supervisor:	Prof. Norbert Nitzsche
Second supervisor:	Prof. Ruth Otto
Issue date:	1 May 2026
Submission date:	1 November 2026

Declaration of Authorship / Verfassererklärung

I hereby declare that this master's thesis entitled

"Design and Implementation of a Real-Time Impedance Controller for a 7-DOF Collaborative Robot"

was written independently and without assistance from third parties. All sources, references, software tools, and other resources used have been cited accordingly. This work has not previously been submitted as an examination requirement in this or any other form.

Hiermit erkläre ich, dass ich die vorliegende Masterarbeit selbstständig und ohne fremde Hilfe angefertigt habe. Alle verwendeten Quellen, Hilfsmittel, Softwarewerkzeuge und sonstigen Ressourcen wurden entsprechend angegeben. Die Arbeit wurde bisher keiner anderen Prüfungsbehörde in gleicher oder ähnlicher Form vorgelegt.

Munich, _____ Heykel Khadhraoui

Date Signature

Colour-Coded Thesis Review

This copy is an editorial and evidential review of the thesis. It carries two kinds of mark. The first four colours appear as boxes and assess the section or subsection that follows the corresponding heading. They do not alter the thesis text, equations, tables, figures, or references.

GREEN The argument and presentation are sufficient for the present thesis. Minor polishing may still be possible.

YELLOW The content is relevant, but the explanation is dense, repetitive, weakly supported, or harder to follow than necessary.

RED The passage retains a generic, templated, or textbook-restatement pattern that can read as machine-produced. This is a stylistic assessment, not a claim about authorship or an AI-detector result.

GREY A reasoning step, evidence link, validation, or required measurement is absent, uncertain, or inconsistent elsewhere.

Green means sufficient, not perfect. A grey assessment does not make the reported data invalid; it identifies where the current evidence does not fully support the surrounding interpretation. Red is reserved for prose structure rather than technical correctness. Where a section contains both sound work and one important weakness, the more serious colour is used and the box names the exact concern.

The remaining two colours record the state of the text rather than an assessment of it, and are therefore applied to the words themselves.

PURPLE The passage has been read, checked and settled. Its wording is final and is not edited again.

BLUE Either new thesis text in a passage not yet settled, or a comment raised against

a settled passage.

Purple is not a judgement that the passage is strong. It records that the passage has been controlled and found correct, and that it already satisfies what the thesis requires of it. Nothing inside it is reworded, tightened, extended, or restructured afterwards. Rewording, sharpening, and stylistic improvement are not grounds for changing a settled passage, and a revised section that carries no blue at all is the normal case rather than an oversight.

Where a settled passage appears to need something, the text is left as it stands and the concern is raised beside it instead, as

The clearance gate holds the pose the tool reached and freezes the descend clock, until a bare Enter releases it. **comment:** “the frozen clock also governs the grind gate, which may be worth stating here”

A comment is a remark about the text, not a part of it. It appears only in this annotated copy and never in the submitted thesis, so a settled passage reads identically in both. It is raised only where the context or the reasoning is genuinely incomplete without the addition, not for anything that would merely read better.

Blue is used directly on the words themselves only in passages that have not yet been settled, where it marks new text as a whole sentence or a whole word and ends in a raised +. The two tints differ in hue so that the boundary survives greyscale printing.

The clean submission files remain `Thesis.pdf` and `Professor_Draft.pdf`. This annotated copy is generated separately as `Review_Draft.pdf`.

Abstract

GREEN — sufficient. The abstract states the problem, implemented mechanism, principal measured ordering, null-space result, and scope limits without overloading the reader with the full parameter matrix.

A small angular deviation causes one edge of a tool to contact a surface first. Under rigid position control the robot resists the rotation that would make the surfaces parallel, so contact forces and moments develop instead.

A real-time Cartesian impedance controller with phase-dependent stiffness and damping was implemented for a seven-degree-of-freedom (DOF) Franka Emika Panda. During contact set-up, gains defined at a virtual centre of compliance were transformed to the tool centre point; the resulting off-diagonal terms coupled translation and rotation, so the commanded normal press generated a corrective moment.

Commanded tilt, directional stiffness, and compliance-centre position were varied experimentally. Within the investigated range, the in-plane position of the compliance centre produced the largest change in alignment. A lever applied in the direction that assisted the corrective rotation gave approximately 6° of improvement in both tangent-axis tests, and the required direction was opposite for t_1 and t_2. Increased rotational stiffness reduced the alignment response, with axis-dependent differences between the softer settings.

The lever-induced moment keeps a fixed direction, whereas the rotational spring follows the measured error. A tangential lever therefore assists one tilt sign and opposes the other. Almost no improvement remained when it opposed the required rotation, so its direction must be selected from the measured tilt.

Separate free-space hold trials showed that projected null-space damping reduced redundant joint motion. A stronger singular-value conditioning torque increased joint motion without producing a larger final conditioning improvement.

The results apply to one robot, tool mounting, and surface. Every commanded tilt had the same sign, so the lever conclusions are limited to that sign. The end-effector-based metric may also underestimate the physical rotation of the grinding face.

Kurzfassung

GREEN — sufficient. The German summary remains aligned with the English abstract in structure, certainty, findings, and limitations.

Eine kleine Winkelabweichung führt dazu, dass eine Werkzeugkante die Oberfläche zuerst berührt. Bei starrer Positionsregelung behindert der Roboter die ausrichtende Drehung, sodass Kontaktkräfte und -momente entstehen.

Für einen 7-achsigen Franka Emika Panda wurde ein phasenabhängiger kartesischer Echtzeit-Impedanzregler implementiert. Beim Kontaktaufbau wurden Steifigkeit und Dämpfung vom virtuellen Nachgiebigkeitszentrum zum Werkzeugmittelpunkt transformiert. Die resultierende Kopplung wandelte den kommandierten Normaldruck in ein korrigierendes Moment um.

Vorgegebene Verkippung, richtungsabhängige Steifigkeit und Lage des Nachgiebigkeitszentrums wurden experimentell variiert. Im untersuchten Bereich hatte dessen Lage in der Ebene den größten Einfluss. Ein unterstützender Hebel verbesserte die Ausrichtung in beiden Tangentenachsen-Versuchen um etwa 6°; die erforderliche Richtung war für t_1 und t_2 entgegengesetzt. Eine höhere Rotationssteifigkeit verringerte die Ausrichtungsreaktion; Unterschiede zwischen den weichen Einstellungen waren achsenabhängig.

Das Hebelmoment behält eine feste Richtung, während die Rotationsfeder dem gemessenen Fehler folgt. Ein tangentialer Hebel unterstützt daher ein Vorzeichen der Verkippung und wirkt dem anderen entgegen. In der entgegenwirkenden Lage blieb nahezu keine Verbesserung, weshalb die Hebelrichtung aus der gemessenen Verkippung gewählt werden muss.

In getrennten Halteversuchen im freien Raum verringerte projizierte Nullraumdämpfung die redundante Gelenkbewegung. Ein stärkeres Konditionierungsdrehmoment erhöhte die Bewegung ohne weitere Konditionierungsverbesserung.

Die Aussagen gelten nur für die untersuchte Anordnung und ein Verkippungsvorzeichen; die EE-Kennzahl kann die Drehung der Schleiffläche unterschätzen.

Contents

List of Figures

List of Tables

List of Abbreviations

GREEN — sufficient. The abbreviations are separated from the symbol list and their printed forms are controlled centrally.

CSV	Comma-Separated Values
DOF	Degree of Freedom
FCI	Franka Control Interface
SVD	Singular Value Decomposition

List of Symbols

GREEN — sufficient. The notation is extensive but well organised, and the unit column makes the mixed translational and rotational quantities traceable.

Frames and Kinematics

$\{0\}$	Robot base frame	[-]
$\{i\}$	Joint frame i	[-]
$\{\mathrm{EE}\}$	End-effector (tool) frame	[-]
n	Number of joints ($n = 7$)	[-]
$q,\ \dot{q}$	Joint position / velocity vector	[rad], [rad/s]
q_0	Home or reference joint configuration	[rad]
x	Stacked Cartesian end-effector state, $x = f(q)$	[m], [rad]
$f(\cdot)$	Forward-kinematics map	[-]
$x,\ y,\ z$	Scalar Cartesian coordinate directions or components; not the stacked state x	[m]
p	Position / translation vector	[m]
R	Rotation matrix	[-]
$R_x,\ R_y,\ R_z$	Elementary rotations about x, y, z	[-]
$\alpha,\ \beta,\ \gamma$	Generic rotations about the x, y, z axes	[rad]

T	Homogeneous transformation matrix	[-]
$J(q)$	Geometric Jacobian; linear rows above angular rows	[m/rad], [-]
z_{i-1}	Rotation axis of joint i, expressed in the base frame	[-]
p_{i-1}	Point on the axis of joint i, expressed in the base frame	[m]
$x_{\mathrm{EE}}, y_{\mathrm{EE}}, z_{\mathrm{EE}}$	End-effector frame axes (base frame)	[-]

Pose and Error

p_{EE}	End-effector position	[m]
$R_{\mathrm{EE}},\ T_{\mathrm{EE}}$	End-effector orientation / transformation	[-]
$p_d,\ R_d,\ T_d$	Desired position / orientation / transformation	[m], [-]
T_{err}	Relative (error) transformation	[-]
ΔR	Current-to-desired relative rotation, $R_{\mathrm{EE}}^{\top} R_d$	[-]
ϕ	Orientation-error angle	[rad]
$u,\ [u]_{\times}$	Current-EE-frame orientation-error axis / its skew matrix	[-]
e_p	Translational (position) error	[m]
e_R	Base-frame rotational error used by the command	[rad]
$e_{R,\mathrm{EE}}$	Rotational error expressed in the current EE frame	[rad]
$e_{p,c},\ e_c$	Compliance-centre position error / stacked error	[m], [rad]
e	Stacked Cartesian error	[m], [rad]
$\dot{x}$	Cartesian velocity (twist)	[m/s], [rad/s]
$\dot{p}_{\mathrm{EE}},\ \dot{p}_d,\ \omega_{\mathrm{EE}}$	Measured linear end-effector velocity, desired linear velocity, and angular end-effector velocity	[m/s], [m/s], [rad/s]

Cartesian Impedance Gains

$K_p,\ D_p$	Generic translational stiffness / damping block; a frame subscript is added when needed	[N/m], [N s/m]
$K_R,\ D_R$	Generic rotational stiffness / damping block; a frame subscript is added when needed	[N m/rad], [N m s/rad]
$R_{\text{task}},\ T_R$	Task-frame orientation and corresponding block rotation	[-]
$K_{p,\text{task}},\ D_{p,\text{task}}$	Local translational stiffness / damping in the chosen task frame	[N/m], [N s/m]
$K_{R,\text{task}},\ D_{R,\text{task}}$	Local rotational stiffness / damping in the chosen task frame	[N m/rad], [N m s/rad]
$K_{p,0},\ D_{p,0}$	Translational stiffness / damping used in the base-frame command	[N/m], [N s/m]
$K_{R,0},\ D_{R,0}$	Rotational stiffness / damping used in the base-frame command	[N m/rad], [N m s/rad]
$K_0,\ D_0$	Full 6×6 stiffness / damping used in the base-frame command	K: [N/m], [N m/rad]; D: [N s/m], [N m s/rad]

Dynamics and Torques

F	Cartesian wrench	[N], [N m]
$f,\ m$	Cartesian force / moment	[N], [N m]
$K,\ D$	Full Cartesian stiffness / damping matrices	K: [N/m], [N m/rad]; D: [N s/m], [N m s/rad]
Λ	Operational-space (Cartesian) inertia matrix	[kg], [kg m], [kg m^2]
$M(q)$	Joint-space mass matrix	[kg m^2]
M_i	Directional translational or rotational inertia	[kg] or [kg m^2]
ζ	Directional inertia-scaled-damping ratio used as a tuning heuristic	[-]
τ	Joint torque vector	[N m]
$\tau_c,\ \tau_g$	Coriolis / gravity compensation torque	[N m]
$\tau_{\text{cmd}},\ \tau_{\text{task}}$	Commanded / task torque	[N m]

Surface and Contact Geometry

$n_s,\ n_{\text{phys}}$	Configured surface normal / independently calibrated physical-plane normal	[-]
$t_1,\ t_2$	Surface tangent directions (unit)	[-]
$a_s,\ \tilde{t}_1$	Auxiliary surface direction / projected tangent before normalization	[-]
R_{surface}	Surface task frame $[\,t_1\ \ t_2\ \ n_s\,]$ in the base frame	[-]
$a_{\text{EE}},\ a_T,\ a_d$	Configured EE-frame tool axis / current base-frame tool axis / signed target axis	[-]
s_a	Tool-axis target sign, with $a_d = s_a n_s$	[-]
K_{t_1}, K_{t_2}, K_n	Translational stiffness along $t_1,\ t_2,$ and n_s	[N/m]
$K_{r,t_1}, K_{r,t_2}, K_{r,n}$	Rotational stiffness entries in surface-frame column order	[N m/rad]
$h_C,\ h_{\text{lead}},\ \ell_{\text{lead}}$	Averaged-feature height / exact leading-corner height / leading descent projection	[m]
p_s	Point on the configured surface	[m]
$p_e,\ a_e$	Tool-edge point / edge direction	[m], [-]
$p_{\Pi,\text{clr}}$	Selected tool feature projected onto the plane at clearance	[m]
$c_{C,\text{EE}},\ R_{\text{clr}}$	Selected-feature offset in the EE frame / orientation selected at clearance; both are held constant during set-up	[m], [-]
p_c	Centre of compliance, held constant from first contact	[m]

r_c	Pole-to-TCP lever, $p_{\mathrm{TCP}} - p_c$	[m]
h	Selected normal offset of the centre of compliance	[m]
p_C	Physical contact point	[m]
r_C	TCP-to-contact lever, $p_C - p_{\mathrm{TCP}}$	[m]
f_C	Environment-on-tool contact force used in the geometric sketch	[N]

Compliance and Alignment

$K_{\mathrm{task}},\ D_{\mathrm{task}}$	Full local Cartesian stiffness / damping in the chosen task frame	K: [N/m], [N m/rad]; D: [N s/m], [N m s/rad]
$K_{pR,\mathrm{task}}$, $K_{Rp,\mathrm{task}}$	Task-frame translation–rotation coupling sub-blocks	[N/rad], [N m/m]
$\mathrm{Ad}(r_c),\ \mathrm{Ad}_g$	Pole-to-TCP point-shift adjoint / alternative notation for that offset	[-]
K_c, D_c	Block-diagonal stiffness / damping at the centre of compliance	K: [N/m], [N m/rad]; D: [N s/m], [N m s/rad]
$K_{\mathrm{TCP}}, D_{\mathrm{TCP}}$	Centre-of-compliance gains mapped by congruence to the TCP	K: [N/m], [N m/rad]; D: [N s/m], [N m s/rad]
K_{coupled}	Coupled full stiffness about the selected centre	[N/m], [N/rad], [N m/m], [N m/rad]
$e_{p,\mathrm{task}},\ e_{R,\mathrm{task}}$, $e_{\mathrm{task}},\ \dot{e}_{\mathrm{task}}$	Task-frame translational / rotational / stacked error and error velocity	[m], [rad], [m/s], [rad/s]
$\theta_{\mathrm{align}},\ \Delta\theta_{\mathrm{align}}$	EE-inferred tool-axis alignment angle / before–after improvement	[rad]
$\Delta\theta_C$	Finite before–after rotation vector during contact	[rad]

Quasi-Static Stiffness and Damping Design

$k_{p,i},\ k_{R,i}$	Principal translational / rotational stiffness	[N/m], [N m/rad]
$k_{p,\mathrm{eff}}(u)$	Effective translational stiffness along unit direction u	[N/m]
d_i	Directional damping coefficient	[N s/m] or [N m s/rad]

Null-Space and Singular Value Decomposition

J_ρ^+	Truncated-SVD Moore–Penrose pseudoinverse	[rad/m], [-]
$N_q,\ N_\tau$	Joint-velocity / torque null-space projector	[-]
$\rho,\ \varepsilon_\rho$	Relative SVD tolerance / resulting singular-value cutoff	[-], [m/rad]
$\sigma_i,\ \sigma_{\min}$	Jacobian singular value / σ_6 conditioning indicator; its value follows the length unit of the linear rows	[m/rad]
λ_i	Eigenvalue of $JJ^\top$, equal to σ_i^2	[m^2/rad^2]
$U_J,\ \Sigma_J,\ V_J$	Singular value decomposition factors of J	[-], [m/rad], [-]
$u_i,\ v_i$	Left / right singular vectors	[-]
$v_6,\ v_7$	σ_6-associated direction / structural null direction	[-]
d_{null}	Null-space damping	[N m s/rad]
k_σ	Sigma-conditioning torque magnitude	[N m]
α_{probe}	Null-direction sampling angle	[rad]
$\delta_\sigma,\ \Delta_\sigma$	Sigma-comparison deadband / sampled difference	[m/rad]

s	Deadbanded conditioning-direction selector	[-]
$\tau_d,\ \tau_\sigma,\ \tau_{\text{null}}$	Projected damping / sigma / total null-space torque	[N m]

Centre-of-Compliance Notation

$K_{p_c},\ D_{p_c}$	Full stiffness / damping matrix defined about the centre of compliance p_c	K: [N/m], [N m/rad]; D: [N s/m], [N m s/rad]
$K_{p,c},\ K_{R,c}$	Translational / rotational stiffness blocks defined about the centre of compliance	[N/m], [N m/rad]
$D_{p,c},\ D_{R,c}$	Translational / rotational damping blocks defined about the centre of compliance	[N s/m], [N m s/rad]

Miscellaneous

T_s	Control sampling time (1 ms)	[s]
$I,\ I_4,\ I_7$	Identity matrices	[-]

Chapter 1

Introduction

PURPLE — revised and confirmed. The chapter was revised in full and every paragraph is settled. No further editing is to be applied to it, and any remaining concern is raised as a comment rather than as a change.

1.1 Motivation

Industrial manipulators commonly apply position control to track commanded trajectories. In surface-finishing applications, the actual surface pose may differ from the programmed pose because of placement tolerances. The surface is positioned manually, the tool is held in a gripper, and the residual angle between them is unknown before contact. Under rigid position control, this angular deviation cannot be absorbed by the commanded motion and generates high contact forces.

Torque-controlled collaborative robots permit contact compliance to be specified directly, and they account for a growing share of industrial deployments [11, 21]. Joint torque sensing and a torque-level command interface let the controller specify how the robot yields under contact, not only where it goes. This capability is particularly useful in surface-finishing tasks, where a small angular mismatch between the tool and the surface can generate undesired contact forces and moments. The objective is therefore not only to reduce these loads, but also to enable passive angular alignment of the grinding face during contact. The controller should allow the tool to rotate toward the surface and reduce the angular mismatch while maintaining the commanded normal contact and the subsequent tangential grinding motion. The resulting alignment is evaluated relative to the physical

surface.

Impedance control specifies this yielding behaviour as a dynamic relation between motion and interaction force [10]. Cartesian impedance control defines this relation in task space, meaning in terms of the Cartesian position and orientation of the end-effector rather than the individual joint coordinates. The compliance can therefore be adjusted separately for different translational and rotational directions. This direction-dependent behaviour is important during contact. A tool moving across a surface should maintain its normal position, while small lateral deviations should not generate large forces. The same principle applies to orientation. When the edge of a tilted tool contacts the surface, the contact wrench contains both a force and a moment. Stiffness values chosen according to the contact geometry allow the tool to rotate slightly toward the surface instead of fully resisting this motion. The resulting angular response is evaluated from the end-effector pose relative to the physical surface. The stiffness and damping matrices therefore determine both the contact response and the free-space tracking behaviour.

1.2 Related Work

The relevant literature comprises three areas: Cartesian impedance control, real-time implementation on torque-controlled robots, and the placement of the centre of compliance. The first area is Cartesian impedance control itself. The controller uses standard serial-manipulator kinematics [19, 3, 15]: homogeneous transformations for representing the end-effector pose and the Jacobian for relating joint motion to Cartesian motion. Khatib established the operational-space formulation, in which a Cartesian wrench is mapped to joint torques through $J^\top$ [12]; this mapping is adopted in the controller. Hogan defined the impedance relation between end-effector motion and interaction force [10]. Albu-Schäffer et al. implemented Cartesian impedance control using joint-torque sensing [1], which corresponds to the torque-controlled interface used in this thesis.

The second area concerns the real-time implementation of Cartesian impedance control on torque-controlled robots. Mayr et al. implemented a modular Cartesian impedance controller for the Franka Emika Panda that combines real-time model access, null-space control, signal treatment, and command limiting [14]. The controller developed in this thesis uses the libfranka model interface to obtain the required robot state and model

quantities during the real-time control loop. It also includes a selectable null-space torque for influencing the redundant joint motion without changing the commanded Cartesian task. No separate application-level saturation of the commanded Cartesian wrench, joint torque, or torque rate was applied. The configured robot-side collision thresholds and collision monitoring therefore remained the independent mechanism for interrupting the motion when the corresponding limits were exceeded.

The third area concerns the location of the centre of compliance. Fasse and Broenink formulate spatial impedance about a selected point, so that translation and rotation become coupled through the position of this point [6]. This formulation provides the theoretical basis for the compliance-centre shift used in this thesis. The same principle appears mechanically in remote-centre-compliance devices, in which elastic elements are arranged so that the tool behaves as if it rotates about a virtual point located away from the mechanism itself [16, 23]. More recent mechanically compliant and variable-compliance-centre strategies apply this principle to peg-in-hole insertion [22, 9]. In the mechanically compliant mechanisms, elastic bending produces the alignment motion rather than software control alone. In these approaches, the compliance is arranged so that contact forces reduce misalignment instead of increasing it. Most of these studies focus on insertion tasks and define the compliance centre through either the mechanical design or a strategy adapted to the hole geometry. In this thesis, an equivalent effect is created in software by shifting the virtual centre of compliance away from the TCP. The independent effects of this position and the axis-specific stiffness during planar tool alignment are then investigated on a torque-controlled robot.

1.3 Problem Statement

The angular relation between the tool and the surface plane is uncertain before contact because both the surface placement and the tool mounting carry tolerances. Under rigid position control, this residual angle produces an undesired contact moment rather than a corrective motion. A tool tilted with respect to the surface cannot be brought flat by translational compliance alone, because the wrench that closes the gap must also contain a moment. Translational compliance therefore does not by itself produce the required corrective rotation. Placing the centre of compliance away from the tool centre point (TCP)

couples translation and rotation, so that pressing the tool into the plane also causes it to rotate toward the plane. This placement defines a task-to-impedance mapping rather than a physical pivot. The resulting response was evaluated from the motion measured during contact.

The alignment response cannot be predicted from the gain values alone because it also depends on the contact geometry, the lever direction, the robot configuration, and the compliance of the tool mounting. The experiments therefore compare the effects of a commanded tilt representing the initial angular mismatch between the tool and the surface, the directional stiffness, and the compliance-centre placement under otherwise matched conditions. Chapters 2 and 3 introduce the frame conventions and controller formulation required for these comparisons.

1.4 Scope and Contributions

Within this scope, the experiments evaluate how a commanded tilt representing the initial angular mismatch between the tool and the surface, the rotational and translational stiffness in the surface directions, and the position of a virtual centre of compliance influence contact-induced alignment. The controller is a phase-scheduled Cartesian impedance controller written in C++ using libfranka and Eigen. It runs in the 1 kHz torque-control loop of the Franka Emika Panda. The commanded Cartesian wrench is mapped to joint torques through $J^\top$, and the Coriolis torque is obtained from the robot model and added to the command. Each control phase uses its own Cartesian stiffness values, while the damping is calculated from an estimate of the Cartesian inertia.

A surface-fixed coordinate frame is constructed from the surface geometry. Its axes consist of the unit surface normal n_s and two mutually orthogonal unit vectors t_1 and t_2 lying in the surface plane. The directions t_1 and t_2 therefore represent the two independent tangential directions along the surface. This frame is used to define the translational and rotational stiffness values according to the contact geometry.

Each run follows a fixed sequence of control phases. First, the tool is brought to the commanded orientation and approaches the surface up to a specified geometric clearance. Based on the rectangular grinding-face geometry and the current tool orientation, the

controller predicts the point on the tool expected to reach the surface first. For this purpose, the four tool corners are compared along the approach direction. A single leading corner is selected for a general tilt. If two neighbouring corners are equally close to the surface, their midpoint is used as the predicted contact point at the centre of the leading edge. If all four corners are equally close to the surface, the centre of the grinding face is used. This predicted contact point is fixed when the clearance is reached and remains unchanged during set-up and grinding.

During contact set-up, the desired position of the predicted contact point is moved gradually from the clearance position toward the surface and slightly beyond the surface plane. The physical surface prevents the tool from reaching this virtual target, so the resulting position error generates the normal pressing force. Because the robot command is defined at the TCP, the controller calculates the corresponding TCP target using the fixed geometric offset between the TCP and the predicted contact point.

The desired tool orientation remains equal to the orientation reached at the clearance point, but it is not enforced rigidly. The finite rotational stiffness allows the actual tool orientation to change when the contact forces produce a moment, so the tool can rotate toward the surface during set-up.

During this phase, the 6×6 stiffness and damping matrices defined at the virtual centre of compliance are transformed to the TCP through an adjoint congruence transformation. The resulting off-diagonal blocks couple translation and rotation, allowing the commanded pressing motion to generate a corrective moment that helps the tool align with the plane surface during set-up. The effects of this translation–rotation coupling are investigated in this thesis. After contact set-up, the controller proceeds to the tangential grinding motion.

Accordingly, the contributions of this thesis are divided into implemented controller components and experimentally established findings. The implemented work includes a frame-consistent real-time Cartesian impedance controller for the 7-DOF Franka Emika Panda. It also includes a spatial compliance-centre formulation in which the 6×6 stiffness and damping matrices are defined at a selected virtual centre of compliance and transformed to the TCP. In addition, a contact and evaluation methodology was developed in which the physical surface orientation and the grinding-face axis are defined independently, allowing the alignment of the tool to be evaluated relative to the surface.

The experiments provide an axis-specific evaluation of how rotational and translational stiffness influence contact-induced alignment. Within the tested range, the position of the virtual centre of compliance produced the largest measured change in alignment. This effect depends on the lever direction: a lever that assists alignment about t_1 must be reversed to assist alignment about t_2.

Projected joint damping and a two-point smallest-singular-value bias were also implemented for the redundant seventh joint. Their effects were evaluated in a separate free-space hold study using an internally generated joint-torque disturbance equivalent to a point force at the end-effector. The results describe the null-space response to this commanded disturbance.

1.5 Thesis Structure

The thesis is structured as follows. Chapter 2 introduces the impedance law, the frame conventions, and the compliance-centre shift. Chapter 3 describes their real-time implementation on the Franka Emika Panda. Chapter 4 presents the experimental methodology, while Chapter 5 reports and discusses the corresponding results. Chapter 6 summarises the main conclusions and outlines future work.

Chapter 2

Theoretical Background

PURPLE — revised and confirmed. The chapter was revised in full and every paragraph is settled. No further editing is to be applied to it, and any remaining concern is raised as a comment rather than as a change.

This chapter presents the mathematical basis of the Cartesian impedance controller. The required frame conventions and pose errors are introduced first. The subsequent sections derive the kinematic mappings, the impedance wrench, the compliance-centre transformation, and the null-space torque.

2.1 Reference Frames

The following right-handed coordinate frames are used in the theoretical formulation and the controller [19, 3].

- **Base frame** $\{0\}$**:** The base frame is fixed to the robot mounting surface. Its origin is the robot base reference point used by the kinematic model. For the upright Panda configuration used in this thesis, the z_0-axis is normal to the mounting surface, while the x_0- and y_0-axes lie in the mounting plane. Unless stated otherwise, Cartesian positions, directions, velocities, forces, and moments are expressed in this frame.
- **Joint frames** $\{1\}, \{2\}, \ldots, \{n\}$**:** These frames are attached to the successive links of the kinematic chain according to the Denavit–Hartenberg convention [4, 3]. Joint i rotates about z_{i-1}, and p_{i-1} denotes a point on this axis. The joint frames define the transformations between consecutive links and the indexing used in the forward

kinematics and geometric Jacobian.

- **End-effector frame** {EE}**:** The end-effector frame is attached to the TCP and moves with the tool. Its position p_{EE} and orientation R_{EE}, both expressed relative to the base frame, define the current Cartesian pose of the robot. The axes x_{EE}, y_{EE}, and z_{EE} describe the current tool orientation. The configured tool approach axis is defined in this frame and is transformed to the base frame using R_{EE}.
- **Desired end-effector frame** $\{d\}$**:** The desired frame represents the commanded TCP pose. Its origin is p_d, and its orientation is R_d, both expressed relative to the base frame. The difference between the desired and current end-effector frames provides the translational and rotational errors used by the impedance controller.
- **Surface task frame** $\{S\}$**:** The surface task frame is tied to the configured surface rather than to the moving tool. Its origin is placed at the configured surface point p_s, and its orientation is defined by the two surface tangents t_1 and t_2 and the surface normal n_s:

$$R_{\mathrm{surface}} = \begin{bmatrix} t_1 & t_2 & n_s \end{bmatrix}. \tag{2.1}$$

The directions t_1 and t_2 lie in the surface plane and are mutually orthogonal, while n_s is normal to the surface. This frame is used to describe the commanded tilt and to assign stiffness and damping values according to the two tangential directions and the normal direction. For the contact task,

$$R_{\mathrm{task}} = R_{\mathrm{surface}}. \tag{2.2}$$

2.2 Rigid-Body Transformations

The positions and orientations of the frames introduced in Section 2.1 are related through rotation matrices and homogeneous transformations.

2.2.1 Rotation Matrices

An orientation is represented by a rotation matrix $R \in SO(3)$, satisfying

$$R^\top R = I, \qquad \det R = 1, \qquad R^{-1} = R^\top. \tag{2.3}$$

The notation ${}^{b}R_{a}$ denotes the orientation of frame $\{a\}$ expressed in frame $\{b\}$. Its columns contain the axes of frame $\{a\}$ expressed in frame $\{b\}$. A vector is transformed from frame $\{a\}$ to frame $\{b\}$ according to

$${}^{b}v = {}^{b}R_{a}\,{}^{a}v. \tag{2.4}$$

The elementary right-handed rotations used later to define the commanded tool tilt are

$$R_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\alpha & -\sin\alpha \\ 0 & \sin\alpha & \cos\alpha \end{bmatrix}, \; R_y(\beta) = \begin{bmatrix} \cos\beta & 0 & \sin\beta \\ 0 & 1 & 0 \\ -\sin\beta & 0 & \cos\beta \end{bmatrix}, \; R_z(\gamma) = \begin{bmatrix} \cos\gamma & -\sin\gamma & 0 \\ \sin\gamma & \cos\gamma & 0 \\ 0 & 0 & 1 \end{bmatrix}. \tag{2.5}$$

2.2.2 Homogeneous Transformations

A position is represented by a vector $p \in \mathbb{R}^3$. Rotation and translation are combined in a homogeneous transformation [3, 19]:

$${}^{b}T_{a} = \begin{bmatrix} {}^{b}R_{a} & {}^{b}p_{a} \\ 0^\top & 1 \end{bmatrix}, \tag{2.6}$$

where ${}^{b}p_{a}$ is the position of the origin of frame $\{a\}$ expressed in frame $\{b\}$. A point expressed in frame $\{a\}$ is transformed to frame $\{b\}$ according to

$${}^{b}p = {}^{b}R_{a}\,{}^{a}p + {}^{b}p_{a}. \tag{2.7}$$

Successive transformations are composed by matrix multiplication:

$${}^{c}T_{a} = {}^{c}T_{b}\,{}^{b}T_{a}. \tag{2.8}$$

2.2.3 End-Effector Pose

The controlled pose consists of the end-effector position

$$p_{\mathrm{EE}} = \begin{bmatrix} p_{x,\mathrm{EE}} & p_{y,\mathrm{EE}} & p_{z,\mathrm{EE}} \end{bmatrix}^{\top} \in \mathbb{R}^3 \tag{2.9}$$

and the orientation $R_{\mathrm{EE}} \in SO(3)$. Both are expressed in the base frame and combined in the homogeneous transformation

$$T_{\mathrm{EE}} = \begin{bmatrix} R_{\mathrm{EE}} & p_{\mathrm{EE}} \\ 0^{\top} & 1 \end{bmatrix}. \tag{2.10}$$

In the implementation, T_{EE} is obtained directly from the reported robot state, and the orientation is not converted to an Euler-angle representation.

2.2.4 Cartesian Pose Error

Let the desired end-effector pose be

$$T_d = \begin{bmatrix} R_d & p_d \\ 0^{\top} & 1 \end{bmatrix}, \tag{2.11}$$

where p_d and R_d are the desired TCP position and orientation, respectively. The current end-effector pose is denoted by T_{EE}.

The relative transformation from the current end-effector frame to the desired frame is

$$T_{\mathrm{err}} = T_{\mathrm{EE}}^{-1} T_d = \begin{bmatrix} R_{\mathrm{EE}}^{\top} R_d & R_{\mathrm{EE}}^{\top}(p_d - p_{\mathrm{EE}}) \\ 0^{\top} & 1 \end{bmatrix}. \tag{2.12}$$

When the current and desired poses coincide, $T_{\mathrm{err}} = I_4$. The rotational and translational blocks of eq. (2.12) therefore both represent zero error in this case.

Position error. The translational block of eq. (2.12) expresses the position error in the current end-effector frame. The Cartesian impedance law uses the equivalent restoring error expressed in the base frame:

$$e_p = p_d - p_{\mathrm{EE}} \in \mathbb{R}^3. \tag{2.13}$$

A positive position error therefore points from the current TCP position toward the desired TCP position.

Orientation error. The rotational block of eq. (2.12) is

$$\Delta R = R_{\mathrm{EE}}^{\top} R_d. \tag{2.14}$$

It represents the rotation required to carry the current end-effector orientation to the desired orientation. If both orientations coincide, then $\Delta R = I$.

For use in the impedance controller, ΔR is represented by a rotation angle ϕ and a unit rotation axis $u \in \mathbb{R}^3$ [19, 15]. The angle follows from

$$\mathrm{tr}(\Delta R) = 1 + 2\cos\phi, \tag{2.15}$$

and is therefore

$$\phi = \arccos\left(\frac{\mathrm{tr}(\Delta R) - 1}{2}\right), \qquad \phi \in [0, \pi]. \tag{2.16}$$

Rodrigues' formula expresses the relative rotation as

$$\Delta R = I + \sin\phi\,[u]_\times + (1 - \cos\phi)[u]_\times^2, \tag{2.17}$$

where

$$[u]_\times = \begin{bmatrix} 0 & -u_z & u_y \\ u_z & 0 & -u_x \\ -u_y & u_x & 0 \end{bmatrix}, \qquad [u]_\times^{\top} = -[u]_\times. \tag{2.18}$$

Subtracting the transpose of ΔR isolates its skew-symmetric part:

$$\frac{1}{2}\left(\Delta R - \Delta R^{\top}\right) = \sin\phi\,[u]_\times. \tag{2.19}$$

For $\sin\phi \neq 0$, the rotation axis is obtained from

$$u = \frac{1}{2\sin\phi}\begin{bmatrix}\Delta R_{32} - \Delta R_{23}\\ \Delta R_{13} - \Delta R_{31}\\ \Delta R_{21} - \Delta R_{12}\end{bmatrix}. \tag{2.20}$$

Multiplying the unit axis by the angle gives the orientation-error vector in the current end-effector frame. It is then rotated into the base frame:

$$e_{R,\mathrm{EE}} = \phi u, \qquad e_R = R_{\mathrm{EE}} e_{R,\mathrm{EE}} \in \mathbb{R}^3. \tag{2.21}$$

The direction of e_R defines the axis of the required correction, while its magnitude is the angular error in radians. When the current and desired orientations coincide, $\phi = 0$ and $e_R = 0$.

The complete Cartesian error used by the impedance controller is

$$e = \begin{bmatrix} e_p \\ e_R \end{bmatrix} \in \mathbb{R}^6. \tag{2.22}$$

2.3 Kinematics of a 7-DOF Manipulator

Having defined the end-effector pose as the controlled variable, its relation to the joint configuration is introduced next.

2.3.1 Forward Kinematics

Forward kinematics maps the joint configuration

$$q = \begin{bmatrix} q_1 & q_2 & \cdots & q_n \end{bmatrix}^\top \in \mathbb{R}^n \tag{2.23}$$

to the end-effector pose. Using the joint frames introduced in Section 2.1, the transformation from the base frame $\{0\}$ to the final link frame $\{n\}$ is

$${}^0T_n(q) = {}^0T_1(q_1)\,{}^1T_2(q_2)\cdots{}^{n-1}T_n(q_n). \tag{2.24}$$

In the standard Denavit–Hartenberg convention [4, 3, 19], joint i rotates by q_i about the axis z_{i-1}. The transformation between two consecutive joint frames is

$$^{i-1}T_i(q_i) = R_z(q_i)\,T_z(d_i)\,T_x(a_i)\,R_x(\alpha_i), \tag{2.25}$$

where a_i, d_i, and α_i describe the fixed geometry between the consecutive joint frames, while q_i is the revolute-joint coordinate.

The complete transformation gives the end-effector pose in the base frame:

$$^{0}T_n(q) = T_{\mathrm{EE}}(q) = \begin{bmatrix} R_{\mathrm{EE}}(q) & p_{\mathrm{EE}}(q) \\ 0^{\top} & 1 \end{bmatrix}. \tag{2.26}$$

This convention establishes the joint and frame indexing used in the subsequent Jacobian formulation. In the implemented controller, the forward-kinematic chain is not evaluated separately; the current end-effector pose is obtained from the libfranka model interface.

2.3.2 Differential Kinematics

Differential kinematics relates the joint velocities to the end-effector twist:

$$\dot{x} = \begin{bmatrix} \dot{p}_{\mathrm{EE}} \\ \omega_{\mathrm{EE}} \end{bmatrix} = J(q)\dot{q}, \tag{2.27}$$

where $\dot{p}_{\mathrm{EE}} \in \mathbb{R}^3$ and $\omega_{\mathrm{EE}} \in \mathbb{R}^3$ are the linear and angular velocities of the end-effector, respectively. The joint-velocity vector is

$$\dot{q} = \begin{bmatrix} \dot{q}_1 & \dot{q}_2 & \cdots & \dot{q}_n \end{bmatrix}^{\top} \in \mathbb{R}^n, \tag{2.28}$$

and $J(q) \in \mathbb{R}^{6\times n}$ is the geometric Jacobian. For the 7-DOF Panda, $n = 7$. The end-effector twist and the Jacobian used in the controller are expressed in the base frame. This relation provides the Cartesian velocity required by the damping term of the impedance controller.

2.3.3 Geometric Jacobian

The geometric Jacobian is written column-wise as

$$J(q) = \begin{bmatrix} J_1 & J_2 & \cdots & J_n \end{bmatrix} \in \mathbb{R}^{6\times n}, \tag{2.29}$$

where J_i describes the contribution of joint i to the end-effector twist. Since all joints of the Panda are revolute, each column is given by

$$J_i = \begin{bmatrix} z_{i-1} \times (p_{\mathrm{EE}} - p_{i-1}) \\ z_{i-1} \end{bmatrix} \in \mathbb{R}^6, \tag{2.30}$$

where z_{i-1} is the axis of joint i, and p_{i-1} is a point on that axis. Both are expressed in the base frame. The upper block describes the linear velocity generated at the end-effector, while the lower block describes the corresponding angular velocity [19, 3].

Because the joint axes and their positions depend on the current configuration, the Jacobian also depends on q. In the implemented controller, $J(q)$ is obtained directly from the libfranka model interface.

The Jacobian describes the local relation between joint velocities and the end-effector twist. For a robot with n joints it is written column-wise,

$$\begin{bmatrix} \dot{p}_{\mathrm{EE}} \\ \omega_{\mathrm{EE}} \end{bmatrix} = J(q)\dot{q}, \qquad J(q) = \begin{bmatrix} J_1 & J_2 & \cdots & J_n \end{bmatrix} \in \mathbb{R}^{6\times n}, \tag{2.31}$$

where each column J_i is the contribution of joint i. The Panda has only revolute joints, so with z_{i-1} the unit vector along the axis of joint i and p_{i-1} a point on that axis, both expressed in the base frame, the column is

$$J_i = \begin{bmatrix} z_{i-1} \times (p_{\mathrm{EE}} - p_{i-1}) \\ z_{i-1} \end{bmatrix} \in \mathbb{R}^6. \tag{2.32}$$

This standard form follows from the instantaneous velocity of a point under a revolute motion [19, 3]. The vector $p_{\mathrm{EE}} - p_{i-1}$ is the lever arm from the joint axis to the end-effector, so the cross product $z_{i-1} \times (p_{\mathrm{EE}} - p_{i-1})$ is the linear velocity that rotation of joint

i produces at the end-effector, while the lower block z_{i-1} is the corresponding angular velocity. Each column therefore stacks a linear contribution above an angular one.

Because z_{i-1}, p_{i-1} and p_{EE} all depend on the current joint configuration, the Jacobian is configuration dependent, and the influence of each joint on the end-effector motion changes as the robot moves.

2.4 Cartesian Impedance Control

The geometric Jacobian is also used through its transpose to map a Cartesian wrench to joint torques [12]. Before introducing this mapping, the desired Cartesian interaction behaviour is defined.

In this thesis, Cartesian impedance control is formulated as a spring–damper relation between the end-effector motion and the commanded wrench [10, 17, 14]. The Cartesian pose error e, defined in Section 2.2.4, produces a restoring wrench through the stiffness matrix K, while the Cartesian velocity produces an opposing wrench through the damping matrix D. Their entries determine the translational and rotational compliance in the selected task directions [18].

2.4.1 Impedance Wrench

The impedance wrench is evaluated in the robot base frame. The corresponding translational stiffness and damping matrices are $K_{p,0}, D_{p,0} \in \mathbb{R}^{3\times3}$, while the rotational matrices are $K_{R,0}, D_{R,0} \in \mathbb{R}^{3\times3}$. These matrices are not generally diagonal in the base frame because the direction-dependent gains are defined relative to the task frame. Their base-frame representation therefore changes with the task-frame orientation, even when the local gain values remain unchanged.

The commanded Cartesian force is given by the translational spring–damper law [10, 14]

$$f = K_{p,0}e_p + D_{p,0}\left(\dot{p}_d - \dot{p}_{\mathrm{EE}}\right) \in \mathbb{R}^3, \tag{2.33}$$

where $\dot{p}_d$ and $\dot{p}_{\mathrm{EE}}$ are the desired and measured linear velocities expressed in the base frame. For a fixed position setpoint, $\dot{p}_d = \mathbf{0}$, and the damping term becomes $-D_{p,0}\dot{p}_{\mathrm{EE}}$.

The commanded Cartesian moment is

$$m = K_{R,0} e_R - D_{R,0} \omega_{\mathrm{EE}} \in \mathbb{R}^3, \tag{2.34}$$

where ω_{EE} is the measured end-effector angular velocity expressed in the base frame. Since no angular reference velocity is commanded, the rotational damping acts directly against ω_{EE}.

The force and moment are combined into the Cartesian wrench

$$F = \begin{bmatrix} f \\ m \end{bmatrix} = \begin{bmatrix} K_{p,0} e_p + D_{p,0} \left(\dot{p}_d - \dot{p}_{\mathrm{EE}} \right) \\ K_{R,0} e_R - D_{R,0} \omega_{\mathrm{EE}} \end{bmatrix} \in \mathbb{R}^6. \tag{2.35}$$

The wrench is ordered as force followed by moment, and all its components are expressed in the base frame. It is mapped to joint torques through the Jacobian transpose in the following subsection.

2.4.2 Joint-Torque Control Law

By the principle of virtual work, the commanded Cartesian wrench is mapped to joint torques through the transpose of the geometric Jacobian [12]:

$$\tau_{\mathrm{cart}} = J^\top(q) F, \tag{2.36}$$

where $\tau_{\mathrm{cart}} \in \mathbb{R}^n$ is the Cartesian impedance torque contribution and $J^\top(q) \in \mathbb{R}^{n \times 6}$. Substituting eq. (2.35) gives

$$\tau_{\mathrm{cart}} = J^\top(q) \begin{bmatrix} K_{p,0} e_p + D_{p,0} \left(\dot{p}_d - \dot{p}_{\mathrm{EE}} \right) \\ K_{R,0} e_R - D_{R,0} \omega_{\mathrm{EE}} \end{bmatrix}. \tag{2.37}$$

The gain matrices in this expression are the base-frame matrices obtained from the selected task-frame gains. A change in the surface-frame orientation therefore changes the Cartesian wrench and the resulting joint-torque contribution, even when the local gain values remain unchanged.

2.4.3 Coriolis and Gravity Compensation in libfranka

In a general model-compensated torque controller, the Cartesian impedance torque is combined with gravity and Coriolis/centrifugal compensation:

$$\tau_{\text{cmd}} = \tau_{\text{cart}} + \tau_g(q) + \tau_c(q, \dot{q}), \tag{2.38}$$

where $\tau_g(q)$ is the gravity torque and $\tau_c(q, \dot{q})$ is the Coriolis and centrifugal torque.

These terms are handled differently by the Franka Control Interface. The Coriolis vector provided by the libfranka model is added explicitly to the command because it is not compensated internally. A separate gravity term is not added. In external joint-torque mode, the robot compensates gravity and motor friction internally, and the commanded torque is interpreted as excluding gravity [7]. Adding $\tau_g(q)$ would therefore compensate gravity twice.

The implemented torque command is consequently

$$\tau_{\text{cmd,impl}} = \tau_{\text{cart}} + \tau_c(q, \dot{q}) = J^\top(q)F + \tau_c(q, \dot{q}), \tag{2.39}$$

which follows the official libfranka Cartesian impedance example [8]. The null-space torque introduced later is added to this expression to obtain the complete commanded torque in eq. (2.92).

2.5 Surface Task Frame

The contact experiment uses a task frame tied to the surface geometry. This frame is defined before the stiffness and damping matrices are introduced, so that the local gain entries can later be interpreted as normal and tangential values instead of as abstract base-frame directions.

Let $n_s \in \mathbb{R}^3$ with $\|n_s\| = 1$ be the unit surface normal, expressed in the robot base frame. To complete the frame, choose an auxiliary direction $a_s \in \mathbb{R}^3$ that is not parallel to n_s. This auxiliary direction is only used to define a repeatable tangent direction on the surface. The first tangent vector is obtained by projecting a_s onto the plane orthogonal to n_s,

and the second tangent vector is then obtained by a cross product. With $\tilde{t}_1$ denoting the unnormalised projected tangent, the construction is a Gram–Schmidt orthonormalization step applied to the surface normal and the auxiliary direction [19, 3]:

$$\begin{aligned} \tilde{t}_1 &= a_s - n_s \left(n_s^\top a_s \right), \\ t_1 &= \frac{\tilde{t}_1}{\|\tilde{t}_1\|}, \\ t_2 &= n_s \times t_1. \end{aligned} \tag{2.40}$$

Here, t_1 and t_2 are unit tangent vectors lying in the surface plane. The construction is valid as long as $\|\tilde{t}_1\| \neq 0$, which means that the auxiliary direction a_s must not be parallel to the normal n_s. To match the implemented controller and its parameter order, the surface task frame collects the two tangents first and the normal last,

$$R_{\text{surface}} = \begin{bmatrix} t_1 & t_2 & n_s \end{bmatrix}. \tag{2.41}$$

The column order is fixed: every surface-frame three-vector in the implementation uses $[\,\text{tangent 1}, \text{tangent 2}, \text{normal}\,]^\top$. In the gain representation below, the contact task uses this frame by setting $R_{\text{task}} = R_{\text{surface}}$. This makes stiffness values such as K_{t_1}, K_{t_2}, and K_n local surface-frame quantities in that same order.

2.6 Stiffness and Damping Representation

Direction-dependent stiffness and damping gains are most conveniently defined in a task frame whose axes coincide with the desired principal compliance directions. The corresponding matrices are diagonal in that frame but are not generally diagonal when expressed in the robot base frame.

Let $R_{\text{task}} \in SO(3)$ denote the orientation of the task frame relative to the base frame. Its columns contain the task-frame axes expressed in the base frame. The translational and rotational gain matrices used in the impedance wrench are obtained from

$$\begin{aligned} K_{p,0} &= R_{\text{task}} K_{p,\text{task}} R_{\text{task}}^\top, \quad & D_{p,0} &= R_{\text{task}} D_{p,\text{task}} R_{\text{task}}^\top, \\ K_{R,0} &= R_{\text{task}} K_{R,\text{task}} R_{\text{task}}^\top, \quad & D_{R,0} &= R_{\text{task}} D_{R,\text{task}} R_{\text{task}}^\top. \end{aligned} \tag{2.42}$$

For an impedance without translation–rotation coupling in the task frame, the full stiffness and damping matrices are

$$K_{\text{task}} = \begin{bmatrix} K_{p,\text{task}} & 0 \\ 0 & K_{R,\text{task}} \end{bmatrix}, \qquad D_{\text{task}} = \begin{bmatrix} D_{p,\text{task}} & 0 \\ 0 & D_{R,\text{task}} \end{bmatrix}. \tag{2.43}$$

Defining

$$T_R = \text{diag}\left(R_{\text{task}}, R_{\text{task}}\right), \tag{2.44}$$

their base-frame representations are

$$K_0 = T_R K_{\text{task}} T_R^\top, \qquad D_0 = T_R D_{\text{task}} T_R^\top. \tag{2.45}$$

For the contact task, the task frame is the surface frame, whose orientation is

$$R_{\text{task}} = R_{\text{surface}} = \begin{bmatrix} t_1 & t_2 & n_s \end{bmatrix}. \tag{2.46}$$

For example, the local translational stiffness is defined as

$$K_{p,\text{task}} = \text{diag}\left(K_{t_1}, K_{t_2}, K_n\right), \tag{2.47}$$

and its base-frame representation is

$$K_{p,0} = R_{\text{surface}}\ \text{diag}\left(K_{t_1}, K_{t_2}, K_n\right) R_{\text{surface}}^\top. \tag{2.48}$$

The rotational stiffness follows the same axis order,

$$K_{R,\text{task}} = \text{diag}\left(K_{r,t_1}, K_{r,t_2}, K_{r,n}\right), \tag{2.49}$$

and the damping matrices are constructed analogously. A change in the configured surface orientation therefore changes the corresponding base-frame gain matrices even when the local gain values remain unchanged.

During the approach phase, this surface-frame construction is applied to both the transla-

tional and rotational gain blocks. During contact set-up and grinding, only the rotational stiffness and damping are constructed in the surface frame; the translational matrices remain diagonal in the base-frame directions $[x, y, z]$, as described in Chapter 3. The active gain matrices are based on the configured surface frame and are not rotated with the current end-effector orientation R_{EE}.

2.7 Centre of Compliance

The task-frame transformation of eq. (2.45) rotates the principal stiffness directions while leaving the impedance reference point at the end-effector origin, identified here with the tool centre point (TCP). Independently of this orientation choice, the virtual spring can be centred at a different point. Shifting the impedance reference from the TCP to a centre of compliance p_c transforms a block-diagonal impedance at p_c into a coupled impedance at the TCP [15, 6, 20, 13]. This construction is applied during contact set-up to generate translation–rotation coupling that supports tool alignment.

The lever from the centre of compliance to the TCP, expressed in the base frame, is

$$r_c = p_{\mathrm{TCP}} - p_c \in \mathbb{R}^3, \qquad p_{\mathrm{TCP}} = p_{\mathrm{EE}}. \tag{2.50}$$

Let $[r_c]_\times$ denote the skew-symmetric matrix satisfying $[r_c]_\times a = r_c \times a$. Under the small-angle approximation, the position error at the centre of compliance is

$$e_{p,c} = e_p + [r_c]_\times e_R. \tag{2.51}$$

The positive sign follows from the selected lever convention. The vector from the TCP to the centre of compliance is $-r_c$, and therefore

$$e_R \times (-r_c) = [r_c]_\times e_R.$$

The translational and rotational errors at the centre of compliance can thus be written as

$$e_c = \begin{bmatrix} e_{p,c} \\ e_R \end{bmatrix} = \mathrm{Ad}(r_c) \begin{bmatrix} e_p \\ e_R \end{bmatrix}, \qquad \mathrm{Ad}(r_c) = \begin{bmatrix} \mathbf{I}_3 & [r_c]_\times \\ 0 & \mathbf{I}_3 \end{bmatrix}. \tag{2.52}$$

The point-shift adjoint $\mathrm{Ad}(r_c)$ changes the reference point of the impedance. By contrast, $T_R = \mathrm{diag}(R_{\mathrm{task}}, R_{\mathrm{task}})$ in eq. (2.45) changes its principal directions.

After any required task-frame rotation, let the decoupled stiffness and damping matrices defined at the centre of compliance be

$$K_c = \begin{bmatrix} K_p & 0 \\ 0 & K_R \end{bmatrix}, \qquad D_c = \begin{bmatrix} D_p & 0 \\ 0 & D_R \end{bmatrix}. \tag{2.53}$$

All blocks in eq. (2.53) are expressed in the base-frame orientation. The wrench at the centre of compliance is

$$w_c = K_c e_c.$$

Transforming this power-conjugate wrench to the TCP gives

$$w_{\mathrm{TCP}} = \mathrm{Ad}(r_c)^\top w_c.$$

The resulting TCP stiffness is therefore

$$K_{\mathrm{TCP}} = \mathrm{Ad}(r_c)^\top K_c \mathrm{Ad}(r_c) = \begin{bmatrix} K_p & K_p[r_c]_\times \\ [r_c]_\times^\top K_p & K_R + [r_c]_\times^\top K_p [r_c]_\times \end{bmatrix}. \tag{2.54}$$

The same point shift is applied to the damping matrix:

$$D_{\mathrm{TCP}} = \mathrm{Ad}(r_c)^\top D_c \mathrm{Ad}(r_c) = \begin{bmatrix} D_p & D_p[r_c]_\times \\ [r_c]_\times^\top D_p & D_R + [r_c]_\times^\top D_p [r_c]_\times \end{bmatrix}. \tag{2.55}$$

The off-diagonal blocks introduce translation–rotation coupling. A translational error can therefore generate a restoring moment, while a rotational error can generate a restoring force. This coupling is absent from the block-diagonal impedance defined directly at the TCP.

The sign of the translation-to-moment coupling can be checked by setting $e_R = 0$. With

$$f = K_p e_p,$$

the corresponding elastic moment at the TCP is

$$m_c = [r_c]_\times^\top f = -r_c \times f. \tag{2.56}$$

Reversing the direction of r_c therefore reverses the generated coupling moment. This relation provides a direct sign check for the selected centre of compliance and pressing direction.

The coupling vanishes for $r_c = 0$. Conversely, if K_p is nonsingular, zero coupling implies $r_c = 0$. Since $\mathrm{Ad}(r_c)$ is invertible, the congruence transformation also preserves definiteness:

$$K_c \succeq 0 \implies K_{\mathrm{TCP}} \succeq 0, \qquad K_c \succ 0 \implies K_{\mathrm{TCP}} \succ 0. \tag{2.57}$$

The same statements apply to D_c and D_{TCP}. Negative off-diagonal entries therefore represent coupling terms rather than negative stiffness directions.

This formulation is a local small-motion model in which r_c is held fixed while the wrench is evaluated. In the implemented set-up mode, the selected lever remains constant, and the mapped gain matrices are therefore constant during the corresponding motion segment. If p_c or r_c were updated continuously, the gain matrices would become configuration dependent.

The selected point p_c is consequently a local linearised centre of compliance rather than a measured physical pivot. It need not coincide with the physical contact point p_C. The observed motion also depends on the finite rotational stiffness, the lever direction, the environmental constraint, friction, and the additional active controller terms introduced later.

2.8 Null-Space Torque

The Franka Emika Panda has seven joints, while the controlled end-effector task contains six components: three translational and three rotational. The robot is therefore kinematically redundant. The Cartesian impedance behaviour at the end-effector is the primary objective, and the remaining joint-space freedom can be used for a secondary objective.

In this thesis, the secondary joint-space action consists of a null-space damping contribution and an active singular-value conditioning contribution. Both are projected using the joint-torque null-space projector N_τ. The required null-space direction, pseudoinverse, and projector are introduced before the two torque contributions are defined.

2.8.1 SVD-Based Null-Space Characterisation

The geometric Jacobian relates the current joint velocity $\dot{q}$ to the end-effector velocity $\dot{x}$:

$$\dot{x} = J(q)\dot{q}, \qquad J(q) \in \mathbb{R}^{6\times 7}, \tag{2.58}$$

where

$$\dot{x} = \begin{bmatrix} \dot{p}_{\mathrm{EE}} \\ \omega_{\mathrm{EE}} \end{bmatrix} \in \mathbb{R}^6 \tag{2.59}$$

contains the three linear and three angular velocity components of the end-effector.

A joint velocity $\dot{q}$ belongs to the kinematic null space when

$$J(q)\dot{q} = 0. \tag{2.60}$$

Such a joint velocity changes the robot configuration while preserving the end-effector position and orientation to first order. The Jacobian is the first-order term in the expansion of the forward kinematics about the current configuration, so a finite displacement along the null space leaves a pose error of second order in its magnitude. When $J(q)$ has full row rank, the null space of the 6×7 Jacobian is one-dimensional.

The singular value decomposition (SVD) is applied to the Jacobian at the current joint

configuration q [2, 5]:

$$J(q) = U_J \Sigma_J V_J^\top, \qquad U_J \in \mathbb{R}^{6\times 6}, \quad \Sigma_J \in \mathbb{R}^{6\times 7}, \quad V_J \in \mathbb{R}^{7\times 7}. \tag{2.61}$$

The Jacobian $J(q)$ is the input to the decomposition. The SVD returns the matrices U_J, Σ_J, and V_J, with the singular values contained in Σ_J.

The columns of V_J form an orthonormal basis of the seven-dimensional joint-velocity space. The joint velocity $\dot{q}$ can therefore be expressed as

$$\dot{q} = V_J a = \sum_{i=1}^{7} a_i v_i, \tag{2.62}$$

where v_i is the i-th column of V_J, and $a \in \mathbb{R}^7$ contains the components of $\dot{q}$ along these basis directions.

Similarly, the columns of U_J form an orthonormal basis of the six-dimensional end-effector-velocity space. Substituting $\dot{q} = V_J a$ into eq. (2.58) gives

$$\dot{x} = J(q) V_J a = U_J \Sigma_J a. \tag{2.63}$$

Thus, V_J provides the basis directions in which the joint velocity $\dot{q}$ is expressed, while U_J provides the corresponding basis directions in which the end-effector velocity $\dot{x}$ is expressed. The matrix Σ_J scales the mapping between these directions.

For the 6×7 Jacobian, the singular-value matrix has the form

$$\Sigma_J = \begin{bmatrix} \operatorname{diag}(\sigma_1, \sigma_2, \ldots, \sigma_6) & 0_{6\times 1} \end{bmatrix}, \tag{2.64}$$

where

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_6 \geq 0. \tag{2.65}$$

For the first six basis directions,

$$J v_i = \sigma_i u_i, \qquad i = 1, \ldots, 6, \tag{2.66}$$

where u_i is the i-th column of U_J. A joint velocity in the direction v_i,

$$\dot{q} = a_i v_i, \tag{2.67}$$

therefore produces the end-effector velocity

$$\dot{x} = a_i \sigma_i u_i. \tag{2.68}$$

The singular value σ_i scales the mapping from the joint-velocity basis direction v_i to the corresponding end-effector-velocity basis direction u_i.

The seventh column of V_J corresponds to the zero column of Σ_J and therefore satisfies

$$Jv_7 = 0. \tag{2.69}$$

A joint velocity of the form

$$\dot{q} = a_7 v_7 \tag{2.70}$$

therefore changes the joint configuration without producing an end-effector velocity. When the Jacobian has full row rank, v_7 spans its one-dimensional kinematic null space.

The smallest of the six task-producing singular values is

$$\sigma_{\min} = \sigma_6. \tag{2.71}$$

The quantities v_7 and σ_6 have different purposes. The vector v_7 represents the redundant joint-motion direction, whereas σ_6 describes the weakest of the six mappings from joint velocity to end-effector velocity. As σ_6 approaches zero, one end-effector velocity direction becomes increasingly difficult to produce, indicating an approach to a loss of Jacobian row rank. In this thesis, $\sigma_{\min}$ is used to compare the conditioning of nearby joint configurations and to select the direction of the conditioning torque.

2.8.2 SVD-Based Pseudoinverse

The Jacobian pseudoinverse maps end-effector-velocity components back to the corresponding joint-velocity directions. Its calculation uses the singular directions obtained from the SVD.

Small singular values lead to large reciprocal values and can amplify numerical errors. A relative singular-value threshold is therefore applied. Let $\rho \geq 0$ denote the configured relative SVD tolerance. The corresponding absolute threshold is calculated from the largest singular value σ_1:

$$\sigma_{\text{thr}} = \rho\sigma_1. \tag{2.72}$$

A singular direction is retained when

$$\sigma_i > \sigma_{\text{thr}}. \tag{2.73}$$

The pseudoinverse is defined as

$$J^+ = \sum_{\substack{i=1 \\ \sigma_i > \sigma_{\text{thr}}}}^{6} \frac{1}{\sigma_i} v_i u_i^\top \in \mathbb{R}^{7\times 6}. \tag{2.74}$$

For each retained singular direction, J^+ maps an end-effector-velocity component along u_i to the corresponding joint-velocity direction v_i and scales it by $1/\sigma_i$. Directions at or below σ_{thr} receive a reciprocal value of zero.

2.8.3 Null-Space Projectors

The pseudoinverse is used to separate a joint-space vector into a task-related component and a null-space component. For an arbitrary joint velocity $\dot{q}$, this decomposition is

$$\dot{q} = \underbrace{J^+ J\dot{q}}_{\text{task-related component}} + \underbrace{\left(I_7 - J^+ J\right)\dot{q}}_{\text{null-space component}}. \tag{2.75}$$

The product J^+J projects $\dot{q}$ onto the joint-space directions that contribute to the end-effector velocity. Since J includes all three linear and all three angular velocity components,

this task-related component accounts for the complete controlled end-effector pose at the velocity level.

The complementary joint-velocity projector is

$$N_q = I_7 - J^+ J \in \mathbb{R}^{7\times 7}. \tag{2.76}$$

Applying N_q to an arbitrary joint velocity retains its null-space component:

$$\dot{q}_{\text{null}} = N_q \dot{q}. \tag{2.77}$$

When J has full row rank and all six task-producing singular directions are retained,

$$J\dot{q}_{\text{null}} = J N_q \dot{q} = 0. \tag{2.78}$$

The projected joint velocity can therefore change the robot configuration while preserving the end-effector position and orientation to first order.

A corresponding decomposition is used for a secondary joint torque τ_0. The joint-torque projector is the transpose of the velocity projector:

$$N_\tau = N_q^\top = I_7 - J^\top J^{+\top} \in \mathbb{R}^{7\times 7}. \tag{2.79}$$

The secondary torque can then be written as

$$\tau_0 = \underbrace{J^\top J^{+\top} \tau_0}_{\text{task-related component}} + \underbrace{N_\tau \tau_0}_{\text{null-space component}}. \tag{2.80}$$

The first component lies in the joint-torque subspace associated with Cartesian wrenches through $J^\top$. The second component is retained as the secondary null-space torque:

$$\tau_{\text{null}} = N_\tau \tau_0. \tag{2.81}$$

The transpose relation follows from the mechanical power $\tau^\top \dot{q}$ between joint torques and joint velocities. Thus, N_q is applied to joint velocities, while N_τ is applied to secondary joint torques.

For the SVD-based pseudoinverse, J^+J is an orthogonal and symmetric projector. Hence,

$$N_q = N_\tau \tag{2.82}$$

holds analytically. The separate symbols distinguish the quantity on which the projector acts. In the implementation, N_τ is additionally symmetrised to reduce floating-point asymmetry.

When J has full row rank and all six task-producing singular directions are retained,

$$N_q = N_\tau = v_7 v_7^\top. \tag{2.83}$$

In this case, projection retains only the component along the redundant direction v_7. When a singular value lies at or below the selected threshold, its associated right-singular direction is also assigned to the numerical null space.

2.8.4 Null-Space Damping Torque

The current joint velocity is separated into task-related and null-space components according to eq. (2.75). The damping contribution acts on the null-space component $N_q\dot{q}$:

$$\tau_d = -d_{\text{null}} N_\tau \dot{q} = -d_{\text{null}} N_q \dot{q}, \tag{2.84}$$

where $d_{\text{null}} \geq 0$ is the null-space damping coefficient. The negative sign makes the torque oppose motion along the redundant joint-space direction and dissipate the associated motion.

2.8.5 Singular-Value Conditioning Torque

The second contribution applies an active torque along the redundant direction v_7. Its sign is selected by comparing $\sigma_{\min}$ at two nearby joint configurations.

Starting from the current joint configuration q, the sampled configurations are

$$q_+ = q + \alpha_{\text{probe}} v_7, \qquad q_- = q - \alpha_{\text{probe}} v_7. \tag{2.85}$$

The Jacobian is recalculated at both configurations. The corresponding smallest singular values are

$$\sigma_+ = \sigma_{\min}(J(q_+)) \,, \qquad \sigma_- = \sigma_{\min}(J(q_-)) \,. \tag{2.86}$$

Their difference is

$$\Delta_\sigma = \sigma_+ - \sigma_-. \tag{2.87}$$

The direction selector is defined as

$$s = \begin{cases} +1, & \Delta_\sigma > \delta_\sigma, \\ -1, & \Delta_\sigma < -\delta_\sigma, \\ 0, & |\Delta_\sigma| \leq \delta_\sigma, \end{cases} \tag{2.88}$$

where $\delta_\sigma \geq 0$ is the comparison deadband. Within this deadband, the selector is set to zero and the conditioning contribution is suppressed.

The selected direction is invariant to the sign convention of v_7 returned by the SVD. Reversing v_7 exchanges q_+ and q_- and reverses s, so the product sv_7 remains unchanged.

The probe angle α_{probe} defines the joint-space distance between the current configuration and the two sampled configurations. The magnitude of the resulting torque is specified independently by k_σ.

The vector v_7 represents the instantaneous null-space direction at the current configuration. For a finite probe angle, q_+ and q_- provide local approximations of the nonlinear constant-pose self-motion. The corresponding preservation of the end-effector pose is therefore a first-order property of the sampled direction.

The selector uses the sign of Δ_σ to choose the sampled direction with the larger value of $\sigma_{\min}$. This produces a two-point directional conditioning strategy based on local comparison rather than on the magnitude of the sampled variation.

The vector sv_7 defines the selected secondary joint-space direction. Projecting it with N_τ retains its component in the numerical joint-torque null space:

$$\tau_\sigma = k_\sigma N_\tau \left(s v_7\right), \tag{2.89}$$

where $k_\sigma \geq 0$ specifies the commanded torque magnitude.

When the Jacobian has full row rank and all six task-producing singular directions are retained, v_7 lies in the one-dimensional null space and

$$N_\tau v_7 = v_7. \tag{2.90}$$

The conditioning contribution therefore biases the robot along the selected redundant joint direction, while the damping contribution opposes motion within the projected subspace.

2.8.6 Complete Null-Space Torque

Combining the damping and conditioning contributions gives

$$\tau_{\text{null}} = \tau_d + \tau_\sigma = -d_{\text{null}} N_\tau \dot{q} + k_\sigma N_\tau \left(s v_7\right) \in \mathbb{R}^7. \tag{2.91}$$

When the Jacobian has full row rank, its null space is one-dimensional and is spanned by v_7. If the Jacobian loses row rank, the null space becomes multidimensional. In that case, the implemented conditioning term continues to probe the directions $+v_7$ and $-v_7$. A more general treatment would require consideration of the additional right singular vectors associated with the enlarged null space. This extension is outside the scope of the present work. The damping contribution, by contrast, acts on the joint-motion directions contained in the numerical null space through $N_\tau \dot{q}$.

The activation of the damping and conditioning contributions is a controller configuration choice. The available combinations and the configuration used in the reported experiments are described in section 3.4.

2.9 Complete Joint-Torque Command

The complete joint-torque command combines the Cartesian impedance torque, the null-space torque, and the Coriolis compensation:

$$\tau_{\text{cmd}} = \tau_{\text{cart}} + \tau_{\text{null}} + \tau_c(q, \dot{q}) = J^\top(q)F + \tau_{\text{null}} + \tau_c(q, \dot{q}). \tag{2.92}$$

Here, $\tau_{\text{cart}} = J^\top(q)F$ is the Cartesian impedance torque, τ_{null} is the secondary null-space torque, and $\tau_c(q, \dot{q})$ is the Coriolis and centrifugal compensation obtained from the robot model and added in the control callback. A separate gravity term is not added because gravity compensation is handled internally by the Franka Control Interface, as described in Section 2.4.3.

Chapter 3

Controller Design and Implementation

PURPLE — revised and confirmed. The chapter was revised in full and every paragraph is settled. No further editing is to be applied to it, and any remaining concern is raised as a comment rather than as a change.

This chapter describes the real-time implementation of the Cartesian impedance controller developed in Chapter 2. The controller is written in C++ using libfranka and Eigen and is executed through the Franka Control Interface (FCI) with a nominal control period of 1 ms.

The software combines the Cartesian impedance law, the SVD-based null-space controller, the configured surface and tool geometry, phase-dependent stiffness and damping, and the virtual centre of compliance. These components are coordinated by a phase state machine that orients the tool, approaches the surface, applies the set-up preload, and executes the grinding motion. Additional operating modes provide Cartesian pose holding, set-up-impedance holding, and manual guidance.

The following sections describe the software architecture, the real-time control loop, the implemented Cartesian and null-space torque contributions, the geometric representation of the surface and tool, the phase-dependent impedance parameters, and the execution of the control sequence.

3.1 Software Architecture and Program Execution

The controller is divided into modules for configuration handling, control calculations, real-time execution, operator interaction, and run evaluation. The main program coordinates these modules and manages the execution of successive controller sessions.

Table 3.1: Software modules of the surface-grinding controller.

Module	Responsibility
`main.cpp`	Connects to the robot, prepares the controller session, starts the selected operating mode, and initiates post-run reporting.
`include/`	Defines the controller configuration, mathematical types, run-time data structures, and interfaces shared between the software modules.
`src/config/`	Reads the parameter files and assigns their values to the typed controller configuration.
`src/control/`	Implements the Cartesian impedance law, null-space torque, phase-dependent gain construction, surface and tool geometry, inertia-scaled damping, and robot support functions.
`src/runtime/`	Executes the phase state machine and the 1 kHz joint-torque control callback.
`src/interface/`	Provides the start-up menu, gripper and tool-handling actions, manual guidance, live parameter commands, and terminal output.
`src/report/`	Stores and writes the recorded controller signals and evaluates the completed run.
`tools/` `measure_plane.cpp`	Provides a separate calibration utility for estimating one point and the orientation of the physical surface plane.

The parameter directory is resolved relative to the controller executable. The complete configuration is distributed over 16 files in the `params/` directory. These files contain the robot connection and logging settings, collision thresholds, gripper parameters, initial

posture, surface and tool geometry, phase settings, Cartesian gains, inertia-scaled-damping settings, virtual-compliance-centre parameters, null-space parameters, disturbance-test settings, and manual-guidance settings.

At program start, the parameter files are read to obtain the robot address and initial controller configuration. The robot connection is then established, automatic error recovery is requested, the selected collision behaviour is assigned, and the libfranka robot model is loaded. A separate keyboard thread is also started to receive operator requests during the controller session. These requests are transferred through atomic variables shared with the control loop.

Before every new session, all parameter files are read again and the resulting values are stored in a new `ControllerConfig` object. This permits controller and experiment parameters to be changed between runs without recompiling the program. The loader requires all 16 parameter files and rejects duplicate parameter names across them.

The start-up interface provides four controller modes:

- the complete tool-orientation, surface-approach, set-up, and grinding sequence;
- Cartesian pose holding with the dedicated hold impedance;
- Cartesian pose holding with the set-up impedance; and
- manual guidance from the current robot pose.

The same interface provides the robot-preparation and gripper actions required before torque control. The robot can be moved to the configured initial joint posture, the gripper can be opened or closed, and the tool can be collected from or returned to its configured holder posture. When manual guidance is selected, the subsequent sequence or hold mode begins from the pose captured after hand-guided positioning.

Once the operating mode and initial state have been selected, the phase-dependent stiffness and damping matrices are constructed from the active configuration. These matrices, together with the robot and model interfaces, are passed to the real-time control loop.

Experimental signals are stored during control in a bounded ring buffer whose memory is allocated before the torque callback begins. After the controller session has ended, the stored samples are arranged in chronological order, written to the configured CSV file, and used to generate the terminal run summary. A new session can then be started from

the start-up menu with a freshly loaded configuration.

3.2 Real-Time Control Loop

The function `runControlLoop` prepares one controller session and executes the libfranka joint-torque callback. It receives the active controller configuration, the robot and model interfaces, the phase-dependent impedance gains, and the operator-input signals.

Before torque control begins, the current robot state is read once. The reported end-effector transformation provides the initial TCP position p_{start} and orientation R_{start}, while the measured joint configuration provides the initial posture q_{start}. These values initialise the desired pose of the selected sequence or hold mode.

The initial state also provides force and moment bias values for the subsequent contact evaluation. In addition, the controller initialises the configured surface point, the surface frame, the target tool orientation, the descent direction, and the four tool-face corners. The initial phase, phase clocks, operator gates, set-up references, damping cache, and recording buffer are prepared before entering the real-time callback.

The controller is passed to `robot.control` as a joint-torque callback. For every control cycle, the callback receives the current `RobotState` and the elapsed cycle duration and returns the seven commanded joint torques, as shown in listing 3.1.

Listing 3.1: Registration of the joint-torque callback through `libfranka`.

```
robot.control(
    [&](const franka::RobotState& state,
        franka::Duration period) -> franka::Torques {

      // Real-time state and torque calculations

      return franka::Torques(tau_array);
    });
```

During each callback, the elapsed controller time is updated from the period reported by libfranka. The joint state, end-effector Jacobian, Cartesian velocity, end-effector pose, and model-estimated external wrench are then obtained as shown in listing 3.2.

Listing 3.2: Acquisition of the robot-state and model quantities at the beginning of one real-time torque callback.

```
time += period.toSec();

Map<const Vec7> dq(state.dq.data());
Map<const Vec7> q_current(state.q.data());
```

```
5
6  std::array<double, 42> jacobian_array =
7      model.zeroJacobian(Frame::kEndEffector, state);
8  Map<const Mat6x7> J(jacobian_array.data());
9
10 const Vec6 xdot = J * dq;
11 const Vec3 pdot = xdot.head<3>();
12 const Vec3 omega = xdot.tail<3>();
13
14 Map<const Mat4x4> T_EE(state.O_T_EE.data());
15 const Vec3 p_EE = T_EE.block<3, 1>(0, 3);
16 const Mat3 R_EE = T_EE.block<3, 3>(0, 0);
17
18 Map<const Vec6> external_wrench(
19     state.O_F_ext_hat_K.data());
20 const Vec3 external_force = external_wrench.head<3>();
21 const Vec3 external_moment = external_wrench.tail<3>();
```

The vectors `q_current` and `dq` contain the measured joint configuration and velocity. The call to `model.zeroJacobian` returns the 6×7 geometric Jacobian of the configured libfranka end-effector frame. Multiplication by the measured joint velocity provides the TCP linear velocity `pdot` and angular velocity `omega`, both expressed in the robot base frame.

The field `state.O_T_EE` provides the current end-effector transformation in the base frame. Its translational and rotational blocks are assigned to `p_EE` and `R_EE`, respectively. The controller therefore uses the pose reported by libfranka rather than reconstructing it from a separate forward-kinematic model.

The field `state.O_F_ext_hat_K` provides the model-estimated external force and moment. During surface approach, the transition to set-up is determined geometrically from the tool-face clearance. At this transition, the current force and moment estimates are stored as bias values. During set-up, the bias-subtracted external moment is used for the moment-based exit condition. The force and moment estimates are also included in the terminal output, set-up evaluation, and recorded experimental data.

Operator requests are exchanged with the callback through atomic variables. They support transitions between the phase sequence, Cartesian pose hold, set-up-impedance hold, and manual guidance. The same interface receives the gate confirmations and stop request. During the dedicated hold modes, selected null-space and set-up-impedance parameters can also be adjusted. An accepted set-up-impedance change rebuilds the corresponding phase gains and damping cache.

The phase state machine updates the desired TCP position p_d, desired linear velocity $\dot{p}_d$, and desired orientation R_d. Their values depend on the active phase:

- tool orientation maintains the initial TCP position while advancing the orientation command toward the configured target;
- surface approach moves the desired position along the descent direction;
- set-up advances the selected tool feature into the surface;
- grinding retains the achieved preload and can add a tangential trajectory; and
- pose hold regulates the captured Cartesian pose.

After the reference has been updated, the callback calculates the Cartesian position and orientation errors using the conventions introduced in Section 2.2.4. The phase-dependent stiffness and damping matrices are then selected. When a phase gate is active, the dedicated gate holding gains replace the normal phase gains.

The displacement and velocity-error vectors supplied to the impedance calculation are

$$\Delta x = \begin{bmatrix} e_p \\ e_R \end{bmatrix}, \qquad \Delta v = \begin{bmatrix} \dot{p}_d - \dot{p}_{\mathrm{EE}} \\ -\omega_{\mathrm{EE}} \end{bmatrix}. \tag{3.1}$$

The Cartesian impedance calculation returns the commanded wrench

$$F = \begin{bmatrix} f \\ m \end{bmatrix} \in \mathbb{R}^6. \tag{3.2}$$

Depending on the active mode, this wrench is obtained from either the decoupled impedance law of eq. (2.37) or the point-shifted set-up impedance described in Section 3.7.

The Cartesian torque contribution is

$$\tau_{\mathrm{cart}} = J^{\top}(q)F. \tag{3.3}$$

The SVD-based null-space routine calculates the secondary torque τ_{null} according to the selected null-space mode. During the automatic disturbance experiment in Cartesian pose hold, an additional joint-torque contribution τ_{dist} is generated. Its value is zero during the remaining controller operation.

The Coriolis and centrifugal torque is obtained from the libfranka model. The complete torque command of an impedance-controlled cycle is therefore

$$\tau_{\text{cmd}} = \tau_{\text{cart}} + \tau_{\text{null}} + \tau_{\text{dist}} + \tau_c(q, \dot{q}). \tag{3.4}$$

Manual guidance follows a separate torque command:

$$\tau_{\text{guide}} = \tau_c(q, \dot{q}) - d_{\text{guide}}\dot{q}, \tag{3.5}$$

where d_{guide} is the configured joint-damping coefficient. After the guided pose is captured, its position, orientation, joint configuration, and wrench biases initialise the resumed sequence or hold mode.

For impedance-controlled cycles, the selected state, reference, error, wrench, phase, null-space, disturbance, and joint-torque quantities are sampled at the configured interval and copied into a preallocated ring buffer. When the buffer capacity is reached, the most recent samples replace the oldest entries.

The callback ends when the configured run duration expires, an operator stop is requested, the maximum approach distance is reached, or libfranka terminates the motion. After torque control has ended, the ring-buffer entries are placed in chronological order and returned together with the final controller state. File writing and post-run evaluation are performed outside the real-time callback.

3.3 Implemented Cartesian Impedance Controller

The mathematical formulation of the Cartesian impedance controller is given in Section 2.4. This section describes how the pose errors, phase-dependent gains, and Cartesian wrench are evaluated in the software.

The callback first selects the desired orientation associated with the active phase. During tool orientation, the desired orientation follows the interpolated orientation command. Surface approach uses the final tool-target orientation. At the transition to set-up, the current orientation is captured and retained throughout set-up and grinding. Cartesian pose hold uses the pose stored for the active hold mode.

The position and orientation errors are evaluated as shown in listing 3.3.

Listing 3.3: Calculation of the phase-dependent Cartesian pose error.

```
const Vec3 e_p = desired.p_d - p_EE;

const Mat3& R_d_used =
    after_contact
        ? R_contact_start
        : (phase == ControlPhase::kPoseHold)
              ? R_d
              : (phase == ControlPhase::kToolOrientation)
                    ? R_orient_command
                    : R_base_tool_target;

const Vec3 e_R = applyRotationalAxisMask(
    params,
    orientationError(R_EE, R_d_used),
    R_base_surface);
```

The function `orientationError` forms the current-to-desired relative rotation and converts it to an axis–angle vector. The resulting local rotation vector is transformed to the robot base frame:

Listing 3.4: Base-frame orientation-error calculation.

```
Mat3 R_error = R_current.transpose() * R_desired;
Eigen::AngleAxisd angle_axis(R_error);

return R_current
       * angle_axis.axis()
       * angle_axis.angle();
```

A rotation angle below the numerical tolerance returns a zero vector. The orientation-error convention therefore corresponds to the corrective base-frame error e_R defined in Section 2.2.4.

The rotational mask is applied after the error has been transformed to the base frame. The function first expresses e_R in the surface frame, selects the enabled components about t_1, t_2, and n_s, and transforms the result back to the base frame. The baseline configuration enables all three components. The mask acts on the rotational spring error, while rotational damping continues to use the measured angular velocity and the selected damping matrix.

The callback then combines the translational and rotational quantities into the six-dimensional displacement and velocity-error vectors used by the impedance routine. The desired angular velocity is zero in the implemented control sequence.

Listing 3.5: Assembly and evaluation of the Cartesian impedance command.

```
Vec6 dx;
dx.head<3>() = e_p;
dx.tail<3>() = e_R;

Vec6 dv;
dv.head<3>() = desired.pdot_d - pdot;
dv.tail<3>() = -omega;
```

```
8
9  const Vec6 wrench =
10     computeCartesianImpedanceWrench(
11         params, phase,
12         Kp_used, Dp_used,
13         KR_used, DR_used,
14         R_base_surface,
15         dx, dv, R_EE);
16
17 const Vec7 tau_task = J.transpose() * wrench;
```

The variables `Kp_used`, `Dp_used`, `KR_used`, and `DR_used` contain the gains selected for the current phase. Their construction and phase assignment are described in Section 3.6. The implementation variable `wrench` follows the force–moment order used in Chapter 2, while `tau_task` corresponds to the Cartesian impedance torque τ_{cart}.

The function `computeCartesianImpedanceWrench` selects between the decoupled Cartesian impedance and the point-shifted impedance associated with the virtual centre of compliance. The decoupled branch is implemented by applying the translational and rotational gain blocks separately:

Listing 3.6: Decoupled Cartesian impedance branch.

```
1  Vec6 wrench;
2  wrench.head<3>() =
3      Kp * dx.head<3>() + Dp * dv.head<3>();
4
5  wrench.tail<3>() =
6      KR * dx.tail<3>() + DR * dv.tail<3>();
7
8  return wrench;
```

This branch is used during tool orientation, surface approach, grinding, and the standard Cartesian pose hold. It is also used during set-up when the virtual centre of compliance is disabled.

The coupled branch is selected when the virtual centre of compliance is enabled during set-up or during the dedicated set-up-impedance hold mode:

Listing 3.7: Selection of compliance-centre coupling.

```
1  const bool coupled =
2      params.use_virtual_compliance_center &&
3      (phase == ControlPhase::kSetup ||
4       (phase == ControlPhase::kPoseHold &&
5        params.use_setup_impedance_hold));
```

In this branch, the translational and rotational gain blocks are assembled into spatial 6×6 matrices and shifted from the configured centre of compliance to the TCP. The construction of these matrices and the two implemented definitions of the compliance-centre lever are described in Section 3.7.

The wrench returned by either branch is mapped to joint torque by `J.transpose() * wrench`. The resulting `tau_task` contribution is subsequently combined with the null-space, disturbance, and Coriolis torque contributions in the real-time callback.

3.4 SVD-Based Null-Space Control

The mathematical formulation of the null-space projector, projected damping, and singular-value conditioning torque is presented in Section 2.8. This section describes their implementation in the function `computeNullspaceTorque`, which is evaluated during every impedance-controlled callback.

The function receives the current geometric Jacobian, measured joint velocity, robot state, model interface, and active null-space parameters. The current joint configuration is mapped from the robot state, and the full singular value decomposition of the 6×7 Jacobian is calculated. The pseudoinverse and joint-torque null-space projector are then constructed as shown in listing 3.8.

Listing 3.8: Construction of the SVD-based pseudoinverse and joint-torque null-space projector.

```
Map<const Vec7> q_current(state.q.data());

Eigen::JacobiSVD<Mat6x7> svd_current(
    J, Eigen::ComputeFullU | Eigen::ComputeFullV);

const Vec6 singular_values =
    svd_current.singularValues();

const double sigma_max =
    singular_values.maxCoeff();

const double svd_cutoff =
    std::max(
        0.0,
        params.nullspace_svd_relative_tolerance)
    * sigma_max;

Mat7x6 J_pinv = Mat7x6::Zero();

for (int i = 0; i < 6; ++i) {
  if (singular_values(i) > svd_cutoff) {
    J_pinv +=
        (1.0 / singular_values(i))
        * svd_current.matrixV().col(i)
        * svd_current.matrixU().col(i).transpose();
  }
}

Mat7x7 N_tau =
    Mat7x7::Identity()
    - J.transpose() * J_pinv.transpose();

N_tau =
    0.5 * (N_tau + N_tau.transpose());
```

The relative tolerance `nullspace_svd_relative_tolerance` is multiplied by the largest singular value of the current Jacobian. This produces the singular-value threshold used during the current callback. Each singular direction above this threshold contributes to the pseudoinverse with the reciprocal $1/\sigma_i$. The remaining reciprocal entries are set to zero.

The implementation forms the joint-torque projector directly according to the expression derived in Section 2.8.3. Averaging the projector with its transpose removes small numerical asymmetries caused by floating-point calculations.

The measured joint velocity is subsequently projected into the numerical null space. The active `NullspaceMode` determines which secondary torque contributions are evaluated:

Listing 3.9: Projection of the joint velocity and selection of the null-space damping contribution.

```
const Vec7 dq_nullspace = N_tau * dq;

if (params.nullspace_mode ==
    NullspaceMode::kOff) {
  return Vec7::Zero();
}

const bool damping_enabled =
    params.nullspace_mode ==
        NullspaceMode::kDampingOnly
    ||
    params.nullspace_mode ==
        NullspaceMode::kDampingAndSigma;

Vec7 tau_damping = Vec7::Zero();

if (damping_enabled) {
  tau_damping.noalias() =
      -params.nullspace_damping
      * dq_nullspace;
}

if (params.nullspace_mode ==
    NullspaceMode::kDampingOnly) {
  return tau_damping;
}

const bool sigma_only =
    params.nullspace_mode ==
        NullspaceMode::kSigmaOnly;

const Vec7 sigma_fallback =
    sigma_only
        ? Vec7::Zero()
        : tau_damping;
```

The four selectable modes are defined in the controller as

Listing 3.10: Selectable null-space operating modes.

```
enum class NullspaceMode {
  kOff = 0,
  kDampingOnly = 1,
  kSigmaOnly = 2,
  kDampingAndSigma = 3
};
```

The disabled mode returns zero secondary torque. The damping-only mode returns the negative projected joint-velocity damping. The sigma-only mode evaluates the singular-value conditioning contribution, while the combined mode adds both contributions.

For singular-value conditioning, the seventh column of the current right-singular-vector matrix is selected and normalised. This vector corresponds to the redundant direction v_7 introduced in Section 2.8.1. Two nearby joint configurations are then formed along its positive and negative directions.

Listing 3.11: Construction and evaluation of the two sampled joint configurations.

```
Vec7 n =
    svd_current.matrixV().col(6);

if (n.norm() <= 1e-9) {
  return sigma_fallback;
}

n.normalize();

const double alpha =
    std::abs(
        params.nullspace_probe_step_rad);

if (alpha <= 1e-12) {
  return sigma_fallback;
}

const Array7 q_plus_array =
    vec7ToArray(
        Vec7(q_current + alpha * n));

const Array7 q_minus_array =
    vec7ToArray(
        Vec7(q_current - alpha * n));

const std::array<double, 42> J_plus_array =
    model.zeroJacobian(
        Frame::kEndEffector,
        q_plus_array,
        state.F_T_EE,
        state.EE_T_K);

const std::array<double, 42> J_minus_array =
    model.zeroJacobian(
        Frame::kEndEffector,
        q_minus_array,
        state.F_T_EE,
        state.EE_T_K);

Map<const Mat6x7> J_plus(
    J_plus_array.data());

Map<const Mat6x7> J_minus(
    J_minus_array.data());

const double sigma_plus =
    smallestSingularValue(J_plus);

const double sigma_minus =
    smallestSingularValue(J_minus);
```

The parameter `nullspace_probe_step_rad` defines the joint-space distance α_{probe} between the current configuration and the two sampled configurations. The Jacobians are evaluated using the current flange-to-end-effector and end-effector-to-stiffness-frame

transformations provided in the robot state.

The smallest singular values of the sampled Jacobians are compared through the configured deadband. The sign of their difference selects the sampled direction with the larger value of $\sigma_{\min}$.

Listing 3.12: Selection and projection of the singular-value conditioning torque.

```
if (!std::isfinite(sigma_plus) ||
    !std::isfinite(sigma_minus)) {
  return sigma_fallback;
}

const double sigma_difference =
    sigma_plus - sigma_minus;

const double deadband =
    std::max(
        0.0,
        params.nullspace_sigma_deadband);

if (std::abs(sigma_difference)
    <= deadband) {
  return sigma_fallback;
}

const double sigma_direction =
    (sigma_difference > 0.0)
        ? 1.0
        : -1.0;

const Vec7 best_direction =
    sigma_direction * n;

const Vec7 tau_sigma =
    params.nullspace_sigma_gain
    * N_tau
    * best_direction;

return sigma_only
           ? tau_sigma
           : tau_damping + tau_sigma;
```

The parameter `nullspace_sigma_deadband` specifies the minimum difference required for selecting one sampled direction. Within the deadband, the sigma-only mode returns zero torque and the combined mode retains the damping contribution.

The vector `best_direction` corresponds to sv_7. Multiplication by the projector retains its numerical null-space component, while `nullspace_sigma_gain` specifies the conditioning-torque magnitude. The function returns the torque associated with the selected mode.

The baseline configuration is loaded from `nullspace.conf`. It uses the combined damping and singular-value conditioning mode with the parameters

$$d_{\text{null}} = 2.0\,\text{N m s/rad}, \qquad k_\sigma = 1.0\,\text{N m}, \tag{3.6}$$

$$\alpha_{\text{probe}} = 0.08\,\text{rad}, \qquad \delta_\sigma = 10^{-6}, \qquad \rho = 10^{-4}. \tag{3.7}$$

During Cartesian pose hold, the operator can adjust the active null-space mode, damping coefficient, conditioning-torque magnitude, and probe step. The requested values are transferred through atomic variables and assigned inside the callback. Live null-space tuning is associated with the standard Cartesian pose hold, while the phase sequence and set-up-impedance hold use the configuration loaded before the controller session.

3.5 Surface and Tool Geometry

The mathematical definition of the surface task frame is given in Section 2.5. In the implementation, this frame is constructed from the calibrated plane orientation and a tangent-direction hint. The resulting frame is then used for the tool-orientation target, the approach direction, the phase-dependent gains, and the surface-relative evaluation quantities.

3.5.1 Surface Task Frame

The plane configuration contains one surface point expressed in the robot base frame, two surface-tilt angles, and a tangent-direction hint. The parameter loader converts the tilt angles to a unit surface normal, after which `makeSurfaceFrame` constructs the orthonormal surface frame.

Listing 3.13: Construction of the configured surface point, normal, and task frame.

```
params.surface_point =
    getVec3Xyz("surface_point",
               params.surface_point);

const double ax =
    params.surface_tilt_x_deg * M_PI / 180.0;
const double ay =
    params.surface_tilt_y_deg * M_PI / 180.0;

params.surface_normal_base =
    Vec3(std::sin(ay) * std::cos(ax),
         -std::sin(ax),
         std::cos(ay) * std::cos(ax));

params.surface_normal_base.normalize();

Vec3 normal =
    normalizedOrFallback(
        params.surface_normal_base,
        Vec3(1.0, 0.0, 0.0));

```

```
Vec3 tangent1 =
    params.surface_tangent1_hint_base
    - normal
      * normal.dot(
          params.surface_tangent1_hint_base);

if (tangent1.norm() <= 1e-9) {
  Vec3 fallback(0.0, 1.0, 0.0);
  if (std::abs(normal.dot(fallback)) > 0.95) {
    fallback = Vec3(0.0, 0.0, 1.0);
  }
  tangent1 =
      fallback - normal * normal.dot(fallback);
}

tangent1.normalize();

Vec3 tangent2 =
    normal.cross(tangent1).normalized();

Mat3 R_base_surface;
R_base_surface.col(0) = tangent1;
R_base_surface.col(1) = tangent2;
R_base_surface.col(2) = normal;
```

The code variable `R_base_surface` corresponds to R_{surface} in Chapter 2. Its columns follow the fixed order

$$[\,t_1,\ t_2,\ n_s\,].$$

Projecting the configured tangent hint onto the plane defines t_1, while the cross product $n_s \times t_1$ defines t_2. A fallback direction is selected when the tangent hint is approximately parallel to the surface normal.

The active baseline uses the calibrated surface point

$$p_s = \begin{bmatrix} 0.5153 & -0.1072 & 0.0031 \end{bmatrix}^\top \text{m},$$

together with the calibrated surface tilts

$$\alpha_s = -1.585^\circ, \qquad \beta_s = +0.988^\circ.$$

The tangent hint is the positive base-frame x-direction. The configuration selector `use_start_as_surface_point` can alternatively assign the captured controller start position to the runtime surface point. The baseline configuration selects the calibrated point from `surface.conf`:

Listing 3.14: Selection of the surface point used during one controller session.

```
Vec3 surface_point_runtime =
```

```
    params.use_start_as_surface_point
        ? p_start
        : params.surface_point;
```

The descent direction is defined once from the surface normal:

Listing 3.15: Surface-normal descent direction.

```
const Vec3 descend_direction =
    -R_base_surface.col(2);
```

It therefore points opposite to n_s, from the free-space starting region toward the configured plane.

3.5.2 Surface-Relative Tool Orientation

The controlled physical tool axis is configured in the end-effector frame. During each callback, its current base-frame direction is obtained from

Listing 3.16: Transformation of the configured tool axis to the robot base frame.

```
Vec3 currentToolAxisInBase(
    const ControllerConfig& params,
    const Mat3& R_EE) {

  return R_EE
         * normalizedOrFallback(
               params.tool_axis_ee,
               Vec3(0.0, 0.0, 1.0));
}
```

The flat alignment direction is selected as either $+n_s$ or $-n_s$ through `tool_axis_target_sign`. The configured angular offsets about the two surface tangents are combined into one rotation vector and applied to this flat direction:

Listing 3.17: Construction of the surface-relative tool-axis target.

```
const double sign =
    (params.tool_axis_target_sign >= 0.0)
        ? 1.0
        : -1.0;

const Vec3 flat_axis =
    sign * R_base_surface.col(2);

const Vec3 rotation_vector =
    (M_PI / 180.0)
    * (params.tool_target_offset_tangent1_deg
           * R_base_surface.col(0)
       + params.tool_target_offset_tangent2_deg
           * R_base_surface.col(1));

const double angle = rotation_vector.norm();

const Vec3 tool_axis_target =
    (angle <= 1e-12)
        ? flat_axis
        : Eigen::AngleAxisd(
              angle,
```

```
            rotation_vector / angle)
            * flat_axis;
```

This construction permits the tool axis to be aligned with the plane or commanded with a defined initial tilt about t_1 and t_2. The shortest rotation from the tool axis at the captured start orientation to the selected target axis is then applied to the complete start orientation:

Listing 3.18: Initial tool-axis alignment while retaining the captured orientation about the tool axis.

```
const Vec3 tool_axis_start =
    currentToolAxisInBase(
        params, R_start).normalized();

const Mat3 R_axis =
    rotationBetweenUnitVectors(
        tool_axis_start,
        tool_axis_target)
    * R_start;
```

When `command_tool_twist` is enabled, the rectangular-face long axis provides an additional in-plane orientation reference. Its current direction and the selected surface tangent are projected onto the plane perpendicular to the desired tool axis. The configured normal-axis offset then determines the desired twist about that axis.

Listing 3.19: Surface-relative twist command about the desired tool axis.

```
const Vec3 face_long_axis_ee =
    normalizedOrFallback(
        params.tool_contact_half_length_ee,
        Vec3(0.0, 1.0, 0.0));

const auto perpendicular =
    [&tool_axis_target](const Vec3& v) {
      return v
             - v.dot(tool_axis_target)
               * tool_axis_target;
    };

const Vec3 current =
    perpendicular(R_axis * face_long_axis_ee);

const Vec3 zero_reference =
    perpendicular(R_base_surface.col(0));

const Vec3 target =
    Eigen::AngleAxisd(
        (M_PI / 180.0)
            * params.tool_target_offset_normal_deg,
        tool_axis_target)
    * zero_reference.normalized();

const Vec3 current_unit = current.normalized();

double twist =
    std::atan2(
        tool_axis_target.dot(
            current_unit.cross(target)),
        current_unit.dot(target));

const Mat3 R_base_tool_target =
    Eigen::AngleAxisd(
        twist, tool_axis_target)
```

```
    * R_axis;
```

For the centred rectangular face, rotations differing by 180° give an equivalent face orientation. The implementation selects the equivalent half-turn nearest to the captured start orientation, thereby reducing the required orientation motion.

The active baseline defines the controlled axis as $+z_{\mathrm{EE}}$ and selects alignment toward the surface through

$$\texttt{tool\_axis\_target\_sign} = -1.$$

The commanded orientation contains an offset of 10° about t_1, 0° about t_2, and an in-plane twist of 90° about the target tool axis. The twist command is enabled in `tool_orientation.conf`.

3.5.3 Rectangular Grinding Face and Clearance

The contact geometry is represented by a rectangular face fixed in the end-effector frame. Its centre, half-width vector, and half-length vector define the four vertices:

Listing 3.20: Construction of the rectangular contact face in the end-effector frame.

```
const Vec3& face_center =
    params.tool_contact_face_center_ee;

const Vec3& half_width =
    params.tool_contact_half_width_ee;

const Vec3& half_length =
    params.tool_contact_half_length_ee;

const std::array<Vec3, 4>
    tool_face_corners_ee = {{
        face_center + half_width + half_length,
        face_center + half_width - half_length,
        face_center - half_width + half_length,
        face_center - half_width - half_length,
    }};
```

The baseline face centre is located 20 mm along $+z_{\mathrm{EE}}$. Its half-width and half-length are 20 mm along x_{EE} and 60 mm along y_{EE}, respectively. The represented contact face therefore measures 40 mm × 120 mm.

Before set-up, the controller transforms all four vertices to the base frame and projects them onto the descent direction. The maximum projection identifies the part of the face that leads during the approach.

Listing 3.21: Selection of the leading face feature along the descent direction.

```
std::array<double, 4> projection;

double leading_projection =
    (R_EE * tool_face_corners_ee[0])
        .dot(descend_direction);

projection[0] = leading_projection;

for (std::size_t i = 1;
     i < tool_face_corners_ee.size();
     ++i) {
  projection[i] =
      (R_EE * tool_face_corners_ee[i])
          .dot(descend_direction);

  leading_projection =
      std::max(
          leading_projection,
          projection[i]);
}

Vec3 selected = Vec3::Zero();
int selected_count = 0;

for (std::size_t i = 0;
     i < tool_face_corners_ee.size();
     ++i) {
  if (leading_projection - projection[i]
      <= params.tool_contact_feature_tie_tolerance) {
    selected += tool_face_corners_ee[i];
    ++selected_count;
  }
}

const Vec3 tool_contact_offset_ee =
    selected
    / static_cast<double>(selected_count);
```

Vertices whose projections differ from the leading value by at most the configured tolerance are averaged. Depending on the current orientation, the selected feature is therefore a corner, the centre of a leading edge, or the centre of the complete face. The baseline tie tolerance is

$$0.0001\,\mathrm{m} = 0.1\,\mathrm{mm}.$$

The baseline enables both contact-point control and automatic leading-feature selection. The selected offset is transformed to the base frame to obtain the current controlled feature:

Listing 3.22: Base-frame position of the selected tool feature.

```
const Vec3 tool_contact_point =
    p_EE + R_EE * tool_contact_offset_ee;
```

The leading feature is reevaluated while the tool is being oriented and approached. On entry to set-up, its end-effector-frame offset is stored in `active_tool_contact_offset_ee` and subsequently retained during set-up and grinding.

During surface approach, the clearance calculation distinguishes the exact leading projection from the position of the averaged controlled feature:

Listing 3.23: Geometric clearance evaluation during surface approach.

```
const Vec3 surface_normal =
    (-descend_direction).normalized();

const double height_above_surface =
    surface_normal.dot(
        p_EE - surface_point_runtime)
    - leading_contact_projection;

const double controlled_point_height =
    surface_normal.dot(
        tool_contact_point
        - surface_point_runtime);

const Vec3 projected_surface_point =
    tool_contact_point
    - controlled_point_height
      * surface_normal;

const bool clearance_reached =
    height_above_surface
    <= params.descend_surface_clearance;
```

The quantity `height_above_surface` represents the normal clearance of the foremost point of the complete rectangular face. It provides the geometric transition condition. The quantity `controlled_point_height` is the signed height of the averaged controlled feature. Its projection onto the configured plane provides the surface reference used by the subsequent set-up motion.

When the clearance condition is satisfied, the feature geometry and current pose are captured:

Listing 3.24: Geometry captured at the transition from approach to set-up.

```
active_tool_contact_offset_ee =
    tool_contact_offset_ee;

first_contact_tcp = p_EE;

first_contact_point =
    projected_surface_point;

setup_push_start =
    -controlled_point_height;

R_contact_start = R_EE;

phase = ControlPhase::kSetup;
phase_start_time = time;
```

The internal variable `first_contact_point` stores the projected clearance point on the configured plane. The live signed height determines `setup_push_start`, which produces a continuous desired position when the controller changes from approach to set-up.

During set-up, the desired position of the selected feature is advanced from this projected

point along the descent direction. The feature target is then converted to the corresponding TCP reference using the orientation captured at the clearance transition:

Listing 3.25: Conversion of the set-up feature target to the desired TCP position.

```
const Vec3 edge_target =
    first_contact_point
    + push * descend_direction;

desired.p_d =
    edge_target
    - R_contact_start
      * tool_contact_offset_ee;

desired.pdot_d =
    setup_push_velocity
    * descend_direction;
```

This construction allows the translational impedance controller to command the selected tool feature while the Cartesian control law remains formulated at the TCP.

3.6 Phase-Dependent Gains and Inertia-Scaled Damping

The controller assigns separate Cartesian stiffness and damping matrices to the approach, set-up, grinding, pose-hold, and phase-gate modes. The matrices are constructed before the real-time callback begins, while the inertia-scaled damping matrices are updated on entry to the corresponding phase group.

3.6.1 Phase-Dependent Gain Construction

The function `buildPhaseImpedanceGains` constructs all phase-dependent gain matrices from the active controller configuration. Gains defined along the surface directions are transformed to the robot base frame using

Listing 3.26: Transformation of diagonal surface-frame gains to the robot base frame.

```
PhaseImpedanceGains gains;
gains.R_base_surface = makeSurfaceFrame(params);

auto taskGain = [&](const Vec3& diagonal) -> Mat3 {
  return makeSpatialGainMatrix(
      diagonal,
      gains.R_base_surface);
};

gains.Kp_approach =
    taskGain(params.approach_Kp_diag);
gains.Dp_approach =
    taskGain(params.approach_Dp_diag);
```

```
gains.KR_approach =
    taskGain(params.approach_KR_diag);
gains.DR_approach =
    taskGain(params.approach_DR_diag);
```

The function `makeSpatialGainMatrix` evaluates

$$G_0 = R_{\text{surface}} G_{\text{surface}} R_{\text{surface}}^{\top}, \tag{3.8}$$

where G_{surface} is diagonal in the fixed axis order $[t_1, t_2, n_s]$. The resulting matrix G_0 is expressed in the robot base frame and can be applied directly to the base-frame Cartesian error.

The gain-frame policy implemented for each controller mode is summarised in table 3.2.

Table 3.2: Coordinate frames used for the phase-dependent Cartesian gains.

Controller mode	**Translational gains**	**Rotational gains**
Tool orientation and surface approach	Surface frame $[t_1, t_2, n_s]$	Surface frame $[t_1, t_2, n_s]$
Set-up and grinding	Selectable between base frame $[x, y, z]$ and surface frame $[t_1, t_2, n_s]$	Surface frame $[t_1, t_2, n_s]$
Cartesian pose hold	Base frame $[x, y, z]$	Base frame $[x, y, z]$
Set-up-impedance hold	Same construction as set-up	Same construction as set-up
Phase-gate hold	Base frame $[x, y, z]$	Surface frame $[t_1, t_2, n_s]$

For set-up translation, the parameter `setup_translation_surface_frame` selects the coordinate frame:

Listing 3.27: Selection of the translational set-up gain frame.

```
gains.Kp_setup =
    params.setup_translation_surface_frame
        ? taskGain(params.setup_Kp_surface_diag)
        : params.setup_Kp_diag.asDiagonal();

gains.Dp_setup =
    params.setup_translation_surface_frame
        ? taskGain(params.setup_Dp_surface_diag)
        : params.setup_Dp_diag.asDiagonal();

gains.KR_setup =
    taskGain(params.setup_KR_diag);

gains.DR_setup =
```

```
15|     taskGain(params.setup_DR_diag);
```

The baseline configuration selects base-frame translational gains for set-up and grinding. The corresponding rotational gains remain aligned with the surface tangents and normal. Cartesian pose-hold gains are assembled as diagonal base-frame matrices:

Listing 3.28: Construction of the pose-hold and phase-gate gains.

```
gains.Kp_hold =
    params.hold_Kp_diag.asDiagonal();
gains.Dp_hold =
    params.hold_Dp_diag.asDiagonal();
gains.KR_hold =
    params.hold_KR_diag.asDiagonal();
gains.DR_hold =
    params.hold_DR_diag.asDiagonal();

gains.Kp_pause =
    params.pause_hold_Kp_diag.asDiagonal();
gains.Dp_pause =
    params.pause_hold_Dp_diag.asDiagonal();
gains.KR_pause =
    taskGain(params.pause_hold_KR_diag);
gains.DR_pause =
    taskGain(params.pause_hold_DR_diag);
```

The real-time callback selects the active matrices from the current control phase:

Listing 3.29: Selection of the active phase-dependent gain matrices.

```
const Mat3* Kp_phase =
    params.use_setup_impedance_hold
        ? &Kp_setup
        : &Kp_hold;

const Mat3* Dp_phase =
    params.use_setup_impedance_hold
        ? &Dp_setup_eff
        : &Dp_hold_eff;

const Mat3* KR_phase =
    params.use_setup_impedance_hold
        ? &KR_setup
        : &KR_hold;

const Mat3* DR_phase =
    params.use_setup_impedance_hold
        ? &DR_setup_eff
        : &DR_hold_eff;

switch (phase) {
  case ControlPhase::kToolOrientation:
  case ControlPhase::kSurfaceApproach:
    Kp_phase = &Kp_approach;
    Dp_phase = &Dp_approach_eff;
    KR_phase = &KR_approach;
    DR_phase = &DR_approach_eff;
    break;

  case ControlPhase::kSetup:
  case ControlPhase::kGrinding:
    Kp_phase = &Kp_setup;
    Dp_phase = &Dp_setup_eff;
    KR_phase = &KR_setup;
    DR_phase = &DR_setup_eff;
    break;

  case ControlPhase::kPoseHold:
  case ControlPhase::kManualGuidance:
    break;
}
```

Set-up and grinding therefore share the same stiffness and damping matrices. The difference between these phases lies in the reference trajectory and in the selection of the coupled or decoupled impedance law.

An active phase gate overrides the normal phase matrices with the dedicated gate-hold gains:

Listing 3.30: Gain override during an operator confirmation gate.

```
const Mat3& Kp_used =
    pause_hold_active ? Kp_pause : *Kp_phase;

const Mat3& Dp_used =
    pause_hold_active ? Dp_pause_eff : *Dp_phase;

const Mat3& KR_used =
    pause_hold_active ? KR_pause : *KR_phase;

const Mat3& DR_used =
    pause_hold_active ? DR_pause_eff : *DR_phase;
```

These four matrices are passed to the Cartesian impedance routine during the current callback.

3.6.2 Calculation of Inertia-Scaled Damping

Each phase first receives the damping matrices configured in the parameter files. When inertia-scaled damping is enabled for a phase group, the controller replaces these matrices with values calculated from the current operational-space inertia.

The joint-space inertia matrix is obtained from the libfranka model. The implementation uses an LDLT factorisation to solve for $M^{-1}J^{\top}$ and constructs the inverse operational-space inertia:

Listing 3.31: Calculation of the operational-space inertia estimate.

```
Eigen::LDLT<Mat7x7> mass_ldlt(joint_mass);

const Mat7x6 Minv_Jt =
    mass_ldlt.solve(J.transpose());

Mat6x6 lambda_inv =
    J * Minv_Jt;

lambda_inv =
    0.5 * (lambda_inv
           + lambda_inv.transpose());

Mat6x6 lambda_inv_regularized =
    lambda_inv;

lambda_inv_regularized
    .diagonal().array() += 1e-9;

Eigen::LDLT<Mat6x6> lambda_ldlt(
    lambda_inv_regularized);
```

```
21
22 Mat6x6 Lambda_base =
23     lambda_ldlt.solve(
24         Mat6x6::Identity());
```

This calculation corresponds to the regularised estimate

$$\Lambda_0 = \left(JM^{-1}J^\top + 10^{-9}I_6\right)^{-1}. \tag{3.9}$$

The matrix is symmetrised before and after its factorisation. It is then transformed from the robot base axes to the coordinate frame of the active gain block:

Listing 3.32: Transformation and extraction of directional Cartesian inertia.

```
1  const Mat6x6 T_task =
2      blockDiagonalRotation(R_task);
3
4  Mat6x6 Lambda_task =
5      T_task.transpose()
6      * Lambda_base
7      * T_task;
8
9  for (int i = 0; i < 3; ++i) {
10   result.translational(i) =
11       Lambda_task(i, i);
12
13   result.rotational(i) =
14       Lambda_task(i + 3, i + 3);
15 }
```

The first three diagonal entries provide the directional translational inertias, while the last three provide the directional rotational inertias. For each direction i, the inertia-scaled damping coefficient is calculated from

$$d_i = 2\zeta\sqrt{\lambda_i k_i}, \tag{3.10}$$

where λ_i is the corresponding directional Cartesian inertia, k_i is the directional stiffness, and ζ is the configured damping ratio. For $\zeta = 1$, the expression gives the critical-damping coefficient of the corresponding decoupled direction.

The calculation is implemented independently for the three translational and three rotational directions:

Listing 3.33: Calculation and limiting of the inertia-scaled damping coefficients.

```
1  for (int i = 0; i < 3; ++i) {
2    const double inertia_i =
3        std::max(0.0, inertia(i));
4
5    const double stiffness_i =
6        std::max(0.0, stiffness(i));
7
8    const double calculated =
```

```
        2.0 * zeta
        * std::sqrt(
            inertia_i * stiffness_i);

    damping(i) =
        std::min(
            max_damping,
            std::max(
                min_damping(i),
                calculated));
}
```

The common upper bound is defined by `auto_damping_max`. The parameter `auto_damping_min_from_manual` selects either zero or the configured damping value as the lower bound.

The coordinate frame used for the inertia estimate follows the frame of the corresponding stiffness matrix:

- approach translation and rotation use the surface frame;
- set-up translation uses the selected base or surface frame, while set-up rotation uses the surface frame;
- Cartesian pose hold uses the robot base frame; and
- the phase gate uses base-frame translation and surface-frame rotation.

The damping matrices are cached by phase group. Entering tool orientation or surface approach calculates the approach damping. Entering set-up, grinding, or set-up-impedance hold calculates the set-up damping. Cartesian pose hold uses its hold damping, while an active operator gate uses the gate-hold damping. Leaving a phase group clears its cached state so that a later entry uses the current robot configuration.

For the phase-gate hold, separate damping ratios are derived from the configured translational and rotational gate-hold damping values. Cartesian pose hold can similarly derive one damping ratio from its configured translational damping and apply it to the translational and rotational inertia-scaled damping calculations.

The baseline configuration enables inertia-scaled damping for approach, set-up, and the phase gates. Standard Cartesian pose hold uses its configured damping matrices. The exact stiffness, damping-factor, and limiting values used during the experiments are listed in Chapter 4.

When the inertia calculation satisfies the numerical validity checks, the calculated damping matrices replace the configured matrices for the active phase group. The configured

matrices remain available as the initial and fallback values.

3.7 Virtual Centre of Compliance

The theoretical transformation from a centre-defined Cartesian impedance to an equivalent TCP impedance is derived in Section 2.7. The implementation applies this transformation inside `computeCartesianImpedanceWrench`. The phase-dependent translational and rotational gain blocks are first expressed in the robot base frame and are then shifted from the selected virtual centre of compliance to the TCP.

The coupled branch is activated during set-up and during the dedicated set-up-impedance hold mode:

Listing 3.34: Selection of the virtual-compliance-centre branch.

```
const bool coupled =
    params.use_virtual_compliance_center &&
    (phase == ControlPhase::kSetup ||
     (phase == ControlPhase::kPoseHold &&
      params.use_setup_impedance_hold));

if (!coupled) {
  Vec6 wrench;
  wrench.head<3>() =
      Kp * dx.head<3>()
      + Dp * dv.head<3>();

  wrench.tail<3>() =
      KR * dx.tail<3>()
      + DR * dv.tail<3>();

  return wrench;
}
```

Tool orientation, surface approach, grinding, and standard Cartesian pose hold therefore use the decoupled impedance branch. A pre-grinding confirmation gate retains `ControlPhase::kSetup`; the coupled set-up impedance consequently remains active while the controller waits at the achieved preload.

The controller supports two definitions of the virtual centre. Both follow the lever convention

$$r_c = p_{\mathrm{TCP}} - p_c$$

introduced in Equation (2.50). The first definition specifies the centre position relative to the TCP in end-effector coordinates. The second specifies the lever r_c directly in surface

coordinates.

Listing 3.35: Resolution of the compliance-centre lever in the robot base frame.

```
Vec3 r_c = Vec3::Zero();

if (params.compliance_center_in_tool_frame) {
  r_c =
      -(R_EE
        * params.compliance_center_offset_ee);
} else {
  r_c =
      R_base_surface
      * params.r_tcp_from_compliance_center_surface;
}
```

For the tool-frame definition, `compliance_center_offset_ee` represents $p_c - p_{\mathrm{TCP}}$. The multiplication by R_{EE} expresses this offset in the robot base frame, while the negative sign converts it to the implemented lever convention. Since the measured orientation R_{EE} is used during every callback, this lever follows the current end-effector orientation.

For the surface-frame definition, `r_tcp_from_compliance_center_surface` contains the components of r_c along $[t_1, t_2, n_s]$. Multiplication by `R_base_surface` expresses the lever in the robot base frame. The surface frame and configured lever remain fixed during one controller session, so this definition produces a constant base-frame lever throughout set-up.

When virtual-compliance-centre control is enabled, the configuration loader requires exactly one of these definitions:

Listing 3.36: Validation of the compliance-centre configuration.

```
if (params.use_virtual_compliance_center &&
    params.compliance_center_in_tool_frame ==
        params.compliance_lever_in_surface_frame) {
  throw std::runtime_error(
      "Virtual compliance-center control requires "
      "exactly one definition.");
}
```

The function `complianceShiftMatrix` constructs the point-shift matrix derived in Equation (2.52). The skew-symmetric lever matrix occupies the upper-right block because the Cartesian vectors follow the ordering [translation, rotation].

Listing 3.37: Implementation of the spatial point shift and gain congruence.

```
Mat6x6 complianceShiftMatrix(
    const Vec3& r_c) {

  Mat6x6 adjoint = Mat6x6::Zero();

  adjoint.block<3, 3>(0, 0) =
      Mat3::Identity();

  adjoint.block<3, 3>(0, 3) =
```

```
      skewMatrix(r_c);

  adjoint.block<3, 3>(3, 3) =
      Mat3::Identity();

  return adjoint;
}

Mat6x6 shiftGainToTcp(
    const Mat6x6& gain_at_center,
    const Vec3& r_c) {

  const Mat6x6 adjoint =
      complianceShiftMatrix(r_c);

  return adjoint.transpose()
         * gain_at_center
         * adjoint;
}
```

The phase-dependent 3×3 translational and rotational matrices are combined into centre-defined spatial gain matrices. The same congruence is then applied independently to stiffness and damping:

Listing 3.38: Construction and evaluation of the coupled TCP impedance.

```
const Mat6x6 K_tcp =
    shiftGainToTcp(
        blockDiagonal(Kp, KR),
        r_c);

const Mat6x6 D_tcp =
    shiftGainToTcp(
        blockDiagonal(Dp, DR),
        r_c);

return K_tcp * dx + D_tcp * dv;
```

The function `blockDiagonal` places K_p and D_p in the translational blocks and K_R and D_R in the rotational blocks. These matrices have already been transformed into the frame selected for the active phase. The subsequent congruence changes the impedance reference point while preserving the selected principal-axis orientation.

The off-diagonal blocks generated by the point shift couple translation and rotation. A translational displacement or velocity error can therefore contribute to the commanded TCP moment, while a rotational error can contribute to the commanded TCP force. The coupling disappears when the configured lever is zero, in which case the shifted matrices reduce to their block-diagonal form.

During set-up, the desired orientation remains the orientation captured at the clearance transition. This orientation acts as a soft rotational reference while the selected tool feature is advanced into the surface. The resulting rotation is determined by the coupled impedance, the contact wrench, the directional stiffness, and the physical tool–surface interaction rather than by a prescribed rotational trajectory.

The set-up and grinding phases share the same 3×3 stiffness and damping blocks. At the transition to grinding, the state machine assigns `ControlPhase::kGrinding` before the Cartesian wrench is evaluated later in the same callback. The first grinding sample therefore uses the decoupled branch. The implementation applies this change as a direct switch from the shifted 6×6 matrices to the decoupled gain blocks, while the achieved penetration and desired orientation remain continuous across the transition.

The set-up-impedance hold mode permits the translational stiffness, rotational stiffness, commanded tilt, and compliance-centre coordinates to be adjusted through the operator interface. Centre-position commands are entered in end-effector coordinates, while lever commands are entered in surface coordinates. An accepted update is converted to the selected internal representation, after which the phase gains and inertia-scaled-damping cache are rebuilt.

After set-up, the reporting module resolves the selected centre in both end-effector and surface coordinates. It also evaluates the effective TCP stiffness and damping matrices, the largest translation–rotation coupling entry, and the eigenvalue ranges of the shifted matrices. These quantities support verification of the configured lever, its sign, and the resulting coupled impedance.

3.8 Control Modes and Phase Sequence

The controller state is represented by the enumeration `ControlPhase`. It contains the four phases of the automatic sequence together with Cartesian pose hold and manual guidance:

Listing 3.39: Control phases and standalone operating modes.

```
enum class ControlPhase {
  kToolOrientation,
  kSurfaceApproach,
  kSetup,
  kGrinding,
  kPoseHold,
  kManualGuidance
};
```

The automatic sequence follows

$$\texttt{kToolOrientation} \longrightarrow \texttt{kSurfaceApproach} \longrightarrow \texttt{kSetup} \longrightarrow \texttt{kGrinding}.$$

The orientation phase can be omitted through the controller configuration. The initial phase is selected before the real-time callback begins:

Listing 3.40: Selection of the initial phase.

```
const ControlPhase sequence_first_phase =
    params.enable_orientation_phase
        ? ControlPhase::kToolOrientation
        : ControlPhase::kSurfaceApproach;

const ControlPhase initial_phase =
    params.run_phase_sequence
        ? sequence_first_phase
        : ControlPhase::kPoseHold;

ControlPhase phase = initial_phase;
ControlPhase restart_phase = initial_phase;
```

The variable `restart_phase` stores the phase resumed after manual guidance. The complete sequence, standard pose hold, and set-up-impedance hold can also be selected during an active controller session. The requested mode is received through an atomic variable and applied inside the callback.

3.8.1 Tool Orientation

The tool-orientation phase maintains the captured start position while the desired orientation is advanced toward the surface-relative target described in Section 3.5.2. The orientation command follows a rate-limited interpolation from the captured start orientation.

The transition condition is evaluated from two errors. The tool-axis error measures the angular separation between the current and desired tool axes. When the in-plane twist command is enabled, the remaining rotation about the target axis is evaluated separately as the spin error.

Listing 3.41: Tool-orientation convergence and phase transition.

```
const bool orientation_reached =
    tool_axis_error
        <= params.approach_orient_error_threshold
    &&
    (!params.command_tool_twist ||
     spin_error
        <= params.approach_orient_spin_error_threshold);

const bool orient_timed_out =
    params.approach_orient_timeout > 0.0
    &&
    phase_time >= params.approach_orient_timeout;

if (phase_time >= params.approach_orient_min_time &&
    (orientation_reached || orient_timed_out)) {
  phase = ControlPhase::kSurfaceApproach;
  phase_start_time = time;
```

```
18
19    contact_force_bias = external_force;
20    contact_moment_bias = external_moment;
21  }
```

The configured minimum duration allows the orientation command to begin settling before convergence is accepted. The timeout provides a second transition path and allows the sequence to continue with the orientation reached within the available interval. The force and moment estimates are reassigned as diagnostic bias values when surface approach begins.

3.8.2 Surface Approach

During surface approach, the desired TCP position advances from the captured start position along the descent direction:

Listing 3.42: Surface-approach translation reference.

```
1  const double phase_time =
2      time - phase_start_time;
3
4  const double distance =
5      std::min(
6          params.descend_speed
7              * (phase_time - gate_paused_time),
8          params.descend_max_distance);
9
10 desired.p_d =
11     p_start + distance * descend_direction;
12
13 desired.pdot_d =
14     params.descend_speed * descend_direction;
```

The variable `gate_paused_time` removes operator-confirmation time from the descent trajectory. The commanded distance is limited by `descend_max_distance`.

The transition to set-up is determined from the geometric clearance of the leading part of the rectangular tool face. The estimated external force is displayed as a diagnostic quantity during approach, while the phase transition uses

Listing 3.43: Geometric transition from surface approach to set-up.

```
1  const double height_above_surface =
2      surface_normal.dot(
3          p_EE - surface_point_runtime)
4      - leading_contact_projection;
5
6  const bool clearance_reached =
7      height_above_surface
8      <= params.descend_surface_clearance;
9
10 if (clearance_reached && !gate_blocking) {
11   active_tool_contact_offset_ee =
12       tool_contact_offset_ee;
13
14   first_contact_tcp = p_EE;
```

```
    first_contact_point =
        projected_surface_point;

    setup_push_start =
        -controlled_point_height;

    R_contact_start = R_EE;

    contact_force_bias = external_force;
    contact_moment_bias = external_moment;

    phase = ControlPhase::kSetup;
    phase_start_time = time;
  }
```

The selected tool-feature offset is locked at this transition. The controller also stores the current TCP pose, the projected point on the surface, the signed starting value of the set-up penetration, and the current force and moment estimates.

When the optional pre-set-up gate is enabled, reaching the clearance first captures the measured TCP position. The controller then holds this position with the dedicated phase-gate gains:

Listing 3.44: Operator gate between approach and set-up.

```
if (!gate_setup_armed) {
  gate_setup_armed = true;
  gate_setup_hold_pd = p_EE;
  signals.gate_continue.store(false);
}

if (signals.gate_continue.load()) {
  gate_setup_passed = true;
  signals.gate_continue.store(false);
} else {
  gate_blocking = true;
  pause_hold_active = true;

  desired.p_d = gate_setup_hold_pd;
  desired.pdot_d.setZero();

  gate_paused_time += period.toSec();
}
```

The geometric clearance condition remains satisfied while the gate is active. After confirmation, the feature and contact references are captured from the current measured state and the controller enters set-up.

Reaching the maximum permitted descent distance before the clearance condition sets the stop request and records the approach as unsuccessful.

3.8.3 Set-Up

The set-up phase advances the selected tool feature from the projected surface point into the plane. The scalar penetration begins at the signed feature coordinate captured during

the approach transition and progresses toward `setup_push_end` at the configured speed.

Listing 3.45: Set-up penetration and TCP reference.

```
double setup_push_velocity = 0.0;

const double push =
    setupPush(
        params,
        phase_time - gate_grind_paused_time,
        setup_push_start,
        setup_push_velocity);

const Vec3 edge_target =
    first_contact_point
    + push * descend_direction;

desired.p_d =
    edge_target
    - R_contact_start
      * tool_contact_offset_ee;

desired.pdot_d =
    setup_push_velocity
    * descend_direction;
```

The function `setupPush` integrates a constant signed speed and clamps the result at the configured final penetration. Subtracting `gate_grind_paused_time` freezes this trajectory while the controller waits at the pre-grinding gate.

The desired orientation remains `R_contact_start`, which is the orientation captured at the clearance transition. The tool can consequently rotate relative to this soft reference under the coupled impedance and the physical contact wrench.

During set-up, the changes in external force and moment are calculated relative to the values captured at the approach transition:

Listing 3.46: Set-up termination condition.

```
const double force_delta_norm =
    (external_force
     - contact_force_bias).norm();

const double moment_delta_norm =
    (external_moment
     - contact_moment_bias).norm();

const bool stopped_on_moment =
    phase_time >= params.setup_min_time
    &&
    moment_delta_norm
        >= params.setup_moment_threshold;

if (!gate_grind_armed &&
    !stopped_on_moment &&
    phase_time < params.setup_timeout) {
  break;
}
```

The force change is recorded and included in the set-up evaluation. The phase exit is governed by the bias-subtracted moment norm after the configured minimum duration. The set-up timeout provides the alternative exit condition.

Before continuing, the controller evaluates and prints the set-up result once. This report includes the achieved pose, feature position, alignment change, force and moment changes, moment transferred to the selected feature, active impedance matrices, and compliance-centre diagnostics.

When the pre-grinding gate is enabled, the controller remains in the set-up phase after the exit condition has been reached:

Listing 3.47: Operator gate between set-up and grinding.

```
if (params.pause_before_grind &&
    !gate_grind_passed) {
  if (!gate_grind_armed) {
    gate_grind_armed = true;
    signals.gate_continue.store(false);
  }

  if (!signals.gate_continue.load()) {
    gate_grind_paused_time
        += period.toSec();

    break;
  }

  gate_grind_passed = true;
  signals.gate_continue.store(false);
}

grind_push = push;
phase = ControlPhase::kGrinding;
phase_start_time = time;
```

The coupled set-up impedance and the achieved pressed-feature reference remain active while the controller waits. After confirmation, the current penetration is stored in `grind_push` and becomes the normal preload of the grinding phase.

3.8.4 Grinding

Grinding retains the final set-up penetration. The controller can either add a tangential sweep or maintain only the normal penetration.

For an active sweep, the selected tangent is obtained from the surface frame. The scalar sweep trajectory is then added to the pressed-feature reference:

Listing 3.48: Grinding reference with an optional tangential sweep.

```
const Vec3 n = descend_direction;
Vec3 edge_target;

if (params.grind_sweep_enabled) {
  const Vec3 grind_tangent =
      (params.grind_tangent_axis == 2)
          ? Vec3(R_base_surface.col(1))
          : Vec3(R_base_surface.col(0));

  double sweep_s = 0.0;
```

```
    double sweep_s_dot = 0.0;

    grindSweep(
        time - phase_start_time,
        params.grind_amplitude_m,
        grindStrokeDuration(params),
        sweep_s,
        sweep_s_dot);

    edge_target =
        first_contact_point
        + grind_push * n
        + sweep_s * grind_tangent;

    desired.pdot_d =
        sweep_s_dot * grind_tangent;
} else {
    const double edge_penetration =
        n.dot(
            tool_contact_point
            - first_contact_point);

    edge_target =
        tool_contact_point
        + (grind_push - edge_penetration) * n;
}

desired.p_d =
    edge_target
    - R_contact_start
      * tool_contact_offset_ee;
```

The function `grindSweep` uses a fifth-order smooth-step trajectory. The first stroke moves from the centre position to the positive amplitude. Subsequent strokes alternate smoothly between the positive and negative amplitudes with zero velocity at each reversal.

When the sweep is disabled, the reference is corrected only along the surface normal. Its tangential coordinates follow the current tool-feature position, which preserves the set-up penetration without imposing a tangential trajectory.

Grinding retains the set-up stiffness and damping values. The Cartesian impedance routine selects the decoupled branch after the phase has changed to `kGrinding`, as described in Section 3.7.

3.8.5 Pose Hold and Manual Guidance

The pose-hold state supports two impedance configurations. Standard pose hold uses the dedicated hold gains, while set-up-impedance hold applies the set-up gains and can retain the virtual centre of compliance. Both modes regulate a captured Cartesian pose.

When the set-up-impedance hold is selected during a run, the measured pose is first captured as the current controller state. The reference is then moved smoothly toward the pose at which the hold session began. Translation uses the configured approach speed, while orientation uses the configured maximum angular rate. After both references have

reached their stored values, the controller continues as a stationary hold.

Manual guidance can be entered from any impedance-controlled phase. The controller then returns Coriolis compensation together with joint-space damping:

Listing 3.49: Manual-guidance torque and pose recapture.

```
const Vec7 tau_guide =
    coriolis
    - params.manual_guidance_damping * dq;

if (signals.proceed_requested.load()) {
  signals.proceed_requested.store(false);

  restartFromPoseReached(true);
  phase = restart_phase;

  hold_return_p = p_start;
  hold_return_R = R_d;
  hold_returning = false;
}
```

After operator confirmation, the measured position, orientation, joint configuration, surface reference when selected, and external-wrench biases are recaptured. The controller then resumes the phase stored in `restart_phase`.

3.8.6 Run Termination

The callback finishes normally when a positive experiment duration has elapsed or the operator issues the stop command:

Listing 3.50: Normal termination of the real-time callback.

```
if ((params.experiment_duration > 0.0 &&
     time >= params.experiment_duration)
    ||
    signals.stop_requested.load()) {
  return MotionFinished(
      Torques(tau_array));
}
```

The same stop signal is assigned when the maximum approach distance is reached. Robot-side safety monitoring can also interrupt the torque-control session through a libfranka exception. The subsequent error handling and collision configuration are described in Section 3.9.

3.9 Robot Preparation and Safety Handling

Robot preparation is performed before the real-time impedance callback begins. It includes

the initial FCI connection, automatic error recovery, collision configuration, motion to the selected initial posture, and the optional gripper and tool-transfer actions.

3.9.1 Robot Connection and Collision Configuration

At program start, the robot address is read from the controller configuration and used to establish the FCI connection. Automatic error recovery is requested before the collision behaviour and robot model are loaded:

Listing 3.51: Robot connection, error recovery, and model initialisation.

```
const ControllerConfig startup_config =
    readControllerConfig(parameter_files);

Robot robot(startup_config.robot_ip);

try {
  robot.automaticErrorRecovery();
} catch (const franka::Exception& e) {
  fprintf(stderr,
          "Automatic error recovery failed: %s\n",
          e.what());
  return -1;
}

configureCollisionBehavior(
    robot, startup_config);

Model model = robot.loadModel();
```

Failure of the recovery request terminates the program before a motion command is issued. Further recovery or unlocking is then performed through Franka Desk before restarting the controller.

The function `configureCollisionBehavior` first applies the standard robot behaviour provided by the libfranka example utilities. The custom thresholds defined in `safety.conf` are subsequently assigned when their selector is enabled:

Listing 3.52: Assignment of the configured collision thresholds.

```
void configureCollisionBehavior(
    Robot& robot,
    const ControllerConfig& params) {

  setDefaultBehavior(robot);

  if (!params.use_custom_collision_behavior) {
    return;
  }

  const Array7 torque_acc =
      filledArray7(
          params.collision_torque_acc);

  const Array7 torque_nom =
      filledArray7(
          params.collision_torque_nom);

  const Array6 force_acc =
```

```
20        filledArray6(
21            params.collision_force_acc);
22
23    const Array6 force_nom =
24        filledArray6(
25            params.collision_force_nom);
26
27    robot.setCollisionBehavior(
28        torque_acc, torque_acc,
29        torque_nom, torque_nom,
30        force_acc, force_acc,
31        force_nom, force_nom);
32  }
```

The two joint-torque parameters define the acceleration-phase and nominal thresholds in N m. Each scalar is assigned uniformly to all seven joints. The two Cartesian parameters are similarly assigned to all six force and moment components in the acceleration and nominal phases. The same array is passed for the lower and upper threshold of each group.

The controller returns the calculated joint torque directly to libfranka:

Listing 3.53: Transfer of the calculated torque command to libfranka.

```
1  const Array7 tau_array =
2      vec7ToArray(tau_cmd);
3
4  return Torques(tau_array);
```

Torque boundedness and collision response are therefore handled by the robot-side monitoring and the configured collision behaviour. The application adds no separate saturation of the Cartesian wrench, null-space torque, complete joint-torque command, or joint-torque rate.

3.9.2 Initial Joint Posture

The parameter `q_init_case` selects one of the joint configurations stored in `initial_pose.conf`. The corresponding seven joint angles are assigned to `q_init` when the configuration is loaded.

The phase sequence and both Cartesian hold modes begin from this posture. The start-up interface moves the robot to it using the libfranka `MotionGenerator`:

Listing 3.54: Motion to the configured initial joint posture.

```
1  const auto move_to_q_init = [&]() {
2    const Vec7 q_init =
3        Map<const Vec7>(params.q_init.data());
4
5    MotionGenerator motion_generator(
6        0.4, params.q_init);
7
```

```
    robot.control(motion_generator);

    q_init_reached = true;
};
```

The speed factor is fixed at 0.4 for this preparation motion. A local flag records whether the posture has already been reached and prevents repeated motion when the operator performs several preparation actions before selecting a run mode.

Selection of the phase sequence, standard pose hold, or set-up-impedance hold calls `ensure_q_init`. Manual guidance forms the alternative start path and begins from the current robot pose. The guided pose is captured before the subsequent sequence or hold mode starts.

3.9.3 Gripper Verification

The start-up interface provides separate commands for opening, grasping, and recalibrating the Franka Hand. Each action reads the gripper state before and after the command and verifies the resulting finger position.

For opening, successful verification requires the libfranka motion command to finish, the calibrated maximum stroke to contain the requested width, and the measured final width to reach the target within a 2 mm tolerance. Grasp verification requires the grasp command to finish, the `is_grasped` flag to be active, and the measured width to lie within the configured inner and outer tolerances.

Before either action, the reported maximum finger width is compared with the largest width used by the controller. A smaller calibrated stroke indicates that gripper homing was performed while an object restricted the finger motion. The start-up interface then requests recalibration with empty fingers.

A configured gripper action can also be executed automatically immediately before torque control. A previously verified manual action is retained and is therefore not repeated. When the requested action is grasping, an already verified tool grasp is also retained:

Listing 3.55: Selection of the automatic gripper action before torque control.

```
if (params.perform_gripper_action_before_run &&
    !params.start_with_manual_guidance &&
    !params.startup_gripper_action_completed) {

  Gripper gripper(params.robot_ip);

  if (params.gripper_startup_grasp_tool) {
```

```
    const franka::GripperState state =
        gripper.readOnce();

    const bool already_holding =
        state.is_grasped &&
        state.width >=
            params.gripper_grasp_width
            - params.gripper_grasp_epsilon_inner &&
        state.width <=
            params.gripper_grasp_width
            + params.gripper_grasp_epsilon_outer;

    gripper_ok =
        already_holding
            ? true
            : graspTool(params, gripper);
  } else {
    gripper_ok =
        openGripper(params, gripper);
  }
}
```

The parameter `abort_on_gripper_action_failure` determines whether a failed automatic action terminates the program or allows the selected controller session to continue.

3.9.4 Tool Transfer

The controller also provides an optional sequence for collecting the tool from its holder or returning it. The holder configuration is represented by the joint posture `q_pickup`. A stand-off posture is calculated at the same end-effector orientation and at the configured distance along $-z_{\mathrm{EE}}$.

The stand-off configuration is obtained through damped least-squares inverse kinematics. Each iteration calculates the position and orientation error, forms the current end-effector Jacobian, and limits the joint-update norm. The resulting posture is accepted only when all joints remain within their configured limits with a 0.05 rad margin.

After successful calculation, the tool-transfer path is executed as

$$q_{\mathrm{init}} \longrightarrow q_{\mathrm{above}} \longrightarrow q_{\mathrm{pickup}} \longrightarrow q_{\mathrm{above}}.$$

The corresponding implementation is:

Listing 3.56: Joint-space motion sequence for collecting or returning the tool.

```
Array7 q_above{};

if (!solveStandoffPosture(
        model,
        robot.readOnce(),
        params.q_pickup,
        params.pickup_standoff,
        q_above)) {
  return false;
```

```
}

int bad_joint = 0;
if (!withinJointLimits(
        q_above, bad_joint)) {
  return false;
}

ensure_q_init();

MotionGenerator to_above(
    0.4, q_above);
robot.control(to_above);

MotionGenerator descend(
    std::max(
        0.05,
        params.pickup_descend_speed_factor),
    params.q_pickup);
robot.control(descend);

const bool action_ok =
    grasp
        ? graspTool(params, gripper)
        : openGripper(params, gripper);

MotionGenerator lift(
    std::max(
        0.05,
        params.pickup_descend_speed_factor),
    q_above);
robot.control(lift);
```

When collecting the tool, the gripper is opened and verified before the arm moves toward the holder. At the pickup posture, the selected gripper action is executed and verified before the robot returns to the stand-off posture.

3.9.5 Exception Handling

Robot communication, motion-generation, and torque-control errors are reported through `franka::Exception`. Configuration, file, and numerical errors are reported through `std::exception`. Both exception types leave the active control operation and terminate the program with an error code.

A robot-side safety event can therefore end the torque callback independently of the controller phase or experiment timer. The corresponding exception is reported after control has stopped, and the robot is recovered through Franka Desk before a new program execution.

3.10 Data Recording and Run Evaluation

The controller records the quantities required for the subsequent run evaluation in a bounded in-memory ring buffer. Its memory is allocated before `robot.control` starts, so

the real-time callback performs no dynamic buffer growth.

Listing 3.57: Allocation of the bounded logging buffer before torque control.

```
const int log_every_n_cycles =
    std::max(
        1,
        params.log_every_n_cycles);

const std::size_t max_log_rows =
    static_cast<std::size_t>(
        std::max(
            0,
            params.max_log_rows));

std::vector<LogData> log_data(
    max_log_rows);

std::size_t control_cycle_count = 0;
std::size_t log_write_index = 0;
std::size_t log_rows_written = 0;
bool log_buffer_wrapped = false;
```

The parameter `log_every_n_cycles` defines the sampling interval in control cycles, while `max_log_rows` defines the maximum number of samples retained in memory. The current baseline records every control cycle and allocates space for 120 000 rows.

The structure `LogData` stores one sample of the measured state, controller reference, calculated command, and active controller mode. The recorded quantities are summarised in table 3.3.

Table 3.3: Signal groups stored during the real-time control loop.

Signal group	Recorded quantities
Time and controller state	Run time, encoded control phase, and encoded null-space mode.
TCP state and reference	Measured and desired TCP positions, translational and rotational errors, measured linear and angular velocities, and desired linear velocity.
Tool and surface geometry	Current controlled tool-feature position, feature offset in the end-effector frame, projected clearance point, TCP position at the clearance transition, desired feature target, and virtual surface penetration.
Alignment	Tool-axis alignment-error components along $[t_1, t_2, n_s]$ and their Euclidean norm.
Cartesian wrench	Commanded force and moment, model-estimated external force and moment, and the force and moment biases captured at the clearance transition.
Null-space controller	Active null-space damping coefficient and the Euclidean norm of the calculated null-space torque.
Disturbance experiment	Disturbance scale, torque-limit scale, force-application point, commanded Cartesian force, and equivalent joint torque.
Joint-torque command	The seven final commanded joint torques returned to libfranka.

The internal variables and CSV columns named `first_contact_tcp` and `first_contact_point` refer to the geometric clearance transition described in Section 3.8.2. They represent the captured TCP position and projected tool-feature point at this transition, rather than a force-detected contact event.

The selected values are copied into the current ring-buffer row at the configured sampling interval:

Listing 3.58: Assignment of one sampled controller state to the ring buffer.

```
1| ++control_cycle_count;
```

```

if (max_log_rows > 0 &&
    control_cycle_count
        % static_cast<std::size_t>(
              log_every_n_cycles)
        == 0) {

  LogData& row =
      log_data[log_write_index];

  row.time = time;
  row.phase = static_cast<int>(phase);
  row.nullspace_mode =
      static_cast<int>(
          params.nullspace_mode);

  row.p_EE = p_EE;
  row.p_d = desired.p_d;
  row.tool_contact_point =
      tool_contact_point;
  row.first_contact_tcp =
      first_contact_tcp;
  row.first_contact_point =
      first_contact_point;
  row.edge_target = edge_target_log;
  row.tool_contact_offset_ee =
      tool_contact_offset_ee;

  row.e_p = e_p;
  row.e_R = e_R;
  row.alignment_error_surface =
      R_base_surface.transpose()
      * toolSurfaceAlignmentErrorBase(
            params,
            R_EE,
            R_base_surface);
  row.alignment_angle =
      row.alignment_error_surface.norm();

  row.pdot = pdot;
  row.pdot_d = desired.pdot_d;
  row.omega = omega;

  row.f = f;
  row.m = m;
  row.external_force = external_force;
  row.external_moment = external_moment;
  row.contact_force_bias =
      contact_force_bias;
  row.contact_moment_bias =
      contact_moment_bias;

  row.push = push_log;
  row.disturbance = disturbance;
  row.tau_nullspace_norm =
      tau_nullspace.norm();
  row.nullspace_damping =
      params.nullspace_damping;
  row.tau_cmd = tau_cmd;
}
```

The current implementation represents the null-space action through the active mode, damping coefficient, and resulting torque norm. These quantities permit the secondary torque to be distinguished from the Cartesian and disturbance contributions during post-processing.

After assigning one row, the write index is advanced cyclically:

Listing 3.59: Advancement of the bounded ring-buffer index.

```
log_write_index =
    (log_write_index + 1)
```

```
    % max_log_rows;

if (log_rows_written < max_log_rows) {
  ++log_rows_written;
} else {
  log_buffer_wrapped = true;
}
```

Once the allocated capacity is reached, new samples replace the oldest rows. The buffer therefore retains the most recent `max_log_rows` samples instead of increasing its memory requirement during control.

3.10.1 Chronological Reconstruction and CSV Output

After the torque callback has finished, the ring-buffer contents are converted to chronological order. When the buffer has wrapped, the oldest retained sample begins at `log_write_index`:

Listing 3.60: Reconstruction of the chronological sample order after control.

```
std::vector<LogData> ordered_log_data;
ordered_log_data.reserve(
    log_rows_written);

if (log_buffer_wrapped) {
  ordered_log_data.insert(
      ordered_log_data.end(),
      log_data.begin()
          + static_cast<std::ptrdiff_t>(
                log_write_index),
      log_data.end());

  ordered_log_data.insert(
      ordered_log_data.end(),
      log_data.begin(),
      log_data.begin()
          + static_cast<std::ptrdiff_t>(
                log_write_index));
} else {
  ordered_log_data.insert(
      ordered_log_data.end(),
      log_data.begin(),
      log_data.begin()
          + static_cast<std::ptrdiff_t>(
                log_rows_written));
}

result.log =
    std::move(ordered_log_data);
```

The ordered samples and final controller state are returned in a `RunResult`. File output is subsequently performed by `writeRunLogs`, outside the real-time callback:

Listing 3.61: Post-run CSV output and terminal summary.

```
void writeRunLogs(
    const ControllerConfig& params,
    const RunResult& result) {

  if (result.descend_failed) {
    printSection("descend stopped");
```

```
    printf(
        "  maximum distance reached "
        "before the clearance height.\n");
  }

  printJointStartEndTableDeg(
      result.q_start,
      result.final_q);

  writeLogToCsv(
      result.log,
      params.csv_file_name);

  printFinalSummary(
      result.final_p_d,
      result.final_p_EE,
      result.final_e_p,
      result.final_e_R,
      params.csv_file_name);
}
```

The CSV writer creates the parent output directory when required and uses a fixed column order. Cartesian positions, velocities, forces, moments, disturbances, and joint torques are written in their stored SI units. Alignment errors are converted from radians to degrees for the output file.

The bias-subtracted external-force and external-moment columns are calculated during CSV writing from the stored estimates:

Listing 3.62: Writing the bias-subtracted external wrench to the CSV file.

```
writeVec3(
    log_file,
    row.external_force
        - row.contact_force_bias);

writeVec3(
    log_file,
    row.external_moment
        - row.contact_moment_bias);
```

The commanded Cartesian wrench and the model-estimated external wrench are therefore retained as separate quantities. The former is generated by the impedance controller, while the latter is supplied by the robot state and used for set-up evaluation.

When several controller sessions are executed without restarting the program, the session number is inserted before the CSV filename extension:

Listing 3.63: Generation of separate CSV filenames for repeated sessions.

```
if (session > 1) {
  config.csv_file_name =
      sessionFileName(
          config.csv_file_name,
          session);
}
```

For example, a second session based on `controller_log.csv` is written as `controller_log_s2.csv`.

3.10.2 Set-Up Evaluation

A dedicated set-up report is generated once when the moment threshold or set-up time-out is reached. The report captures the final TCP pose, selected tool-feature position, external wrench change, clearance-transition references, active gain matrices, and set-up termination condition.

The external moment is also transferred from the TCP to the selected tool feature. The implementation uses the bias-subtracted external force and moment:

Listing 3.64: Calculation of the external moment at the selected tool feature.

```
const Vec3 edge_from_tcp =
    tool_contact_point - p_EE;

const Vec3 contact_moment_at_edge =
    (external_moment
     - contact_moment_bias)
    -
    edge_from_tcp.cross(
        external_force
        - contact_force_bias);
```

The terminal set-up report contains:

- the set-up termination condition and phase duration;
- the external-force and external-moment changes;
- the TCP and tool-feature displacements from the clearance transition;
- the tool-alignment error before and after set-up;
- the alignment-error components along t_1, t_2, and n_s;
- the force, moment, displacement, and rotation resolved in the surface frame; and
- the active translational and rotational stiffness and damping matrices.

The displayed force–displacement and moment–angle ratios provide a pointwise diagnostic of the completed set-up state. They are calculated only when the corresponding displacement or rotation exceeds its numerical minimum.

When compliance-centre diagnostics are enabled, the report additionally resolves r_c in surface coordinates and p_c in end-effector coordinates. It reconstructs the shifted TCP stiffness and damping matrices and reports their diagonal entries, largest translation–rotation coupling terms, and eigenvalue ranges. The complete 6×6 matrices can also be printed for verification of the configured lever and gain transformation.

The final post-run summary displays the joint displacement between the start and end

configurations, the desired and measured final TCP positions, the final translational and rotational errors, and the path of the generated CSV file.

The controller implementation described in this chapter forms the basis of all reported experiments. Chapter 4 presents the calibration procedure, the controller parameters used during the measurements, the varied experimental conditions, and the quantities derived from the recorded signals.

Chapter 4

Experimental Methodology

PURPLE — revised and confirmed. The chapter was revised in full and every paragraph is settled. No further editing is to be applied to it, and any remaining concern is raised as a comment rather than as a change.

This chapter presents the experimental configuration, calibration procedure, controller parameters, test sequence, and evaluation quantities used to characterise the contact-induced alignment of the grinding tool. The main test series consists of surface-contact measurements in which the commanded tilt, directional Cartesian stiffness, and virtual centre of compliance were varied under otherwise matched conditions. A separate Cartesian pose-hold study was used to examine the two null-space torque contributions without surface contact.

4.1 Experimental Design

The surface-contact measurements were divided into five cases. Case A established the response to commanded angular offsets about the two calibrated surface tangents. Case B varied the rotational stiffness about the tilted tangent, while Case C varied the translational stiffness perpendicular to that tangent. Case D changed the position and sign of the virtual-centre-of- compliance lever. Case E applied the angular offset about the grinding-face axes and compared the resulting motion with the corresponding surface-tangent conditions.

The commanded angular offset defines the orientation difference present at the begin-

ning of set-up. The rotational stiffness determines the resistance to rotation about the tilted tangent, whereas the translational stiffness perpendicular to that tangent affects the translational response coupled to the alignment motion. In Case D, shifting the Cartesian impedance from a selected centre of compliance to the TCP introduces translation–rotation coupling and therefore changes the moment produced by the commanded press.

The surface plane and the grinding-face direction were calibrated independently and used to define the surface and tool geometry throughout the measurements. The rotational stiffness about t_1 and t_2 was varied separately, while the orthogonal rotational-stiffness entry remained at its baseline value. This arrangement allowed the response associated with each surface tangent to be evaluated independently.

Unless stated otherwise for a particular case, the translational stiffness used during set-up and grinding was

$$K_p = \begin{bmatrix} 2000 & 0 & 0 \\ 0 & 2000 & 0 \\ 0 & 0 & 800 \end{bmatrix} \text{N/m}, \tag{4.1}$$

expressed in the surface frame. The corresponding rotational stiffness was

$$K_R = \begin{bmatrix} 5 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 50 \end{bmatrix} \text{N m/rad}, \tag{4.2}$$

also in the surface frame. Both matrices use the coordinate order $[t_1, t_2, n_s]$. The fixed phase, damping, null-space, and trajectory parameters are specified in Section 4.4.

Table 4.1: Parameter variations used in the surface-contact measurements.

Case	Varied quantity	Tilt condition	Tested values	n	Comparison
A	Commanded angular offset	About t_1 or t_2	0°, +5°, and +10° about t_1; +5° and +10° about t_2	15	Reference response to the commanded tilt about each surface tangent.
B	Rotational stiffness	+10° about t_1 or t_2	15 and 50 N m/rad about the tilted tangent. The orthogonal tangent remained at 5 N m/rad; the matched 5 N m/rad reference was taken from Case A.	12	Tangent-specific influence of rotational stiffness on the alignment response.
C	Translational stiffness	+10° about t_1 or t_2	300 and 800 N/m perpendicular to the tilted tangent. The matched 2000 N/m reference was taken from Case A. One additional condition for each tilt direction combined 300 N/m with 50 N m/rad.	18	Influence of the cross-axis translational stiffness and its interaction with rotational stiffness.
D	Virtual centre of compliance	+10° about t_1 or t_2	Perpendicular surface-tangent lever of −60, 0, and +60 mm. The grinding-face-centre settings were $(0, -3.5, 19.7)$ mm for the t_1 measurements and $(3.5, 0, 19.7)$ mm for the t_2 measurements.	24	Influence of lever position and sign on the coupled alignment response.
E	Axis used for the commanded tilt	+10° about y_{EE} or x_{EE}	The y_{EE} tilt corresponds to $t_1 = -4.23°$ and $t_2 = +9.06°$. The x_{EE} tilt corresponds to $t_1 = +9.06°$ and $t_2 = +4.23°$.	6	Comparison of tilts defined by the surface tangents and by the grinding-face axes.

Each parameter setting was repeated three times. Cases A, B, C, and E used the decoupled Cartesian impedance. Case D used the point-shifted impedance with the listed levers expressed in the surface frame according to $r_c = p_{\mathrm{TCP}} - p_c$. The baseline gains, calibrated geometry, phase sequence, press motion, and null-space settings were retained except where a varied value is stated explicitly.

The five cases contain 75 surface-contact runs distributed across 25 parameter settings. Within each controlled comparison, only the stated parameter was changed. The remaining gain entries, calibrated surface and tool geometry, phase sequence, and evaluation procedure were retained.

The principal evaluation quantity was the change in the angle between the calibrated surface normal and the grinding-face direction calculated from the measured end-effector orientation. The rotation during set-up, the displacement of the point or edge of the grinding face used for the set-up motion, the TCP displacement, and the model-estimated external load were evaluated alongside this angle. Their definitions and treatment across repeated runs are presented in Section 4.6.

The surface-contact measurements and the Cartesian pose-hold study were treated as separate data sets. The pose-hold study described in Section 4.7 applies an internally commanded disturbance and examines projected null-space damping and singular-value conditioning apart from the surface-contact response.

4.2 Experimental System

4.2.1 Robot, Tool, and Surface

The experiments were performed with a seven-degree-of-freedom Franka Emika Panda operated through the Franka Control Interface. Joint-torque commands were transmitted through libfranka at the nominal control period

$$T_s = 1\,\mathrm{ms}, \tag{4.3}$$

corresponding to a nominal control rate of 1 kHz. The real-time controller and its phase sequence are described in Chapter 3.

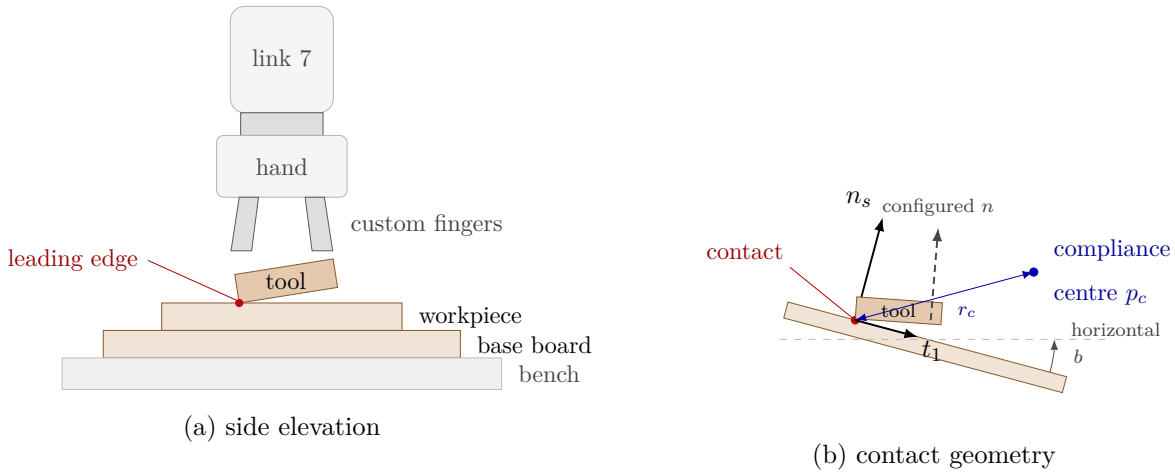
The rectangular grinding tool was held by the Franka Hand using custom gripper fingers. Its grinding face measured 40 mm along X_{EE} and 120 mm along Y_{EE}. The centre of the grinding face was located 20 mm along $+Z_{\mathrm{EE}}$ from the reported end-effector origin. The complete calibrated tool geometry is presented in Section 4.3.

The gripper permitted approximately $\pm 2^{\circ}$ of relative tool rotation about y_{EE}. Conse-

quently, the grinding-face direction used in the evaluation was calculated from the measured end-effector orientation and the calibrated grinding-face direction in the end-effector frame. Relative motion between the tool and the gripper contributes to the measurement uncertainty.

The surface plate was mounted on a raised base board. All experiments used dry contact between the grinding face and the plate. The controller included no explicit environment-friction model. The surface plane and grinding-face direction were calibrated separately before the experimental campaign, as described in Section 4.3.

During approach, the controller determines which point or edge of the rectangular grinding face lies furthest in the descent direction. This point or edge is used for the geometric clearance calculation and the subsequent set-up motion. Its position was calculated from the measured end-effector pose and the calibrated tool geometry.



(a) side elevation

(b) contact geometry

Figure 4.1: Experimental system and surface-frame contact geometry.

4.2.2 Software and Real-Time Computer

The controller computer ran Ubuntu 20.04.3 with the `5.9.1-rt20` Linux kernel and the `PREEMPT_RT` patch. The controller was compiled using `g++ 9.3`, the C++14 language standard, and the `-O2` optimisation level. Eigen 3.3.7 and libfranka 0.7.1 were used for the matrix calculations and communication with the robot.

The controller operated through the libfranka joint-torque interface. At each control cycle, the measured robot state and the quantities provided by the robot model were used to calculate the commanded joint torque. The implementation of the real-time callback

is documented in Section 3.2.

Each contact run followed the implemented sequence of tool orientation, surface approach, set-up, and grinding. Pose hold and manual guidance were used only for preparation or separate experiments. The combined null-space mode remained active throughout the contact sequence.

4.2.3 Recorded Experimental Quantities

The quantities required for the subsequent evaluation were written to the controller's in-memory logging buffer during each run and exported to a CSV file after torque control had ended. The recorded quantities included:

- the measured end-effector position and orientation;
- the measured joint position and velocity;
- the commanded Cartesian reference and Cartesian error;
- the position of the point or edge of the grinding face used for the set-up motion;
- the active controller phase and null-space mode;
- the commanded Cartesian wrench and final joint-torque command; and
- the model-estimated external force and moment reported by libfranka.

The commanded wrench and the model-estimated external wrench were retained as separate quantities. The latter was used as a supporting measure of the contact response and represents the model-based estimate supplied by libfranka rather than a direct force measurement. The complete logging implementation and CSV structure are described in Section 3.10.

4.2.4 Reproducibility Record

The documented experimental system is summarised in Section 4.2.4. Controller parameters that differ between the individual experimental cases are reported separately in Section 4.4.

Table 4.2: Documented configuration of the experimental system.

Item	**Configuration used in the reported experiments**
Robot	Franka Emika Panda with seven actuated joints.
Robot interface	libfranka joint-torque interface with a nominal control period of 1 ms.
Gripper and tool mounting	Franka Hand with custom gripper fingers and an approximately $\pm 2^\circ$ relative rotational play about y_{EE}.
Grinding tool	Rectangular grinding face measuring 40×120 mm, with its centre located 20 mm along $+Z_{\mathrm{EE}}$ from the reported end-effector origin.
Surface	Plane surface mounted on a raised base board and used under dry-contact conditions.
Operating system	Ubuntu 20.04.3.
Real-time kernel	Linux `5.9.1-rt20` with `PREEMPT_RT`.
Compiler	`g++ 9.3`, C++14, optimisation level `-O2`.
Software libraries	Eigen 3.3.7 and libfranka 0.7.1.
Recorded state quantities	Robot pose, joint position and velocity, Cartesian reference and error, controller phase, and position of the grinding-face point or edge used for the set-up motion.
Recorded command and load quantities	Commanded Cartesian wrench, commanded joint torque, and the model-estimated external force and moment reported by libfranka.
Contact condition	Dry contact; no explicit environment-friction model was included in the controller.

The software versions, nominal control period, tool dimensions, and surface calibration were documented. The real-time-computer hardware, payload parameters, and complete robot transform configuration were not retained in the experimental record. Their influence on exact dynamic and timing reproducibility is considered in Chapter 6.

4.3 Surface and Tool Calibration

The surface plane and the direction normal to the grinding face were calibrated independently before the contact experiments. The surface calibration provided the point, normal direction, and tangent directions used by the controller. The tool calibration provided the grinding-face direction expressed in the end-effector frame.

The calibrated quantities were written to the controller parameter files and were subsequently used for approach, clearance calculation, commanded tool orientation, gain orientation, compliance-centre placement, and evaluation of the experimental results.

4.3.1 Surface-Plane Calibration

Three positions distributed over the plate were probed to determine the surface plane. A fourth position was reserved as an independent geometric check. At each position, the point on the surface was calculated from the measured end-effector pose and the calibrated centre of the grinding face.

The plane used by the controller is represented by

$$n_s^\top (p - p_s) = 0, \tag{4.4}$$

where p_s is one point on the plane and n_s is its unit normal, expressed in the robot base frame.

The final calibrated plane point stored in the controller parameters was

$$p_s = \begin{bmatrix} 0.5153 \\ -0.1072 \\ 0.0031 \end{bmatrix} \text{m}. \tag{4.5}$$

This point represents the position of the seated grinding-face centre used during calibration.

The controller stores the surface orientation through rotations a about the base x-axis and b about the base y-axis. The corresponding normal is calculated from

$$n_s = R_y(b)R_x(a)e_z = \begin{bmatrix} \sin(b)\cos(a) \\ -\sin(a) \\ \cos(b)\cos(a) \end{bmatrix}. \tag{4.6}$$

The calibrated angles in the final controller configuration were

$$a = -1.585^\circ, \qquad b = +0.988^\circ. \tag{4.7}$$

These values give the base-frame surface normal

$$n_s \approx \begin{bmatrix} 0.017245 \\ 0.027663 \\ 0.999469 \end{bmatrix}, \qquad \|n_s\| = 1. \tag{4.8}$$

The perpendicular distance of the independently probed fourth point p_4 from the calibrated plane was calculated as

$$\varepsilon_{\text{plane}} = \left| n_s^\top (p_4 - p_s) \right|. \tag{4.9}$$

The resulting validation residual was

$$\varepsilon_{\text{plane}} = 0.82\,\text{mm}. \tag{4.10}$$

The first surface tangent is taken from the calibration itself. The direction a_s from the first to the second probed point is projected into the calibrated plane and normalised:

$$t_1 = \frac{(I - n_s n_s^\top)a_s}{\|(I - n_s n_s^\top)a_s\|}. \tag{4.11}$$

The tangent therefore follows the probed geometry rather than a base-frame axis, and the surface frame is rotated in the plane relative to the base x- and y-directions.

The second tangent completes the right-handed surface frame:

$$t_2 = n_s \times t_1. \tag{4.12}$$

For the calibrated normal, the resulting tangent directions are approximately

$$t_1 \approx \begin{bmatrix} -0.905786 \\ -0.422854 \\ 0.027332 \end{bmatrix}, \qquad t_2 \approx \begin{bmatrix} 0.423385 \\ -0.905776 \\ 0.017765 \end{bmatrix}. \tag{4.13}$$

The complete surface frame is therefore

$$R_{\text{surface}} = \begin{bmatrix} t_1 & t_2 & n_s \end{bmatrix} \approx \begin{bmatrix} -0.905786 & 0.423385 & 0.017245 \\ -0.422854 & -0.905776 & 0.027663 \\ 0.027332 & 0.017765 & 0.999469 \end{bmatrix}. \tag{4.14}$$

The surface normal lies approximately 1.9° from the base z-direction, while the tangents are rotated substantially within the plane relative to the base x- and y-directions.

All surface-frame quantities in the contact campaign use the coordinate order

$$[\, t_1, t_2, n_s \,]. \tag{4.15}$$

This order applies to commanded angular offsets, rotational stiffness, surface-resolved displacement and rotation, and the compliance-centre levers used in Case D.

4.3.2 Grinding-Face Calibration

A separate set of seatings was used to establish the relationship between the end-effector orientation and the direction normal to the grinding face. The final controller configuration represents this direction by

$$a_{\text{EE}} = \begin{bmatrix} -0.006011 \\ 0.000246 \\ 0.999982 \end{bmatrix}, \qquad \|a_{\text{EE}}\| = 1, \tag{4.16}$$

expressed in the end-effector frame. The direction departs from $+Z_{\text{EE}}$ by 0.34°, which is the residual seating offset of the tool in the gripper.

An additional seating was reserved as an independent validation measurement. The an-

gular residual between the grinding-face direction calculated from this end-effector orientation and the calibrated surface normal was

$$\varepsilon_{\text{tool}} = 0.50^\circ. \tag{4.17}$$

During each controller cycle, the grinding-face direction in the robot base frame is calculated from the measured end-effector orientation:

$$a_T = R_{\text{EE}} a_{\text{EE}}. \tag{4.18}$$

The direction a_T is therefore calculated from the measured end-effector orientation and the calibrated grinding-face direction. It is not obtained from an independent orientation sensor on the tool.

The commanded flat orientation turns the grinding face toward the calibrated surface according to

$$R_d a_{\text{EE}} = -n_s. \tag{4.19}$$

The negative sign follows from the configured value

$$s_a = -1, \tag{4.20}$$

which directs the positive grinding-face axis opposite the outward surface normal. The remaining rotation about the grinding-face direction was set by a commanded offset of 115.05° about the surface normal and was held at that value in every reported run. The offset places y_{EE} approximately 25° from t_2.

The parallel-jaw mounting permitted approximately ±2° of relative tool rotation about y_{EE}. This relative motion contributes to the uncertainty of the alignment calculated from the end-effector orientation.

4.3.3 Calibrated Grinding-Face Geometry

The grinding face was represented by a rectangle expressed in the end-effector frame. Its

centre was configured as

$$r_{\text{face,EE}} = \begin{bmatrix} 0 \\ 0 \\ 0.020 \end{bmatrix} \text{m}. \tag{4.21}$$

The half-width and half-length vectors were

$$r_{\text{width,EE}} = \begin{bmatrix} 0.020 \\ 0 \\ 0 \end{bmatrix} \text{m}, \qquad r_{\text{length,EE}} = \begin{bmatrix} 0 \\ 0.060 \\ 0 \end{bmatrix} \text{m}. \tag{4.22}$$

The complete grinding face therefore measured

$$40\,\text{mm} \times 120\,\text{mm}, \tag{4.23}$$

with the width along X_{EE}, the length along Y_{EE}, and its centre 20 mm along $+Z_{\text{EE}}$ from the reported end-effector origin.

For a point on the grinding face with end-effector-frame offset $r_{g,\text{EE}}$, its base-frame position is calculated from

$$p_g = p_{\text{EE}} + R_{\text{EE}} r_{g,\text{EE}}. \tag{4.24}$$

The controller transforms all four corners of the grinding face into the robot base frame and compares their positions along the descent direction. The point or edge of the grinding face that approaches the surface first is then used for the geometric clearance calculation and the subsequent set-up motion.

Projected corner heights differing by at most

$$\varepsilon_g = 0.0001\,\text{m} = 0.1\,\text{mm} \tag{4.25}$$

are treated as belonging to the same edge or face. Their positions are averaged to form one geometric reference. This prevents a nominal edge configuration from switching between its two endpoints because of small numerical differences.

In the following sections, this geometry-based reference is called the point or edge of the grinding face used for the set-up motion. Its position is used for the clearance calculation, commanded set-up displacement, and reported grinding-face displacement.

The point or edge is obtained entirely from the measured end-effector pose, the calibrated grinding-face geometry, and the descent direction. The controller therefore uses a geometric surface reference rather than a separately measured physical contact position. The implementation of the corner transformation and geometric selection is described in Section 3.5.

4.4 Controller Parameters Used in the Experiments

The controller parameters were assigned according to the role of each phase. Approach used a comparatively soft isotropic Cartesian impedance in the surface frame. The operator gate applied a stiffer translational hold at the clearance pose. Set-up and grinding used lower stiffness along base z than along base x and y, together with lower rotational stiffness about the surface tangents than about the surface normal. The coordinate frame therefore forms part of every gain definition.

4.4.1 Fixed Phase and Motion Parameters

The phase-transition and trajectory parameters retained during the surface-contact measurements are listed in Table 4.3. The set-up push coordinate starts at the signed distance captured at the clearance transition, approximately $-0.020\,\mathrm{m}$, and moves toward $+0.060\,\mathrm{m}$. At the configured speed, the reference reaches this endpoint after approximately $1.6\,\mathrm{s}$ and is then held there until the phase ends. The endpoint is a Cartesian spring reference beyond the calibrated plane and is not interpreted as physical penetration of the same magnitude.

Table 4.3: Fixed phase-transition and trajectory parameters.

Quantity	**Value**	**Role**
Minimum orientation time	0.5 s	Prevents an immediate phase transition.
Tool-axis transition tolerance	0.035 rad	Maximum remaining tool-axis error before descent.
Descent speed	0.1 m/s	Reference speed along the descent direction.
Maximum descent distance	0.640 m	Terminates an unsuccessful approach.
Surface clearance	0.020 m	Geometric hand-off from approach to set-up.
Minimum set-up time	2.0 s	Earliest activation of the moment-based exit condition.
Set-up timeout	5.0 s	Time-based set-up termination.
Moment-change threshold	150 N m	Alternative set-up termination based on the model-estimated external moment.
Set-up push speed	0.050 m/s	Rate of the signed press coordinate.
Set-up push endpoint	+0.060 m	Final signed Cartesian spring reference.
Grinding sweep	disabled	No tangential reference was added after set-up.
Pre-set-up gate	enabled	Holds the clearance pose until operator confirmation.
Pre-grind gate	enabled	Holds the set-up pose until operator confirmation.

4.4.2 Phase-Dependent Cartesian Stiffness

During tool orientation and surface approach, the translational and rotational stiffness matrices were isotropic in the calibrated surface frame:

$$K_{p,\mathrm{app}} = \begin{bmatrix} 150 & 0 & 0 \\ 0 & 150 & 0 \\ 0 & 0 & 150 \end{bmatrix} \mathrm{N/m}, \qquad K_{R,\mathrm{app}} = \begin{bmatrix} 90 & 0 & 0 \\ 0 & 90 & 0 \\ 0 & 0 & 90 \end{bmatrix} \mathrm{N\,m/rad}. \tag{4.26}$$

At the pre-set-up gate, the translational stiffness was increased in the robot base frame while the approach rotational stiffness was retained:

$$K_{p,\mathrm{gate}} = \begin{bmatrix} 5000 & 0 & 0 \\ 0 & 5000 & 0 \\ 0 & 0 & 5000 \end{bmatrix} \mathrm{N/m}, \qquad K_{R,\mathrm{gate}} = \begin{bmatrix} 90 & 0 & 0 \\ 0 & 90 & 0 \\ 0 & 0 & 90 \end{bmatrix} \mathrm{N\,m/rad}. \tag{4.27}$$

The translational matrix used during set-up and grinding was expressed in the surface frame,

$$K_{p,\mathrm{set}} = K_{p,\mathrm{grind}} = \begin{bmatrix} 2000 & 0 & 0 \\ 0 & 2000 & 0 \\ 0 & 0 & 800 \end{bmatrix} \mathrm{N/m}, \tag{4.28}$$

as was the rotational matrix,

$$K_{R,\mathrm{set}} = K_{R,\mathrm{grind}} = \begin{bmatrix} 5 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 50 \end{bmatrix} \mathrm{N\,m/rad}. \tag{4.29}$$

The lower rotational tangent entries allow rotation about t_1 and t_2, while the larger normal entry resists twist about n_s. Because both matrices are diagonal in the surface frame, the translational stiffness along the surface normal is the normal entry itself,

$$k_{p,n} = n_s^{\top} K_{p,\mathrm{set}} n_s = 800\,\mathrm{N/m}, \tag{4.30}$$

and the tangential entries act along t_1 and t_2 without contributing to the normal direction.

The Cartesian pose-hold study used the isotropic base-frame matrices

$$K_{p,\mathrm{hold}} = \begin{bmatrix} 2000 & 0 & 0 \\ 0 & 2000 & 0 \\ 0 & 0 & 2000 \end{bmatrix} \mathrm{N/m}, \qquad K_{R,\mathrm{hold}} = \begin{bmatrix} 50 & 0 & 0 \\ 0 & 50 & 0 \\ 0 & 0 & 50 \end{bmatrix} \mathrm{N\,m/rad}. \tag{4.31}$$

Manual guidance used joint damping of $0.5\,\mathrm{N\,m\,s/rad}$ and no Cartesian stiffness.

4.4.3 Inertia-Scaled Damping

The directional damping coefficients were calculated at entry to the corresponding phase group from the current Cartesian inertia, the active stiffness, and the configured damping ratio, as described in Section 3.6.2. Approach used $\zeta = 1.9$, while set-up and grinding used $\zeta = 1.0$. The calculated coefficients were retained for the associated phase group.

The pre-set-up gate used separate damping ratios for translation and rotation. They were derived from the configured translational target

$$D_{p,\mathrm{gate}}^{\mathrm{ref}} = \begin{bmatrix} 200 & 0 & 0 \\ 0 & 200 & 0 \\ 0 & 0 & 200 \end{bmatrix} \mathrm{N\,s/m} \tag{4.32}$$

and the rotational damping calculated for approach. For the pose-hold study, one damping ratio was derived from the translational target

$$D_{p,\mathrm{hold}}^{\mathrm{ref}} = \begin{bmatrix} 50 & 0 & 0 \\ 0 & 50 & 0 \\ 0 & 0 & 50 \end{bmatrix} \mathrm{N\,s/m} \tag{4.33}$$

and applied to the translational and rotational inertia-scaled damping calculations.

The configured matrices available when an inertia estimate could not be used were

$$D_{p,\mathrm{app}}^{\mathrm{ref}} = \begin{bmatrix} 20 & 0 & 0 \\ 0 & 20 & 0 \\ 0 & 0 & 20 \end{bmatrix} \mathrm{N\,s/m}, \qquad D_{R,\mathrm{app}}^{\mathrm{ref}} = \begin{bmatrix} 12 & 0 & 0 \\ 0 & 12 & 0 \\ 0 & 0 & 12 \end{bmatrix} \mathrm{N\,m\,s/rad}, \tag{4.34}$$

and

$$D_{p,\mathrm{set}}^{\mathrm{ref}} = \begin{bmatrix} 50 & 0 & 0 \\ 0 & 50 & 0 \\ 0 & 0 & 25 \end{bmatrix} \mathrm{N\,s/m}, \qquad D_{R,\mathrm{set}}^{\mathrm{ref}} = \begin{bmatrix} 10.01 & 0 & 0 \\ 0 & 10.01 & 0 \\ 0 & 0 & 10 \end{bmatrix} \mathrm{N\,m\,s/rad}. \tag{4.35}$$

Both set-up matrices are expressed in the surface frame with the coordinate order $[t_1, t_2, n_s]$.

The factor $\zeta = 1$ gives the critical-damping coefficient of each individual decoupled direction represented by the calculation. It does not establish critical damping of the complete robot–tool–surface system after the compliance-centre shift and contact coupling are included.

4.4.4 Compliance-Centre Settings

Cases A, B, C, and E used the decoupled Cartesian impedance at the TCP. Case D used the point-shifted impedance during set-up. The centre of compliance was specified through the lever

$$r_c = p_{\mathrm{TCP}} - p_c, \tag{4.36}$$

resolved in the surface frame $[t_1, t_2, n_s]$. The lever was captured with the clearance pose and remained constant during the set-up phase. The stiffness and damping matrices were shifted to the TCP according to the transformation derived in Section 2.7.

For a commanded tilt about t_1, the varied component was r_{c,t_2}, while $r_{c,t_1} = 0$. For a commanded tilt about t_2, the varied component was r_{c,t_1}, while $r_{c,t_2} = 0$. The tested perpendicular components were -60, 0, and $+60$ mm, with $r_{c,n} = 0$.

The fourth setting placed the centre of compliance at the centre of the grinding face. The corresponding surface-frame levers were

$$r_c^{(t_1)} = \begin{bmatrix} 0 \\ -3.5 \\ 19.7 \end{bmatrix} \text{mm}, \qquad r_c^{(t_2)} = \begin{bmatrix} 3.5 \\ 0 \\ 19.7 \end{bmatrix} \text{mm}. \tag{4.37}$$

The sign of the perpendicular component determines the direction of the moment generated by the translational response. Reversing this component reverses the corresponding coupling moment about the tilted surface tangent. The measured response is presented only for the tested lever values and does not imply a continuously optimal location.

4.4.5 Null-Space Settings for the Contact Measurements

The null-space torque was restricted to projected damping throughout the surface-contact measurements. The singular-value-conditioning term was not applied. The damping gain was

$$d_{\text{null}} = 2\,\text{N m s/rad}. \tag{4.38}$$

The remaining null-space quantities were configured but inactive in this mode. The conditioning gain, SVD probe step, sign deadband, and relative singular-value threshold were

$$k_\sigma = 1\,\text{N m}, \qquad \alpha_{\text{probe}} = 0.08\,\text{rad}, \qquad \delta_\sigma = 10^{-6}, \qquad \rho = 10^{-4}. \tag{4.39}$$

These parameters were held fixed in Cases A–E. Their role in the null-space torque is described in Section 2.8. The contact measurements therefore characterise the controller with projected null-space damping alone, and the alignment results carry no contribution from singular-value conditioning.

4.5 Contact Experiment Procedure

Before each run, the active parameter set was recorded and the robot was moved to the configured initial joint posture. Damped manual guidance could be used to adjust this posture, after which the selected start pose was captured. The tool was held by the Franka Hand using the configured grasp width of 66 mm and a grasping force of 60 N. The seating

was repeated in the same manner for every measurement, while the approximately $\pm 2^\circ$ rotational play about y_{EE} remained part of the physical mounting condition.

The calibrated surface frame and grinding-face geometry were constructed before torque control began. The active Cartesian gains, damping settings, null-space parameters, and compliance-centre definition were then loaded, and the robot-side collision behaviour was configured.

During tool orientation, the TCP position was held while the calibrated grinding-face direction was rotated toward $-n_s$, including the commanded angular offsets of the active case. The controller remained in this phase for at least 0.5 s and continued to surface approach when the remaining tool-axis error was at most 0.035 rad.

Surface approach moved the Cartesian reference along the descent direction at 0.1 m/s, with a maximum commanded travel of 0.640 m. During this motion, the controller continuously calculated the point or edge of the rectangular grinding face that approached the surface first. The geometric transition was reached when the leading corner height was at most 20 mm above the calibrated plane. Corner heights agreeing within 0.1 mm were averaged according to the geometric selection described in Section 4.3.3.

At the clearance transition, the operator gate held the reached pose until the set-up motion was confirmed. The controller then stored the TCP pose, the orientation reference, the point or edge of the grinding face used for the set-up motion, its projection onto the plane, and the model-estimated external force and moment used as bias values. This transition was defined by the calibrated geometry; the model-estimated force was recorded for evaluation but did not determine the transition.

During set-up, the signed press coordinate moved from its captured value, approximately -0.020 m, toward $+0.060$ m at 0.050 m/s along the descent direction. Cases A, B, C, and E used the decoupled impedance, whereas Case D applied the point-shifted 6×6 stiffness and damping matrices associated with the selected centre of compliance. The orientation stored at the clearance transition remained the rotational reference, so the alignment motion arose from the contact response rather than from a time-varying tipping command.

After the minimum set-up time of 2 s, the phase could end when the norm of the bias-subtracted model-estimated external moment reached 150 N m. The independent 5 s timeout provided the second termination condition. The final signed coordinate is a virtual

Cartesian spring reference and therefore describes the controller command rather than a measured penetration depth.

The pre-grind gate then held the reached pose until the transition to grinding was confirmed. After set-up, the controller retained the final press reference and switched to the decoupled set-up gain matrices. The tangential grinding sweep was disabled for the reported measurements, so grinding maintained the press reference without a commanded tangential motion.

The operator ended the run through the terminal interface. The buffered data were then written to the run-specific CSV file, and the stored parameter set was retained with the measurement. Between the three repetitions of one setting, the calibrated geometry, fixed gain entries, phase timing, and procedure remained unchanged. Only the parameter assigned to the corresponding case was modified.

4.6 Evaluation Metrics and Repeated-Run Treatment

4.6.1 Alignment Change

The grinding-face direction used in the evaluation was calculated from the measured end-effector orientation and the calibrated direction expressed in the end-effector frame:

$$a_T = R_{\mathrm{EE}} a_{\mathrm{EE}}. \tag{4.40}$$

Its angle to the calibrated surface was

$$\theta_{\mathrm{align}} = \cos^{-1}\left(-a_T^{\top} n_s\right). \tag{4.41}$$

The sign convention follows the orientation target $R_d a_{\mathrm{EE}} = -n_s$. The angle is calculated from the measured end-effector orientation and the calibrated tool geometry. Relative rotation of the tool in the gripper therefore contributes to its uncertainty.

The alignment change during set-up was calculated from

$$\Delta\theta_{\text{align}} = \theta_{\text{align,before}} - \theta_{\text{align,after}}, \tag{4.42}$$

where the initial value is taken at the clearance transition and the final value at the end of set-up. Positive values represent a reduction of the angular difference between the grinding face and the calibrated surface.

4.6.2 Rotation, Displacement, and Model-Estimated Load

The finite rotation during set-up was evaluated from

$$\Delta\theta_{\text{set}} = \phi_{\text{set}} u_{\text{set}}. \tag{4.43}$$

Here ϕ_{set} and u_{set} are the rotation angle and the unit rotation axis of the relative rotation $R_{\text{after}} R_{\text{before}}^{\top}$, obtained from its trace and its skew-symmetric part exactly as in Equation (2.16) and Equation (2.20). The vector is expressed in the robot base frame. Its magnitude is the angle turned during set-up, in radians, and its direction is the axis about which that rotation occurred.

Its components in the surface frame were obtained from

$$\Delta\theta_{\text{set}}^{S} = R_{\text{surface}}^{\top} \Delta\theta_{\text{set}}. \tag{4.44}$$

This representation separates the rotations about t_1, t_2, and n_s and permits direct comparison with the commanded tilt direction.

Let p_g denote the position of the point or edge of the grinding face used for the set-up motion. Its displacement was

$$\Delta p_g = p_{g,\text{after}} - p_{g,\text{before}}, \qquad s_g = \|\Delta p_g\|. \tag{4.45}$$

The TCP displacement was evaluated in the same form:

$$\Delta p_{\text{TCP}} = p_{\text{TCP,after}} - p_{\text{TCP,before}}, \qquad s_{\text{TCP}} = \|\Delta p_{\text{TCP}}\|. \tag{4.46}$$

The surface-frame components of both displacement vectors were retained to show whether the angular change was accompanied by normal motion or tangential sliding.

The supporting load quantity was calculated from the model-estimated external force reported by libfranka after subtraction of the bias captured at the clearance transition. Its normal magnitude was

$$F_n = \left| n_s^\top \left(\hat{f}_{\text{ext}} - \hat{f}_{\text{bias}} \right) \right|. \tag{4.47}$$

This quantity represents the robot-model estimate associated with the external wrench. It was used to compare the load level between matched runs and was not treated as a direct force measurement.

The controller orientation error e_R is referenced to the orientation stored at the clearance transition. It therefore describes deviation from the held controller reference, while θ_{align} describes the relationship between the grinding face and the calibrated surface. The latter was used as the principal alignment quantity.

4.6.3 Treatment of Repeated Runs

Each parameter setting was repeated three times. For a measured quantity y, the three values were summarised by the arithmetic mean

$$\bar{y} = \frac{1}{3} \sum_{j=1}^{3} y_j \tag{4.48}$$

and the sample standard deviation

$$s_y = \sqrt{\frac{1}{2} \sum_{j=1}^{3} (y_j - \bar{y})^2}. \tag{4.49}$$

Case A provides the reference response to commanded tilt. Cases B and C compare the alignment change with the corresponding stiffness coordinate, while retaining the rotation, displacement, and model-estimated load as supporting quantities. Case D compares the response with the surface-frame lever component and its sign. Case E compares tool-axis and surface-tangent definitions of the commanded tilt.

The comparison was restricted to settings with matching calibrated geometry, phase timing, fixed gains, and null-space parameters. Differences were interpreted together with the within-setting variation and the zero-tilt reference reported in Section 5.1. No continuous optimum was estimated from the discrete parameter levels.

4.7 Secondary Null-Space Pose-Hold Study

The two null-space torque contributions were examined separately in Cartesian pose hold without surface contact. The pose-hold controller retained the initial TCP pose, while an internally commanded joint torque represented the effect of a point force applied at a specified robot-link location. This repeatable software command provided a common disturbance input for all settings.

The force point was assigned to link 3 at the link-frame offset

$$r_p = \begin{bmatrix} 0 \\ 0 \\ 0.200 \end{bmatrix} \mathrm{m}. \tag{4.50}$$

At the initial posture q_0, the structural null direction $v_7(q_0)$ and the translational point Jacobian $J_p(q_0)$ defined the base-frame force direction

$$u_f = \frac{J_p(q_0)v_7(q_0)}{\|J_p(q_0)v_7(q_0)\|_2}. \tag{4.51}$$

The direction was retained throughout the trial. The controller evaluated the current point Jacobian and applied

$$\tau_{\mathrm{dist}}(t) = J_p(q(t))^\top f_d(t), \qquad f_d(t) = 20 s(t) u_f \,\mathrm{N}. \tag{4.52}$$

The scale $s(t)$ increased from zero to one through a half-cosine between 5 s and 7 s, remained at one until 8 s, and returned to zero during the following second. The joint-torque-equivalent disturbance was limited to

$$\|\tau_{\mathrm{dist}}\|_2 \leq 3 \,\mathrm{N\,m}. \tag{4.53}$$

Four null-space settings were tested with three repetitions each. The first setting disabled the null-space torque. The second used projected damping with $d_{\text{null}} = 2\,\text{N m s/rad}$. The remaining settings used singular-value conditioning with $k_\sigma = 1.5\,\text{N m}$ and $k_\sigma = 2\,\text{N m}$, respectively. The SVD probe step, sign deadband, and relative singular-value threshold remained at 0.08 rad, 10^{-6}, and 10^{-4}.

The predefined task-retention limit was a peak Cartesian position error of 2 mm. The projected joint motion during the disturbance was measured by

$$E_N = \int_{5\,\text{s}}^{9\,\text{s}} \|N_q(q)\dot{q}\|_2 \,\mathrm{d}t. \tag{4.54}$$

The integral was converted to degrees for comparison with the angular quantities reported elsewhere. Singular-value conditioning was evaluated from the change in the smallest singular value between the initial and final states. Cartesian task retention was evaluated from the peak position-error norm during the pose hold. The acceptance outcome and measured differences between the four settings are reported in the results chapter.

The disturbance in this study is the torque equivalent of a commanded point force. It characterises the response to a repeatable controller input and does not represent a separately measured physical force or an accidental link contact.

4.8 Robot-Side Monitoring and Procedural Limits

Before torque control started, the robot-side collision behaviour was configured with equal acceleration and nominal joint-torque thresholds of 140 N m. The six Cartesian force and moment components used equal thresholds of 240 N or 240 N m, according to the component type. These values define the conditions for the Franka reflex response.

The joint-torque callback returned the calculated controller command directly to libfranka. Robot-side monitoring therefore supplied the reflex mechanism, while the configured values represented collision thresholds rather than Cartesian-wrench or joint-torque saturation limits. They document the robot configuration used during the measurements and are not interpreted as a validated human-safety margin.

The motion sequence added several procedural limits. Surface approach was bounded

to 0.640 m, and the transition to set-up occurred at a geometrically defined clearance of 20 mm. The operator gate held the reached pose with the translational stiffness of Equation (4.27) until the set-up motion was confirmed. Set-up was bounded by a 5 s time-out and by the alternative 150 N m change in the norm of the model-estimated external moment after the minimum phase time.

The set-up stiffness and signed press reference also define a useful command scale. For a completely blocked 0.060 m displacement along the calibrated normal, the elastic force calculated from Equation (4.28) has a normal component of

$$F_{n,\text{block}} = 0.060\, n_s^\top K_{p,\text{set}} n_s = 48.0\,\text{N}, \tag{4.55}$$

and, because $K_{p,\text{set}}$ is diagonal in the surface frame, a total magnitude of the same value,

$$\left\| K_{p,\text{set}}\,(0.060 n_s) \right\|_2 = 48.0\,\text{N}. \tag{4.56}$$

This value describes the virtual elastic command under a fully blocked reference. The actual robot motion reduces the tracking error, and the value is therefore a command-scale bound rather than a measured contact load.

The set-up moment threshold was applied to the change in libfranka's model-estimated external moment after subtraction of the value stored at the clearance transition. It served as a controller termination condition rather than a calibrated contact-force or contact-moment boundary. The timeout, collision configuration, operator gates, logged torque command, displacement, and model-estimated wrench were considered together when assessing each run.

Chapter 5

Results and Discussion

PURPLE — revised and confirmed. The chapter was revised in full and every paragraph is settled. No further editing is to be applied to it, and any remaining concern is raised as a comment rather than as a change.

This chapter presents the measured response of the surface-contact tests and the separate Cartesian pose-hold study. The five contact cases introduced in table 4.1 are evaluated using the alignment change relative to the calibrated surface, together with the measured rotation, displacement, and model-estimated external load. Case A provides the reference for the stiffness and compliance-centre comparisons in Cases B–E.

5.1 Data Quality and Evaluation Basis

All contact runs completed the set-up phase and were retained for analysis. Each parameter setting was repeated three times, and the tables report the mean and standard deviation of the corresponding repetitions. One repetition in the 10° t_2 condition differed substantially from the other two. It was retained so that the reported mean and dispersion include the complete measured response. Its possible relation to the tool mounting is discussed in section 5.9.

The primary result is the change during set-up in the angle between the calibrated surface normal and the grinding-face direction calculated from the measured end-effector orientation. A positive value denotes a reduction of this angle and therefore an improvement in alignment. Because the tool could rotate by approximately ±2° relative to the gripper

about y_{EE}, the quantity represents the combined response of the robot, gripper, and tool. The physical rotation of the grinding face was not measured independently.

Repeatability was assessed from the three repetitions of each of the 25 contact settings. The median within-setting standard deviation of the alignment improvement was 0.02° for tests about t_1 and 0.04° for tests about t_2. Differences smaller than approximately 0.1° are therefore treated as being of the same order as the observed experimental scatter. This value is used as a descriptive comparison scale rather than as a statistical significance limit.

The reported values were calculated from settled intervals. During the final second of set-up, the median change in the model-estimated normal load was 0.10 N, and the median change in the alignment angle was 0.00°. The largest corresponding changes were 1.10 N and 0.60°. For settings that retained the zero compliance-centre lever, the steady model-estimated normal load remained between 44.5 and 49.4 N. The contact comparisons were therefore made under similar estimated preload levels, except where the lever position itself altered the contact response.

The alignment angle at the beginning of set-up differed from the commanded angular offset. This difference increased with the command and reflects the orientation-transition tolerance, the calibrated geometry, and the compliance of the tool mounting. For example, a 5° command about t_1 produced an initial alignment angle of 2.45°, whereas a 10° command about t_2 produced 8.45°. The measured angle at the beginning of set-up is therefore used as the initial value in all subsequent comparisons.

5.2 Case A: Effect of the Commanded Tilt

The zero-tilt condition was used to quantify the orientation change introduced by the set-up procedure in the absence of a commanded angular offset. The alignment angle increased from 1.07° to 2.04°, which gives an alignment improvement of −0.97°. Set-up therefore introduced a systematic orientation change in this condition. The zero-tilt value was kept as a separate reference and was not subtracted from the tilted results in table 5.1.

Table 5.1: Case-A alignment response, reported as mean ± standard deviation for three repetitions.

Commanded tilt	Initial angle [deg]	Final angle [deg]	Alignment improvement [deg]
None	1.07 ± 0.00	2.04 ± 0.07	-0.97 ± 0.07
$+5°$ about t_1	2.45 ± 0.02	1.51 ± 0.02	$+0.94 \pm 0.03$
$+10°$ about t_1	6.99 ± 0.01	6.35 ± 0.07	$+0.64 \pm 0.08$
$+5°$ about t_2	4.40 ± 0.02	2.88 ± 0.02	$+1.52 \pm 0.00$
$+10°$ about t_2	8.45 ± 0.96	7.39 ± 0.05	$+1.06 \pm 0.92$

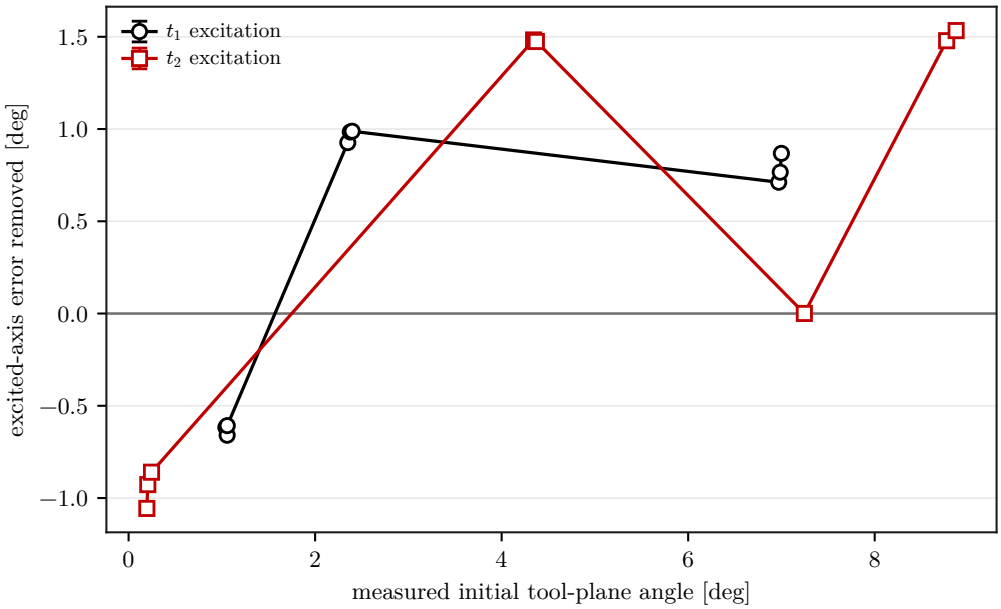


Figure 5.1: Case-A alignment improvement as a function of the measured initial alignment angle.

At both commanded magnitudes, the measured improvement was larger for a tilt about t_2 than for the corresponding tilt about t_1. At 5°, the improvements were 1.52° and 0.94°, respectively. At 10°, they were 1.06° and 0.64°. The response therefore depended on the tilted surface tangent.

The fraction of the measured initial angle removed during set-up decreased as the initial angle increased. About t_1, this fraction fell from approximately 38 % for the 5° command to approximately 9 % for the 10° command. About t_2, the corresponding values were approximately 35 % and 13 %. Within the tested range, the alignment improvement therefore did not increase proportionally with the initial angular error.

The 10° t_2 condition showed substantially greater variability than the remaining settings. One repetition produced 0.00° of alignment improvement, whereas the other two produced 1.56° and 1.62°. Retaining all three repetitions gives the reported standard deviation of 0.92°.

5.3 Case B: Effect of Rotational Stiffness

In Case B, only the rotational stiffness about the tilted surface tangent was varied. The stiffness about the orthogonal tangent remained at 5 N m/rad. The 5 N m/rad entries in table 5.2 are the corresponding 10° settings from Case A and serve as the reference values.

Table 5.2: Case-B alignment improvement for the tested rotational stiffnesses.

Tilted axis	5 N m/rad	15 N m/rad	50 N m/rad
t_1	$+0.64 \pm 0.08$	$+0.57 \pm 0.03$	$+0.49 \pm 0.02$
t_2	$+1.06 \pm 0.92$	$+1.58 \pm 0.04$	$+1.14 \pm 0.03$

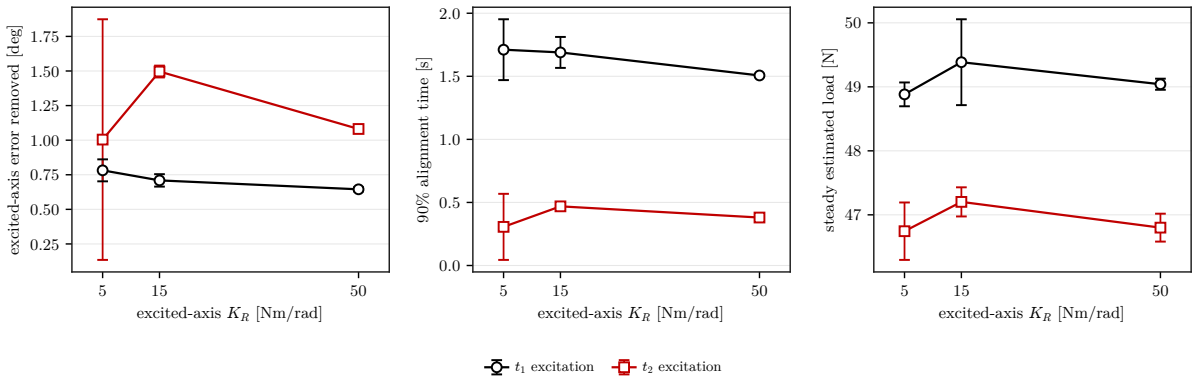


Figure 5.2: Case-B alignment improvement for the tested tilted-axis rotational stiffnesses.

For the t_1 tests, the alignment improvement decreased from 0.64° at 5 N m/rad to 0.57° at 15 N m/rad and 0.49° at 50 N m/rad. Increasing the rotational stiffness therefore reduced the measured alignment response about t_1 over the tested range.

For t_2, the highest tested stiffness produced less improvement than the 15 N m/rad setting. The reused 5 N m/rad reference contains the anomalous repetition from Case A; the remaining two repetitions at this setting average approximately 1.59°, close to the 1.58° measured at 15 N m/rad. The data therefore support a reduction at 50 N m/rad, while the

variability of the reference prevents a reliable ordering of the two softer settings.

Because the two tangent entries were varied separately, these findings apply to the individual tilted-axis stiffness and do not characterise a common paired rotational-stiffness value.

5.4 Case C: Effect of Translational Stiffness

Case C varied the translational stiffness perpendicular to the tilted surface tangent. The 2000 N/m entries in table 5.3 are the corresponding 10° Case-A settings. One additional condition for each tilted axis combined the lowest tested translational stiffness, 300 N/m, with the highest tested rotational stiffness, 50 N m/rad.

Table 5.3: Case-C alignment improvement for the tested perpendicular translational stiffnesses.

Tilted axis	300 N/m	800 N/m	2000 N/m	Combined condition
t_1	$+0.58 \pm 0.02$	$+0.52 \pm 0.01$	$+0.64 \pm 0.08$	$+0.50 \pm 0.02$
t_2	$+1.79 \pm 0.09$	$+1.57 \pm 0.00$	$+1.06 \pm 0.92$	$+1.20 \pm 0.04$

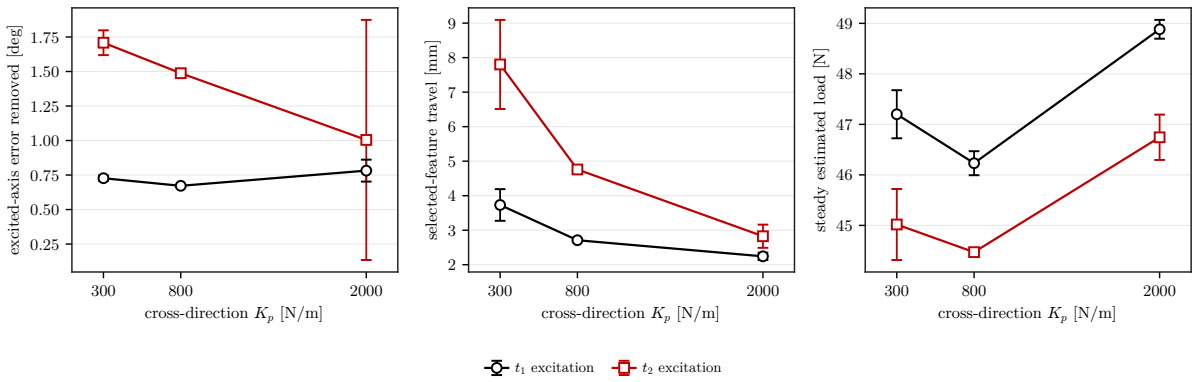


Figure 5.3: Case-C alignment improvement for the tested perpendicular translational stiffnesses.

For a tilt about t_2, reducing the perpendicular translational stiffness increased the measured alignment improvement. The largest value, 1.79°, occurred at 300 N/m. This was the largest improvement measured with the compliance-centre lever kept at zero. For a tilt about t_1, the difference across the tested translational-stiffness range was 0.12°, which is close to the descriptive comparison scale. The measurements therefore show no clear translational-stiffness effect about t_1 within the tested range.

The combined low-translational-stiffness and high-rotational-stiffness condition produced less improvement than the low-translational-stiffness condition alone. About t_2, the improvement decreased from 1.79° to 1.20° when the rotational stiffness was raised from 5 to 50 N m/rad. The response at this tested combination was therefore non-additive. Additional combinations would be required to characterise the interaction systematically.

5.5 Case D: Effect of the Virtual Centre of Compliance

In Case D, the Cartesian source gains were retained at their baseline values and only the lever $r_c = p_{\mathrm{TCP}} - p_c$ was varied. For a tilt about t_1, the lever was varied along t_2; for a tilt about t_2, it was varied along t_1. A fourth setting placed the virtual centre of compliance at the centre of the grinding face.

Table 5.4: Case-D alignment improvement for the tested perpendicular compliance-centre levers.

Tilted axis	−60 mm	0 mm	+60 mm	Grinding-face centre
t_1	$+6.05 \pm 0.14$	$+0.56 \pm 0.01$	$+0.41 \pm 0.01$	$+0.59 \pm 0.03$
t_2	$+0.02 \pm 0.06$	$+1.38 \pm 0.01$	$+6.36 \pm 0.31$	$+1.35 \pm 0.02$

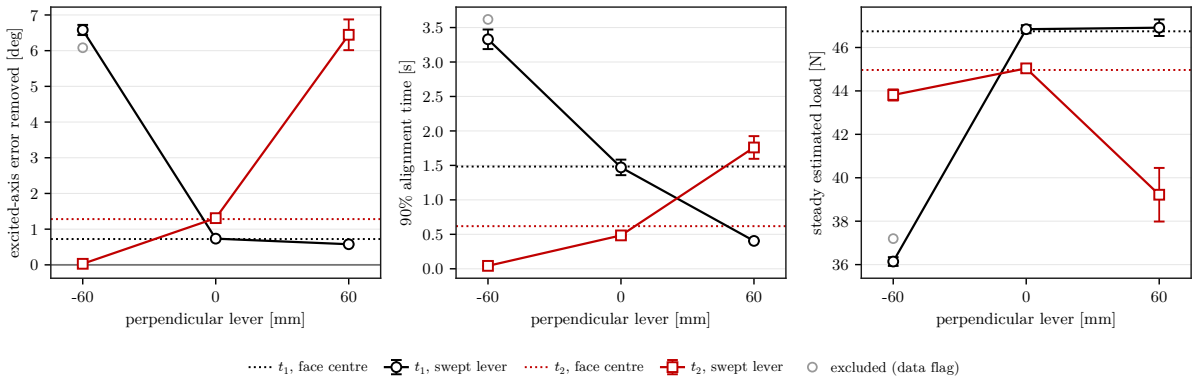


Figure 5.4: Case-D alignment response for the tested compliance-centre levers.

For each tilted axis, one lever direction substantially increased the alignment improvement. This direction is referred to as the assisting lever direction. The opposite direction produced no comparable improvement. For a tilt about t_1, the assisting setting was $r_{c,t_2} = -60$ mm. For a tilt about t_2, it was $r_{c,t_1} = +60$ mm.

With the assisting lever, the residual alignment angle decreased from 6.99° to 0.93° for the t_1 condition and from 8.50° to 2.14° for the t_2 condition. The corresponding alignment improvements were 6.05° and 6.36°. Relative to the zero-lever settings, the improvements increased by 5.49° and 4.98°, respectively. These were the largest alignment improvements measured among the tested parameter settings.

The opposing lever direction produced only 0.41° for the t_1 condition and 0.02° for the t_2 condition. The sign of the in-plane lever therefore determined whether the induced moment assisted or opposed the measured alignment motion.

The model-estimated normal load was also lower for the assisting placements. The steady values were 36.5 N and 39.2 N, compared with approximately 45 to 47 N for the zero-lever settings. The commanded press remained unchanged.

Placing the centre of compliance at the grinding-face centre produced 0.59° of improvement for the t_1 condition and 1.35° for the t_2 condition. These values differ from the corresponding zero-lever results by 0.03° and −0.03°, which is below the descriptive comparison scale. The geometric interpretation of the in-plane and tool-axis lever components is discussed in section 5.8.2.

5.6 Case E: Comparison of Surface and Tool Axes

Case E compared tilts defined about the calibrated surface tangents with tilts defined about the grinding-face axes. The commanded spin places the long axis of the grinding face approximately 25° from t_2. A command about x_{EE} or y_{EE} therefore contains components about both surface tangents.

Table 5.5: Case-E alignment improvement for surface-tangent and grinding-face-axis tilts.

Tilt axis	Initial angle [deg]	Alignment improvement [deg]
t_1	6.99 ± 0.01	$+0.64 \pm 0.08$
t_2	8.45 ± 0.96	$+1.06 \pm 0.92$
y_{EE}	6.81 ± 0.01	$+1.07 \pm 0.03$
x_{EE}	6.78 ± 0.02	$+1.29 \pm 0.02$

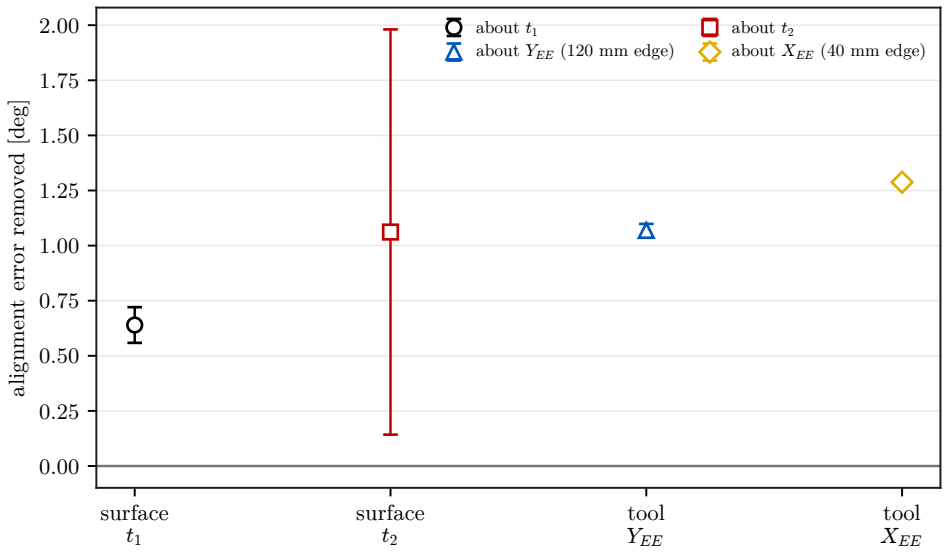


Figure 5.5: Case-E alignment improvement for surface-tangent and grinding-face-axis tilts.

The two grinding-face-axis results lie between the two surface-tangent results. Their difference was 0.22°, compared with 0.42° between the t_1 and t_2 values. For comparison, resolving the two grinding-face-axis commands into their t_1 and t_2 components and combining the corresponding Case-A responses gives 1.25° for the x_{EE} command and 1.17° for the y_{EE} command. The measured values were 1.29° and 1.07°, respectively.

This agreement provides a consistency check for the interpretation that the observed asymmetry is associated mainly with the surface-frame directions rather than solely with the geometry of the grinding face. It does not establish linear superposition as a general response model, and the conclusion is limited to the four tested settings.

5.7 Null-Space Pose-Hold Results

The null-space terms were evaluated separately in Cartesian pose hold without surface contact. The internally commanded point-force equivalent reached 20.00 N in all twelve trials. Its peak equivalent joint-torque norm lay between 2.17 and 2.45 N m. The torque-limit scale remained equal to one, so the requested waveform was applied without clipping. All trials completed and were retained.

Projected null-space damping reduced the cumulative redundant joint excursion from $(7.596 \pm 0.916)°$ without null-space torque to $(5.687 \pm 0.228)°$ at 2 N m s/rad. This corresponds to a reduction of approximately 25 % relative to the zero-torque condi-

tion. The final change in the minimum singular value also became less negative, from $(-2.13 \pm 0.47) \times 10^{-3}$ to $(-1.26 \pm 0.10) \times 10^{-3}$.

Both sigma-only settings produced a small positive final change in the minimum singular value. At $k_\sigma = 1.5\,\mathrm{N\,m}$, the measured value was $(1.95 \pm 0.07) \times 10^{-5}$; at $2\,\mathrm{N\,m}$, it was $(1.93 \pm 0.03) \times 10^{-5}$. Increasing the sigma-torque magnitude therefore did not increase the retained conditioning change. It did, however, increase the cumulative excursion from $(0.283 \pm 0.007)°$ to $(1.619 \pm 0.130)°$. The mean peak Cartesian position error increased from $0.889 \pm 0.024\,\mathrm{mm}$ to $0.983 \pm 0.053\,\mathrm{mm}$. The largest position error measured in the complete pose-hold study was $1.304\,\mathrm{mm}$, below the predefined $2\,\mathrm{mm}$ acceptance limit.

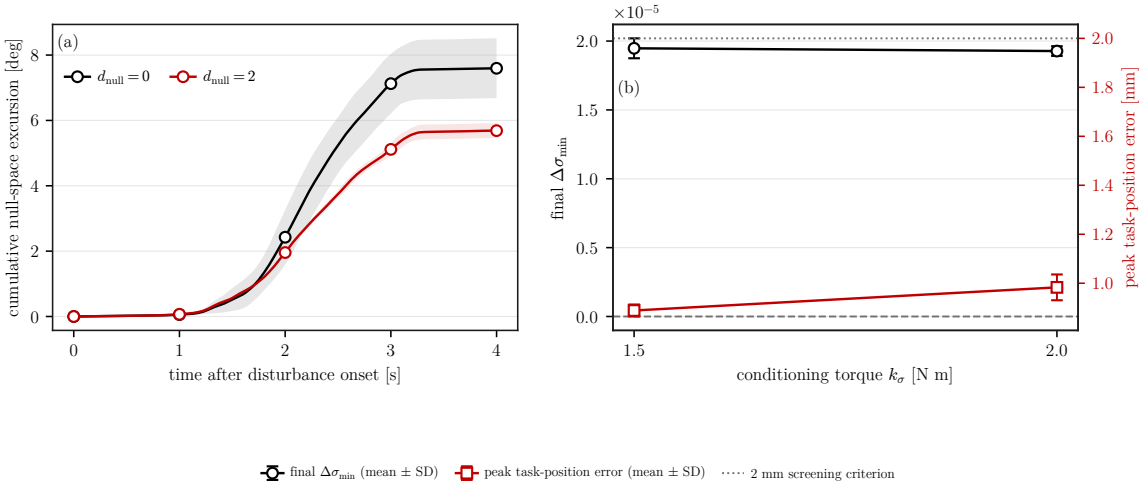


Figure 5.6: Null-space response during the Cartesian pose-hold disturbance test.

The pose-hold study separates the projected damping and sigma-torque terms in free space under an internally commanded disturbance. It characterises this specific hold condition; physical link contact and the damping-only mode used during the surface-contact tests remain outside its scope.

5.8 Discussion

5.8.1 Relative Influence of the Varied Parameters

The measured effect sizes differed substantially across the varied parameter families. For t_1, increasing the tilted-axis rotational stiffness from 5 to $50\,\mathrm{N\,m/rad}$ reduced the alignment improvement by 0.15°. For t_2, the highest tested rotational stiffness reduced the

response by 0.44° relative to the 15 N m/rad condition. Reducing the perpendicular translational stiffness from 2000 to 300 N/m changed the response by −0.06° about t_1 and by +0.73° about t_2.

The compliance-centre lever produced the largest measured changes. Relative to the zero-lever condition, the assisting 60 mm lever increased the alignment improvement by 5.49° about t_1 and 4.98° about t_2. The opposing lever direction reduced the improvement to 0.41° and 0.02°, respectively. Within the tested configuration and parameter range, the compliance-centre position and its sign therefore had a substantially stronger influence than the individual stiffness variations.

The measurements support considering the compliance-centre lever before refining the stiffness entries. The rotational stiffness controls the angular resistance of the impedance, and the perpendicular translational stiffness can modify the response for particular directions. The lever, however, directly determines whether the normal press produces a moment that assists or opposes the required rotation.

5.8.2 Role of the Compliance-Centre Lever

The sign dependence observed in Case D follows from the point-shifted impedance. With the press directed approximately along the negative surface normal, $f \approx -Fn_s$, the coupling moment is

$$m_c = -r_c \times f. \tag{5.1}$$

A lever component along the tangent perpendicular to the tilted axis therefore creates a moment about that axis. Reversing the lever reverses the moment. One sign assists the rotation that brings the grinding face towards the surface, whereas the opposite sign opposes it. Exchanging the roles of t_1 and t_2 changes the cross-product direction, which is consistent with the assisting lever signs being opposite for the two tilted axes.

The component of r_c parallel to the press direction contributes no moment because its cross product with f is zero. This explains why the compliance-centre position at the grinding-face centre produced a response close to the zero-lever setting. At a 10° tilt, the 20 mm displacement along the grinding-face direction has a tangential projection of only

approximately 3.5 mm. This is about one seventeenth of the 60 mm tangential lever used in the main comparison.

For an axial displacement d and a tilt angle θ, the effective tangential component is $d \sin \theta$. At 10°, an axial displacement of approximately 345 mm would be required to produce a tangential component of 60 mm. The tested axial placement therefore generated substantially less coupling moment than the direct tangential placement.

The lower model-estimated normal loads at the assisting lever positions are consistent with the same mechanism. A larger alignment motion allows part of the commanded displacement to be accommodated through rotation, reducing the remaining translational compression. This explanation remains model-based because the external load was estimated by the robot model and the tool rotation relative to the gripper was not measured independently.

The lever acts as a directional bias rather than as a feedback term that changes sign with the measured orientation error. All commanded tilts in the surface-contact tests had the same sign. The measured assisting directions therefore apply to that sign, while the response to the opposite sign follows from the coupling model and was not tested directly.

5.8.3 Implications for Controller Parameter Selection

The measured ordering suggests a practical sequence for selecting controller parameters within a similar contact geometry. The compliance-centre lever is considered first because it determines the direction and approximate magnitude of the coupling moment. Its tangential direction should be perpendicular to the expected tilt axis, and its sign should be selected so that the induced moment assists alignment.

After the lever has been selected, the rotational stiffness about the tilted axis can be refined. Within the tested range, increasing this stiffness reduced the available alignment motion. The perpendicular translational stiffness can then be adjusted where the measurements show a directional benefit. In the present tests, reducing it increased the response about t_2, whereas no clear effect above the descriptive comparison scale was observed about t_1.

This sequence is a starting point rather than a general tuning rule. It is based on one robot configuration, one tool mounting, positive commanded tilts, and a limited set of

parameter values. In particular, the three tested lever levels identify an assisting direction and demonstrate the size of its effect, but they do not identify an optimum lever length.

Where the sign of the initial tilt is unknown, a fixed tangential lever can assist one direction and oppose the other. A placement at the TCP or along the grinding-face direction avoids a strong directional bias, but the measured alignment improvement then remains close to the zero-lever response. A larger correction would require selecting the lever direction from the measured sign of the initial tilt before the set-up motion begins. The response of such a selection strategy to negative tilts was not evaluated in the present tests.

5.9 Limitations

5.9.1 Experimental Range

The experimental conclusions are limited to the tested parameter matrix. All commanded tilts had the same sign. The rotational and translational stiffnesses were varied at three levels, and only one combined low-translational-stiffness and high-rotational-stiffness condition was tested for each axis. The compliance-centre comparison used three tangential lever values and one position at the grinding-face centre. These settings identify directional trends and relative effect sizes within the investigated range, but they do not establish an optimum lever length, a complete stiffness interaction, or the response to the opposite tilt sign.

The results were obtained with one surface, one rectangular tool, one gripper configuration, and one commanded press trajectory. Changes in friction, contact geometry, tool dimensions, payload, or robot configuration may change the measured alignment response and the assisting lever magnitude.

5.9.2 Measurement and Tool-Mount Uncertainty

The alignment metric combines the measured end-effector orientation with the calibrated grinding-face direction and the calibrated surface normal. Its absolute value therefore inherits uncertainty from the surface calibration, the grinding-face calibration, and the

relative motion between the tool and the gripper. The approximately ±2° rotational freedom about y_{EE} was of the same order as several measured alignment changes.

For the tested mounting orientation, y_{EE} lay approximately 25° from t_2. Relative tool motion may therefore have contributed more strongly to the t_2 conditions than to the t_1 conditions. The 10° t_2 repetition that produced no measured alignment improvement is consistent with this possibility, although the tool mounting cannot be identified as its sole cause. Retaining the repetition ensures that the reported mean and standard deviation include the observed variability.

Motion of this kind absorbs rotation that the arm would otherwise have produced. Since the alignment angle is inferred from the end-effector orientation, an unobserved rotation of the tool inside the gripper lowers the measured improvement and cannot raise it. The alignment changes measured about t_2 are therefore lower bounds on the physical response. Their larger within-setting scatter is consistent with the same mechanism. Scatter also grew with the size of the correction on both tangents, however, and the largest spread about t_1 was measured in the 6.05° compliance-centre condition.

The normal load used in the comparison is the model-estimated external load reported by libfranka. It is a supporting quantity rather than an independent force measurement. The measured relationship between alignment and estimated load should therefore be interpreted as a relationship between two signals derived from the robot state and model.

5.9.3 Scope of the Null-Space and Timing Evaluation

The surface-contact tests used projected null-space damping alone. The singular-value-conditioning term was inactive, so the alignment results describe the controller with one null-space contribution rather than both. The pose-hold study isolates the two terms only in free space under an internally commanded point-force equivalent. It does not establish their response to physical link contact, and the conditioning term was not exercised during surface contact at all.

The controller operated through the nominal 1 kHz Franka Control Interface. The experiments evaluated the resulting motion and contact response, but they did not measure the worst-case execution time of the real-time callback. The results therefore establish experimental behaviour at the nominal control rate rather than a complete timing guarantee.

Chapter 6

Conclusion and Future Work

PURPLE — revised and confirmed. The chapter was revised in full and every paragraph is settled. No further editing is to be applied to it, and any remaining concern is raised as a comment rather than as a change.

6.1 Conclusion

A real-time Cartesian impedance controller was implemented for the contact-induced alignment of a grinding tool with a plane surface. During the set-up phase, the Cartesian stiffness and damping were spatially shifted from a selected centre of compliance to the TCP. This shift introduced translation–rotation coupling, so that the normal interaction load could generate a moment about the surface tangents. The controller was evaluated in 75 surface-contact runs covering commanded tilt, rotational stiffness, translational stiffness, and compliance-centre placement. A separate Cartesian pose-hold study was used to examine the projected null-space damping and singular-value-conditioning terms.

The zero-tilt condition showed that the set-up procedure itself introduced an orientation change. The alignment angle increased from 1.07° to 2.04°, corresponding to an alignment change of −0.97°. The tilted conditions were therefore interpreted together with this baseline response. The zero-tilt value was retained as a separate measurement and was not subtracted from the remaining results.

Among the varied parameters, the position of the virtual centre of compliance had the largest measured influence on alignment. With the lever directed so that the induced moment assisted the required rotation, the alignment improvement reached 6.05° for a tilt

about t_1 and 6.36° for a tilt about t_2. The corresponding zero-lever conditions produced 0.56° and 1.38°, respectively. The assisting lever direction changed with the tilted surface tangent: $r_{c,t_2} = -60\,\text{mm}$ was required for the t_1 condition, whereas $r_{c,t_1} = +60\,\text{mm}$ was required for the t_2 condition. Applying the lever in the opposite direction produced no comparable improvement. The compliance-centre lever therefore acts as a directional bias whose sign must agree with the required corrective rotation.

The tested stiffness variations produced smaller changes. Increasing the rotational stiffness about the tilted tangent reduced the alignment improvement about t_1, and the highest tested rotational stiffness also produced less improvement than the softer settings about t_2. Reducing the translational stiffness perpendicular to the tilted tangent increased the response about t_2, reaching 1.79° at 300 N/m. The corresponding variation about t_1 remained of the same order as the experimental scatter. Combining the lowest translational stiffness with the highest rotational stiffness produced less improvement than the low-translational-stiffness condition alone. This non-additive response indicates that the two stiffness entries cannot be selected independently over the tested range.

Placing the centre of compliance at the centre of the grinding face produced nearly the same response as the zero-lever condition. At the commanded 10° tilt, the 20 mm axial displacement generated only a small tangential lever component. The measurements are therefore consistent with the point-shifted impedance formulation: the tangential component of the lever determines the moment about the tilted surface tangent, while the component parallel to the normal interaction load contributes no moment.

Tilts commanded about the grinding-face axes produced alignment improvements between those obtained for the two surface-tangent directions. Their agreement with a linear combination of the tangent-axis measurements provides a consistency check for the surface-frame interpretation. The comparison does not establish linear superposition as a general response model, but it indicates that the measured directional asymmetry was associated mainly with the surface-frame response rather than solely with the rectangular grinding-face geometry.

The Cartesian pose-hold study showed that projected null-space damping reduced the cumulative redundant joint excursion by approximately 25 % relative to the matched condition without a null-space torque. The two tested singular-value-conditioning commands produced similar final conditioning changes, while the larger command caused greater

joint excursion and a larger Cartesian position error. These measurements characterise the separated null-space terms for the investigated free-space hold condition.

Overall, the experiments show that the spatial placement and sign of the virtual centre of compliance governed the alignment response more strongly than the stiffness variations considered in this work. The point-shifted Cartesian impedance provided a direct means of converting the commanded normal interaction into an alignment moment without prescribing an explicit rotational trajectory during set-up. A fixed tangential lever is effective only when its induced moment acts in the required direction. Alignment for an unknown tilt direction therefore requires either a lever selected from the measured initial error or a configuration that relies primarily on rotational compliance.

6.2 Limitations

6.2.1 Experimental Range

The contact experiments were performed with one robot, one tool mounting, one plane surface, and a finite set of controller parameters. All commanded tilts had the same sign. The measured assisting lever directions therefore apply to the investigated tilt sign and geometry. Their response for the opposite tilt sign follows from the point-shifted model but was not measured directly.

The tangential compliance-centre study used three lever values along one surface tangent at a time. These measurements identify the assisting direction and demonstrate the magnitude of the resulting alignment improvement within the tested range. They do not identify an optimum lever length or the response to simultaneous changes of both in-plane lever components. The stiffness comparison likewise covered a limited number of values, and the combined low-translational-stiffness and high-rotational-stiffness condition represents one selected combination rather than a complete factorial study.

6.2.2 Measurement and Tool-Mount Uncertainty

The grinding-face direction used in the evaluation was calculated from the measured end-effector orientation and the calibrated grinding-face direction in the end-effector frame.

The tool could rotate by approximately $\pm 2^\circ$ relative to the gripper about y_{EE}. The reported alignment angle therefore describes the combined robot–gripper–tool response and cannot separate controller-induced rotation from motion within the gripper. This uncertainty is particularly relevant to the t_2 conditions, for which the largest within-setting variability was observed. The alignment changes reported for those conditions are therefore lower bounds on the physical response.

The alignment metric also inherits uncertainty from the calibrated surface normal and the tool calibration. The reported external force and moment were the model-based estimates supplied by libfranka. They were not obtained from an independent force/torque sensor and were therefore used as supporting quantities rather than as direct measurements of the contact load.

6.2.3 Null-Space and Timing Scope

Only projected damping was active during the surface-contact tests. The singular-value-conditioning term was not applied, so the contact results describe one null-space contribution rather than both. The pose-hold study isolated the two terms only in free space under an internally commanded point-force equivalent. It does not establish their response to physical link contact, to surface contact, or to the combined null-space mode.

The controller operated through the nominal 1 kHz libfranka interface. The experiments evaluated motion and alignment behaviour rather than worst-case callback execution time. A formal real-time timing analysis was therefore outside the scope of the reported measurements.

6.3 Future Work

The compliance-centre study should first be extended to positive and negative initial tilts about both surface tangents. This would test whether reversing the lever according to the sign of the measured initial error produces comparable alignment in both directions. A subsequent sweep of the lever length, followed by a two-dimensional variation of its in-plane components, would allow the relation between initial angle, lever direction, and lever magnitude to be characterised more completely.

The grinding-face orientation should be measured independently of the end-effector orientation. A more rigid tool mount would reduce the relative motion observed at the gripper, while an external angular measurement would allow controller rotation and tool-mount rotation to be separated. An independent force/torque sensor would also permit the model-estimated external wrench to be compared with a direct contact-load measurement.

The null-space study should be repeated with a physical disturbance applied to an intermediate robot link and should include the combined damping and singular-value-conditioning mode, which the contact tests did not use. Repeating this comparison during surface interaction would show whether the ordering observed in free-space hold is retained in contact. Recording callback execution times and deadline overruns during these tests would additionally provide a direct assessment of the controller's real-time performance.

Finally, the compliance-centre selection can be incorporated into the controller as a geometry-based scheduling step. After the initial tilt is estimated at the clearance transition, the lever direction could be selected so that the induced moment opposes the measured angular error. Such a strategy would replace the fixed directional bias with a run-specific placement while retaining the point-shifted Cartesian impedance used in this work.

Bibliography

[1] Alin Albu-Schäffer, Christian Ott, Udo Frese, and Gerd Hirzinger. Cartesian impedance control of redundant robots: Recent results with the DLR-light-weight-arms. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, volume 3, pages 3704–3709, 2003. doi: 10.1109/ROBOT.2003.1242165.

[2] Stefano Chiaverini, Bruno Siciliano, and Olav Egeland. Review of the damped least-squares inverse kinematics with experiments on an industrial robot manipulator. *IEEE Transactions on Control Systems Technology*, 2(2):123–134, 1994. doi: 10.1109/87.294335.

[3] John J. Craig. *Introduction to Robotics: Mechanics and Control*. Pearson Prentice Hall, Upper Saddle River, NJ, 3 edition, 2005.

[4] Jacques Denavit and Richard S. Hartenberg. A kinematic notation for lower-pair mechanisms based on matrices. *Journal of Applied Mechanics*, 22(2):215–221, 1955.

[5] Alexander Dietrich, Christian Ott, and Alin Albu-Schäffer. An overview of null space projections for redundant, torque-controlled robots. *The International Journal of Robotics Research*, 34(11):1385–1400, 2015. doi: 10.1177/0278364914566516.

[6] Ernest D. Fasse and Jan F. Broenink. A spatial impedance controller for robotic manipulation. *IEEE Transactions on Robotics and Automation*, 13(4):546–556, 1997. doi: 10.1109/70.611320.

[7] Franka Emika GmbH. Franka control interface documentation. https://frankaemika.github.io/docs/, 2023. Accessed for robot interface, libfranka model access, and torque control documentation.

[8] Franka Emika GmbH. libfranka: Cartesian Impedance Control Example. https://github.com/frankaemika/libfranka/blob/master/examples/cartesian_impedance_control.cpp, 2023. Official libfranka example, file `examples/cartesian_impedance_control.cpp`.

[9] Richard Hartisch and Kevin Haninger. Flexure-based environmental compliance for high-speed robotic contact tasks, 2022.

[10] Neville Hogan. Impedance control: An approach to manipulation—parts i–iii. *Journal of Dynamic Systems, Measurement, and Control*, 107(1):1–24, 1985.

[11] International Federation of Robotics. World robotics 2023 report. Technical report, IFR Statistical Department, Frankfurt am Main, 2023.

[12] Oussama Khatib. A unified approach for motion and force control of robot manipulators. *IEEE Journal on Robotics and Automation*, 3(1):43–53, 1987.

[13] Kevin M. Lynch and Frank C. Park. *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017. URL https://hades.mech.northwestern.edu/images/7/7f/MR.pdf. Free preprint.

[14] Matthias Mayr and Julian M. Salt-Ducaju. A C++ implementation of a cartesian impedance controller for robotic manipulators. *Journal of Open Source Software*, 9 (93):5194, 2024. doi: 10.21105/joss.05194.

[15] Richard M. Murray, Zexiang Li, and S. Shankar Sastry. *A Mathematical Introduction to Robotic Manipulation*. CRC Press, 1994.

[16] James L. Nevins and Daniel E. Whitney. Computer-controlled assembly. *Scientific American*, 1989. Background source on robotic assembly and compliance.

[17] Christian Ott, Alin Albu-Schäffer, Andreas Kugi, and Gerd Hirzinger. On the passivity-based impedance control of flexible joint robots. *IEEE Transactions on Robotics*, 24(2):416–429, 2008.

[18] Mathew Jose Pollayil, Franco Angelini, Guiyang Xin, Michael Mistry, Sethu Vijayakumar, Antonio Bicchi, and Manolo Garabini. Choosing stiffness and damping

for optimal impedance planning. *IEEE Transactions on Robotics*, 39(2):1281–1300, 2023. doi: 10.1109/TRO.2022.3216078.

[19] Bruno Siciliano, Lorenzo Sciavicco, Luigi Villani, and Giuseppe Oriolo. *Robotics: Modelling, Planning and Control*. Springer, London, 2009.

[20] Stefano Stramigioli, Claudio Melchiorri, and Stefano Andreotti. Intrinsically passive grasping and manipulation. *IEEE Transactions on Robotics and Automation*, 2000. URL http://robby.caltech.edu/~jwb/courses/CDS270/papers/passive.grasp.pdf. Open-access preprint.

[21] Valeria Villani, Francesco Pini, Francesco Leali, and Cristian Secchi. Survey on human–robot collaboration in industrial settings: Safety, intuitive interfaces and applications. *Mechatronics*, 55:248–266, 2018. doi: 10.1016/j.mechatronics.2018.02.009.

[22] Sen Wang, Guodong Chen, Hui Xu, and Zheng Wang. A robotic peg-in-hole assembly strategy based on variable compliance center. *IEEE Access*, 2019. doi: 10.1109/ACCESS.2019.2954459.

[23] Paul C. Watson. Remote center compliance system. U.S. Patent 4,098,001, 1978. Assigned to The Charles Stark Draper Laboratory.

Appendix A

Controller Source Listings

GREEN — sufficient. The listings are abridged to the mathematical paths used by the thesis and connect each important equation to an implementation excerpt.

This appendix contains abridged source excerpts from the surface-grinding controller used in the reported experiments. Only the paths needed to connect the equations to the implementation are shown. The excerpts use fixed-size Eigen aliases such as `Vec3`, `Vec7`, `Mat3`, `Mat6x6`, and `Mat6x7`. The identifier `R_alignment_target` is the source name for the surface frame R_{surface} used in the chapters.

A.1 Surface Task Frame and Spatial Gains

The frame order is $[t_1, t_2, n_s]$. Because $t_2 = n_s \times t_1$, this column order is right-handed. A configured tangent that is close to the normal is replaced by a fallback direction, so the frame stays defined for any configured plane.

Listing A.1: Surface frame and gain transformation used by the controller.

```
Mat3 makeSurfaceFrameFromNormalTangent(
    const Vec3& normal_input, const Vec3& tangent1_input) {
  const Vec3 normal =
      normalizedOrFallback(normal_input, Vec3(1.0, 0.0, 0.0));
  Vec3 tangent1 =
      tangent1_input - normal * normal.dot(tangent1_input);

  if (tangent1.norm() <= 1e-9) {
    Vec3 fallback(0.0, 1.0, 0.0);
    if (std::abs(normal.dot(fallback)) > 0.95) {
      fallback = Vec3(0.0, 0.0, 1.0);
    }
    tangent1 = fallback - normal * normal.dot(fallback);
  }

  tangent1.normalize();
  Vec3 tangent2 = normal.cross(tangent1);
```

```
  tangent2.normalize();

  Mat3 R_alignment_target;
  R_alignment_target.col(0) = tangent1;
  R_alignment_target.col(1) = tangent2;
  R_alignment_target.col(2) = normal;
  return R_alignment_target;
}

Mat3 makeSpatialGainMatrix(
    const Vec3& diagonal_in_task_frame, const Mat3& R_task) {
  return R_task * diagonal_in_task_frame.asDiagonal()
       * R_task.transpose();
}
```

A.2 Tool Orientation and Rotation Error

The physical tool axis is configured in the EE frame. The desired orientation is the minimum rotation of the captured start orientation that maps this axis onto the signed normal; it is not obtained by assigning the complete surface frame to the EE frame. The commanded angular offsets of a case are applied to that axis before the rotation is formed, so a tilt about t_1 or t_2 enters as a rotation of the orientation target rather than as a trajectory.

Listing A.2: Signed tool-axis target, commanded tilt offsets, and restoring rotation-vector error.

```
Vec3 surfaceToolAxisInBase(
    const Parameters& params, const Mat3& R_alignment_target) {
  const double sign =
      (params.tool_axis_target_sign >= 0.0) ? 1.0 : -1.0;
  return sign * R_alignment_target.col(2);  // normal = 3rd column
}

Vec3 desiredToolAxisInBase(
    const Parameters& params, const Mat3& R_alignment_target) {
  const Vec3 flat_axis =
      surfaceToolAxisInBase(params, R_alignment_target);
  const double deg_to_rad = M_PI / 180.0;
  const Vec3 rotation_vector =
      deg_to_rad *
      (params.tool_target_offset_tangent1_deg
           * R_alignment_target.col(0) +
       params.tool_target_offset_tangent2_deg
           * R_alignment_target.col(1));
  const double angle = rotation_vector.norm();
  if (angle <= 1e-12) {
    return flat_axis;
  }
  return Eigen::AngleAxisd(angle, rotation_vector / angle)
       * flat_axis;
}

Vec3 orientationError(
    const Mat3& R_current, const Mat3& R_desired) {
  Mat3 R_error = R_current.transpose() * R_desired;
  Eigen::AngleAxisd angle_axis(R_error);
  if (std::abs(angle_axis.angle()) < 1e-9) {
    return Vec3::Zero();
  }
  return R_current * angle_axis.axis() * angle_axis.angle();
}
```

The shortest arc that carries the current tool axis onto the target turns about an axis perpendicular to the tool axis, so it leaves the spin about that axis at whatever the start pose held. That spin decides which edge of a rectangular face leads into the surface, so it is commanded separately through `tool_target_offset_normal_deg` rather than inherited from q_{init}.

Listing A.3: Commanded spin about the tool axis (abridged).

```
const Mat3 R_axis =
    rotationBetweenUnitVectors(tool_axis_start, tool_axis_target)
    * R_start;
if (!params.command_tool_twist) {
  return R_axis;
}

// Both directions are measured in the plane perpendicular to the
// tool axis.
const Vec3 current = perpendicular(R_axis * face_long_axis_ee);
const Vec3 zero_reference =
    perpendicular(R_alignment_target.col(0));
if (current.norm() <= 1e-9 || zero_reference.norm() <= 1e-9) {
  return R_axis;  // no twist is defined
}

const Vec3 target =
    Eigen::AngleAxisd(
        (M_PI / 180.0) * params.tool_target_offset_normal_deg,
        tool_axis_target)
    * zero_reference.normalized();
const Vec3 current_unit = current.normalized();
double twist = std::atan2(
    tool_axis_target.dot(current_unit.cross(target)),
    current_unit.dot(target));

// A face centred on the tool axis maps onto itself every half turn,
// so take the representative nearest the start pose.
if (face_center_off_axis.norm() <= 0.05 * face_scale) {
  twist -= M_PI * std::round(twist / M_PI);
}
return Eigen::AngleAxisd(twist, tool_axis_target) * R_axis;
```

A.3 Geometric Clearance and Phase Targets

The leading point, edge, or face of the rectangular tool is selected along the descent direction. The transition into set-up is based on the exact most-leading corner height; the average of corners tied within 0.1 mm supplies the controlled feature and projected reference. The transition depends on this height; the external-force estimate does not enter the decision. The following excerpt is abridged from the phase switch.

Listing A.4: Phase-dependent tool-feature targets (abridged).

```
const Vec3 descend_direction = -R_alignment_target.col(2);
double leading_contact_projection = 0.0;
const Vec3 tool_contact_offset_ee =
    selectLeadingToolContact(R_EE, leading_contact_projection);
const Vec3 tool_contact_point =
    p_EE + R_EE * tool_contact_offset_ee;

```

```
case ControlPhase::kApproachDescend: {
  // The gate freezes this clock, so waiting does not go on ramping
  // the commanded distance.
  const double distance = std::min(
      params.descend_speed * (phase_time - gate_paused_time),
      params.descend_max_distance);
  desired.p_d = p_start + distance * descend_direction;
  desired.pdot_d = params.descend_speed * descend_direction;

  const Vec3 surface_normal = (-descend_direction).normalized();
  const double height_above_surface =
      surface_normal.dot(p_EE - surface_point_runtime)
      - leading_contact_projection;
  const double controlled_point_height =
      surface_normal.dot(tool_contact_point - surface_point_runtime);
  const Vec3 projected_surface_point =
      tool_contact_point - controlled_point_height * surface_normal;
  const bool clearance_reached =
      height_above_surface <= params.descend_surface_clearance;

  if (clearance_reached && !gate_blocking) {
    active_tool_contact_offset_ee = tool_contact_offset_ee;
    first_contact_tcp = p_EE;  // captured at clearance
    first_contact_point = projected_surface_point;
    setup_push_start = -controlled_point_height;
    R_contact_start = R_EE;
    contact_force_bias = external_force;
    contact_moment_bias = external_moment;
    phase = ControlPhase::kSetUp;
  }
  break;
}

case ControlPhase::kSetUp: {
  const double push =
      setUpPush(params, phase_time - gate_grind_paused_time,
                setup_push_start, setup_push_velocity);
  const Vec3 edge_target =
      first_contact_point + push * descend_direction;
  desired.p_d =
      edge_target - R_contact_start * tool_contact_offset_ee;
  desired.pdot_d = setup_push_velocity * descend_direction;
  // Exit after minimum time on moment-delta norm, or on timeout.
  break;
}

case ControlPhase::kGrind: {
  const Vec3 n = descend_direction;  // unit, into the surface
  Vec3 edge_target;
  if (params.grind_sweep_enabled) {
    edge_target = first_contact_point + grind_push * n
                + sweep_s * grind_tangent;
    desired.pdot_d = sweep_s_dot * grind_tangent;
  } else {
    const double edge_penetration =
        n.dot(tool_contact_point - first_contact_point);
    edge_target =
        tool_contact_point + (grind_push - edge_penetration) * n;
  }
  desired.p_d =
      edge_target - R_contact_start * tool_contact_offset_ee;
  break;
}
```

The identifiers `first_contact_*` are retained source names. They are captured at the configured clearance, not at a force-detected first contact. The model-estimated wrench recorded at the same instant becomes the bias subtracted from every later sample.

Both operator gates latch once armed. The gate compares a held pose against the clearance height, and re-testing that comparison every cycle released the gate for single cycles on measurement noise; on each release the descent clock advanced and commanded the

tool further down, so a longer wait produced a harder press. Latching removes that dependence on waiting time.

A.4 Centre of Compliance and the Offset Adjoint

The lever passed to the implementation points from the selected centre of compliance to the TCP:

$$r_c = p_{\mathrm{TCP}} - p_c.$$

The same congruence is applied to stiffness and damping.

Listing A.5: Pole-to-TCP adjoint and congruence transform.

```
Mat3 skewMatrix(const Vec3& v) {
  Mat3 s;
  s << 0.0, -v(2),  v(1),
       v(2),  0.0, -v(0),
      -v(1),  v(0),  0.0;
  return s;
}

Mat6x6 offsetAdjoint(const Vec3& r_c) {
  Mat6x6 adjoint = Mat6x6::Zero();
  adjoint.block<3, 3>(0, 0) = Mat3::Identity();
  adjoint.block<3, 3>(0, 3) = skewMatrix(r_c);
  adjoint.block<3, 3>(3, 3) = Mat3::Identity();
  return adjoint;
}

Mat6x6 adjointTransformedGain(
    const Mat6x6& pole_gain, const Vec3& r_c) {
  const Mat6x6 adjoint = offsetAdjoint(r_c);
  return adjoint.transpose() * pole_gain * adjoint;
}
```

The wrench routine selects between the decoupled and the coupled law and constructs the lever. Three conventions are implemented. The reported experiments used `coupled_use_direct_rc_surface`, which reads the lever directly in surface coordinates; the offset-from-edge branch is retained only for archived configuration files.

Listing A.6: Selection of the decoupled or coupled spring wrench (abridged).

```
const bool coupled =
    params.use_coupled_stiffness &&
    (phase == ControlPhase::kSetUp ||
     (phase == ControlPhase::kHold && params.hold_with_setup_gains));
if (!coupled) {
  // Two independent springs. dv.tail is -omega, so the damping
  // signs match.
  Vec6 wrench;
  wrench.head<3>() = Kp * dx.head<3>() + Dp * dv.head<3>();
  wrench.tail<3>() = KR * dx.tail<3>() + DR * dv.tail<3>();
  return wrench;
}

Vec3 r_c;
if (params.coupled_use_pole_ee) {
  // A point on the tool: it rides with the tool instead of
```

```
  // standing still in the plane.
  r_c = -(contact.R_EE * params.coupled_pole_ee);
} else if (params.coupled_use_direct_rc_surface) {
  r_c = R_alignment_target * params.coupled_rc_surface;
} else {
  // Legacy convention retained only for archived setup files.
  const Vec3 edge_ref = params.coupled_pole_freeze_at_contact
                          ? contact.edge_at_contact : contact.edge;
  const Vec3 tcp_ref = params.coupled_pole_freeze_at_contact
                          ? contact.tcp_at_contact : contact.tcp;
  r_c = tcp_ref - (edge_ref + params.coupled_pole_from_edge);
}
const Mat6x6 K_tcp =
    adjointTransformedGain(blockDiagonal(Kp, KR), r_c);
const Mat6x6 D_tcp =
    adjointTransformedGain(blockDiagonal(Dp, DR), r_c);
return K_tcp * dx + D_tcp * dv;
```

With `coupled_pole_freeze_at_contact` set, the references are those captured at the clearance transition, so r_c stays constant during set-up.

A.5 Task-Inertia Damping

The joint-space mass matrix is solved rather than explicitly inverted. A 10^{-9} diagonal regularisation is used only for the task-inertia calculation; it is not the null-space pseudoinverse cutoff. Every intermediate result is checked before use, and a single non-finite or non-positive entry marks the estimate invalid.

Listing A.7: Task-inertia estimate and per-axis damping (abridged).

```
Eigen::LDLT<Mat7x7> mass_ldlt(joint_mass);
const Mat7x6 Minv_Jt = mass_ldlt.solve(J.transpose());
Mat6x6 lambda_inv = J * Minv_Jt;
lambda_inv = 0.5 * (lambda_inv + lambda_inv.transpose());

Mat6x6 lambda_inv_damped = lambda_inv;
lambda_inv_damped.diagonal().array() += 1e-9;
Eigen::LDLT<Mat6x6> lambda_ldlt(lambda_inv_damped);
Mat6x6 Lambda_base = lambda_ldlt.solve(Mat6x6::Identity());
Lambda_base = 0.5 * (Lambda_base + Lambda_base.transpose());

const Mat6x6 T_task = blockDiagonalRotation(R_task);
Mat6x6 Lambda_task = T_task.transpose() * Lambda_base * T_task;
Lambda_task = 0.5 * (Lambda_task + Lambda_task.transpose());

for (int i = 0; i < 3; ++i) {
  const double m = Lambda_task(i, i);
  const double I = Lambda_task(i + 3, i + 3);
  if (m <= 0.0 || I <= 0.0) {
    return result;  // estimate stays invalid
  }
  result.translational(i) = m;
  result.rotational(i) = I;
}

Vec3 criticalDampingFromStiffness(const Vec3& inertia,
                                  const Vec3& stiffness,
                                  double damping_ratio,
                                  const Vec3& min_damping,
                                  double max_damping) {
  Vec3 damping = Vec3::Zero();
  const double zeta = std::max(0.0, damping_ratio);
  for (int i = 0; i < 3; ++i) {
    const double m = std::max(0.0, inertia(i));
    const double k = std::max(0.0, stiffness(i));
```

```
    const double critical = 2.0 * zeta * std::sqrt(m * k);
    damping(i) = std::min(
        max_damping,
        std::max(std::max(0.0, min_damping(i)), critical));
  }
  return damping;
}
```

Only finite, positive directional inertia values are used. Otherwise, the phase-specific fixed damping matrix is applied. Each inertia-scaled coefficient is bounded above by `auto_damping_max`; the value configured for the reported experiments is listed in Appendix C. The lower bound `min_damping` is the configured manual matrix only when `auto_damping_min_from_manual` is set, and is zero otherwise.

A.6 Null-Space Torque

For $J \in \mathbb{R}^{6\times7}$, Eigen returns six singular values and a full 7×7 right-singular matrix. Column six in zero-based C++ indexing is therefore v_7, the structural null vector; it is distinct from v_6, the right-singular vector paired with $\sigma_{\min} = \sigma_6$. The two sampled configurations are evaluated by re-querying the robot model for the Jacobian at $q \pm \alpha n$, so the comparison uses the same kinematics as the control cycle.

Listing A.8: Truncated-SVD torque projector and the null-space law (abridged).

```
Eigen::JacobiSVD<Mat6x7> svd_current(
    J, Eigen::ComputeFullU | Eigen::ComputeFullV);
const auto singular_values = svd_current.singularValues();
const double svd_cutoff =
    std::max(0.0, params.nullspace_svd_relative_tolerance)
    * singular_values.maxCoeff();

Mat7x6 J_pinv = Mat7x6::Zero();
for (int i = 0; i < 6; ++i) {
  if (singular_values(i) > svd_cutoff) {
    J_pinv += (1.0 / singular_values(i))
            * svd_current.matrixV().col(i)
            * svd_current.matrixU().col(i).transpose();
  }
}

Mat7x7 N_tau =
    Mat7x7::Identity() - J.transpose() * J_pinv.transpose();
N_tau = 0.5 * (N_tau + N_tau.transpose());
const Vec7 dq_nullspace = N_tau * dq;

// Mode 0 commands nothing, but the projector above still observes
// the redundant axis for the diagnostics.
if (!params.use_nullspace_optimization ||
    params.nullspace_mode == NullspaceMode::kOff) {
  return Vec7::Zero();
}

const bool damping_enabled =
    params.nullspace_mode == NullspaceMode::kDampingOnly ||
    params.nullspace_mode == NullspaceMode::kDampingAndSigma;
Vec7 tau_damping = Vec7::Zero();
if (damping_enabled) {
  tau_damping.noalias() =
      -params.nullspace_damping * dq_nullspace;
```

```
}
if (params.nullspace_mode == NullspaceMode::kDampingOnly) {
  return tau_damping;
}
const bool sigma_only =
    params.nullspace_mode == NullspaceMode::kSigmaOnly;
const Vec7 sigma_fallback = sigma_only ? Vec7::Zero() : tau_damping;

Vec7 n = svd_current.matrixV().col(6);       // v_7
if (n.norm() <= 1e-9) { return sigma_fallback; }
n.normalize();
const double alpha = std::abs(params.nullspace_alpha);
if (alpha <= 1e-12) { return sigma_fallback; }

// The Jacobian is re-evaluated at the two sampled configurations.
Map<const Mat6x7> J_plus(model.zeroJacobian(
    Frame::kEndEffector, vec7ToArray(Vec7(q_current + alpha * n)),
    state.F_T_EE, state.EE_T_K).data());
Map<const Mat6x7> J_minus(model.zeroJacobian(
    Frame::kEndEffector, vec7ToArray(Vec7(q_current - alpha * n)),
    state.F_T_EE, state.EE_T_K).data());

const double sigma_difference =
    smallestSingularValue(J_plus) - smallestSingularValue(J_minus);
if (std::abs(sigma_difference) <= sigma.deadband) {
  return sigma_fallback;
}

const double sigma_direction = (sigma_difference > 0.0) ? 1.0 : -1.0;
const Vec7 best_direction = sigma_direction * n;
// alpha determines only where sigma is sampled. k_sigma is the
// commanded torque magnitude along the better direction.
const Vec7 tau_sigma =
    params.nullspace_k_sigma * N_tau * best_direction;

return sigma_only ? tau_sigma : tau_damping + tau_sigma;
```

Here the code variable `alpha` represents α_{probe} and changes only the two sampled configurations. It does not multiply the active torque. Singular values at or below the cutoff are omitted from the inverse. The routine also fills a diagnostics structure that carries the sampled singular values, the selected direction, and the projected joint velocity into the recorded data.

A.7 Core Control Callback

Listing A.9: Core impedance, coupled set-up, and torque composition.

```
const Vec3 e_p = desired.p_d - p_EE;
const Vec3 e_R = applyRotationalAxisMask(
    params, orientationError(R_EE, R_d_used), R_alignment_target);

Vec6 dx;
dx.head<3>() = e_p;
dx.tail<3>() = e_R;
Vec6 dv;
dv.head<3>() = desired.pdot_d - pdot;
dv.tail<3>() = -omega;

ContactReference contact;
contact.tcp = p_EE;
contact.edge = tool_contact_point;
contact.tcp_at_contact = first_contact_tcp;
contact.edge_at_contact = first_contact_point;
contact.R_EE = R_EE;
const Vec6 wrench =
    computeSpringWrench(params, phase, Kp_used, Dp_used,
```

```
                        KR_used, DR_used, R_alignment_target,
                        dx, dv, contact);

SigmaDiagnostics sigma_diagnostics;
const Vec7 tau_nullspace = computeNullspaceTorque(
    params, model, state, J, dq, sigma_diagnostics);

AutomaticDisturbance disturbance;
if (phase == ControlPhase::kHold &&
    params.disturbance_auto_enabled) {
  disturbance = computeAutomaticDisturbance(
      params, model, state, disturbance_force_direction_base,
      time - phase_start_time);
}

Array7 coriolis_array = model.coriolis(state);
Map<const Vec7> coriolis(coriolis_array.data());
const Vec7 tau_task = J.transpose() * wrench;
const Vec7 tau_cmd =
    tau_task + tau_nullspace + disturbance.tau + coriolis;
```

The null-space function is called in every non-manual phase. Manual guidance uses its separate Coriolis-plus-joint-damping branch. The disturbance term is non-zero only in the Cartesian pose-hold study of Section 4.7; it is zero in every surface-contact run.

Appendix B

Recorded Experimental Data

GREEN — sufficient. The appendix records the signals actually used in the evaluation and distinguishes controller fields from recalculated physical-plane quantities.

The controller stores experimental signals in a preallocated ring buffer. This avoids file access and dynamic buffer growth inside the 1 kHz control callback. After the controller stops, the retained rows are written in chronological order to a CSV file.

B.1 Control Phases

GREEN — sufficient. The numeric phase values are mapped directly to their recorded state names.

The integer `phase` identifies the active part of the experiment:

phase	0	1	2	3	4	5
state	approach_orient	approach_descend	set_up	grind	hold	manual_guide.

B.2 Signals Used in the Evaluation

GREEN — sufficient. The table is a useful reproducibility record and keeps units, frames, and model-estimated quantities explicit.

Appendix B.2 lists the groups used to calculate the reported quantities. A suffix `_x,_y,_z` denotes base-frame components, and `_1.._7` denotes joint components.

Table B.1: Recorded signal groups used in the evaluation.

Column group	Meaning
`time`, `phase`	Run time and active control state.
`p_EE`, `p_d`	Measured and desired TCP positions [m].
`tool_contact`, `edge_target`	Selected tool feature and its reference [m].
`tool_contact_offset_ee`	Offset of the selected feature from the TCP, in EE coordinates [m].
`first_contact_tcp`, `first_contact`	TCP position and projected surface reference captured at the geometric clearance transition; the transition is not force triggered.
`e_p`, `e_R`, `pdot`, `pdot_d`, `omega`	Cartesian errors and velocities. During set-up, `e_R` is measured relative to the clearance orientation selected at the transition and held constant during set-up.
`alignment_angle_deg`	Angle between the EE-inferred tool axis and the configured surface target [°]. The reported physical-plane angle is recalculated with n_{phys}.
`alignment_error_t1_deg`, `alignment_error_t2_deg`, `alignment_error_normal_deg`	The same angular error resolved on t_1, t_2, and n_s [°]. These supply the per-tangent decomposition reported for Cases A–E.
`f`, `m`	Commanded Cartesian force [N] and moment [N m].
`external_force`, `external_moment`	Model-estimated external wrench from libfranka, expressed in the base orientation and referenced to stiffness frame K.
`contact_force_bias`, `contact_moment_bias`	External-wrench values captured at the clearance transition.
`force_after_contact`, `moment_after_contact`	The model-estimated wrench with those bias values already subtracted. The reported normal load is taken from this group.

Continued on next page

Table B.1 continued

Column group	Meaning
`push`	Set-up penetration reference or grind preload retained after set-up [m].
`nullspace_mode`, `sigma_current`	Selected null-space law and current smallest singular value.
`sigma_plus`, `sigma_minus`, `sigma_difference`, `sigma_direction`	Smallest singular value at the two sampled configurations $q \pm \alpha_{\text{probe}} v_7$, their difference, and the selected sign. Recorded in every mode, including when no torque is commanded.
`nullspace_dq_1..7`	Projected joint velocity [rad/s].
`tau_sigma_1..7`, `tau_sigma_norm`, `tau_nullspace_norm`	Singular-value torque and total null-space torque [N m].
`nullspace_damping`	Active projected damping gain [N m s/rad].
`disturbance_scale`, `disturbance_force_base_x..z`, `disturbance_point_base_x..z`	Commanded hold-disturbance waveform, point force [N], and the base-frame position of the link point it acts at [m].
`tau_disturbance_1..7`	Equivalent joint torque $J_p^\top f_d$ in the internally commanded hold study [N m].
`tau_cmd_1..7`	Final commanded joint torque [N m].

B.3 Evaluation Quantities

GREEN — sufficient. The short summary routes each reported metric to its recorded source without reintroducing unused fields.

Contact alignment is calculated from the measured end-effector orientation and the independently calibrated physical-plane normal. The before value is taken at the start of set-up and the after value from the settled set-up interval. Selected-feature displacement is calculated from `tool_contact`, while the reported normal load is the bias-corrected

model estimate.

For the internally commanded hold study, redundant motion is calculated by integrating the norm of the recorded projected joint velocity over the disturbance interval. The conditioning response is the final change in $\sigma_{\min}$, and Cartesian task retention is represented by the peak position-error norm.

Appendix C

Controller Parameters

GREEN — sufficient. The active geometry, phase gains, damping policy, timings, null-space values, and collision settings are collected in one reproducibility record.

This appendix records the active controller configuration used for the reported experiments. It is a reproducibility record for the implementation described in Chapters 3 and 4. The controller loads fifteen disjoint files in a fixed order: `Run_Settings.txt`, `safety.txt`, `Gripper_Action.txt`, `Auto_Damping.txt`, `Plane_Definition.txt`, `Tool_Orientation.txt`, `Tool_Geometry.txt`, `Q_Init.txt`, `Approach_Phase.txt`, `Clearance_Gate.txt`, `SetUp_Phase.txt`, `Grind_Phase.txt`, `Nullspace.txt`, `hold.txt`, and `guidance.txt`. The values below are those recorded with each run rather than the defaults compiled into the controller.

C.1 Surface and Tool Geometry

GREEN — sufficient. The configured geometry and initial joint vector are stated with their exact keys and units.

The plane satisfies

$$n_s^\top (p - p_s) = 0, \qquad n_s = R_y(b) R_x(a) e_z,$$

and the surface-frame column order is $[t_1, t_2, n_s]$.

Table C.1: Surface and tool parameters.

Quantity	Configuration key	Value
Surface point	`surface_point`	$[0.515267, -0.107172, 0.003108]$ m
Surface tilts	a, b	$-1.585°,\ +0.988°$
Entered first tangent	`alignment_target_tangent1`	$[-0.905786, -0.422854, 0.027332]$
Tool axis in EE	`tool_axis_ee`	$[-0.006011, 0.000246, 0.999982]$
Signed alignment target	`tool_axis_target_sign`	$-n_s$
Commanded spin about n_s	`tool_target_offset_normal_deg`	$115.05°$
Grinding-face centre in EE	`tool_contact_face_center_ee`	$[0, 0, 0.020]$ m
Grinding-face size	width × length	0.040×0.120 m
Feature tie tolerance	`tool_contact_feature_tie_tolerance`	10^{-4} m
Initial-joint case	`q_init_case`	`horizontal_table_search`

This case selects the `q_init_table` entries, so the corresponding initial joint vector is

$$q_{\text{init}} = [0.014777,\ 0.210911,\ -0.009320,\ -2.284835,\ -0.002725,\ 2.493410,\ 0.780694]^{\top} \text{ rad}.$$

C.2 Phase Sequence and Impedance

GREEN — sufficient. The phase-specific gain frames and transition parameters reconcile the values stated across Chapters 3 and 4.

Table C.2: Phase and mode gains.

Phase / quantity	Active stiffness and frame	Damping fallback
Approach translation	$[150, 150, 150]$ N/m, surface $[t_1, t_2, n_s]$	$[20, 20, 20]$ N s/m
Approach rotation	$[90, 90, 90]$ N m/rad, surface $[t_1, t_2, n_s]$	$[12, 12, 12]$ N m s/rad
Pre-set-up gate translation	5000 N/m, isotropic base	200 N s/m
Pre-set-up gate rotation	90 N m/rad, inherited approach rotation	inherited cached approach D_R
Set-up translation (compliance-centre source)	$[2000, 2000, 800]$ N/m, surface $[t_1, t_2, n_s]$	$[50, 50, 25]$ N s/m
Set-up rotation (compliance-centre source)	$[5, 5, 50]$ N m/rad, surface $[t_1, t_2, n_s]$	$[10.01, 10.01, 10]$ N m s/rad
Grind translation (decoupled)	$[2000, 2000, 800]$ N/m, surface $[t_1, t_2, n_s]$	$[50, 50, 25]$ N s/m
Grind rotation (decoupled)	$[5, 5, 50]$ N m/rad, surface $[t_1, t_2, n_s]$	$[10.01, 10.01, 10]$ N m s/rad
Hold translation	2000 N/m, isotropic base	50 N s/m
Hold rotation	50 N m/rad, isotropic base	8 N m s/rad
Manual guidance	no Cartesian stiffness; joint coordinates	0.5 N m s/rad joint damping

The enabled pre-set-up gate overrides only translational stiffness. The pre-grind gate was also enabled; it remains in set-up and keeps the coupled pressed law rather than using the pre-set-up gate gains.

With `setup_translation_surface_frame` set, the set-up and grind translational blocks are read from the surface-frame keys listed above. The base-frame keys `setup_Kp` and `setup_Dp` remain in the file at $[2000, 2000, 350]$ N/m and $[10, 10, 175]$ N s/m but take no part in the reported runs.

Table C.3: Phase-transition and trajectory parameters.

Quantity	Configuration key	Value
Orient minimum time	`approach_orient_min_time`	0.5 s
Tool-axis switch tolerance	`approach_orient_error_threshold`	0.035 rad
Descent speed / maximum distance	`descend_speed`, `descend_max_distance`	0.1 m/s, 0.640 m
Surface clearance	`descend_surface_clearance`	0.020 m
Set-up minimum time / timeout	`setup_min_time`, `setup_timeout`	2.0 s, 5.0 s
Set-up moment-change threshold	`setup_moment_threshold`	150 N m
Set-up push speed / endpoint	`setup_push_speed`, `setup_push_end`	0.050 m/s, 0.060 m
Grind sweep	`grind_sweep_enabled`	disabled (configured t_2, 0.030 m, 0.2 Hz)
Pre-set-up / pre-grind gates	`pause_before_set_up`, `pause_before_grind`	enabled, enabled

C.3 Inertia-Scaled Damping and Centre of Compliance

YELLOW — weak or unclear. The active policy is complete, but the fitted damping factors are not listed per run and the $\zeta = 1$ heuristic is not a validated modal damping ratio.

Inertia-scaled damping follows eqs. (3.9) and (3.10). It is enabled for hold, approach, set-up, grind, and the pause gate. The approach factor is $\zeta = 1.9$, while set-up and grind use $\zeta = 1.0$. Hold uses one derived damping ratio for translation and rotation. The pause gate uses separate translational and rotational factors because the two gain blocks are expressed in different frames. The configured maximum coefficient `auto_damping_max` is 400, and the manual values in table C.2 provide the fixed damping when an inertia-scaled value is unavailable. With `auto_damping_min_from_manual` left at zero, those manual values act only as that fallback and impose no lower bound on a calculated coefficient.

For set-up, the source damping blocks are calculated from TCP task inertia and then shifted to the selected centre of compliance. This preserves symmetry and positive semidefiniteness, but the factor $\zeta = 1$ remains a directional tuning heuristic; it does not establish a pole-level modal damping ratio for the coupled robot–contact system.

The coupled 6×6 law is enabled only during set-up, and only in Case D. The lever was entered directly in surface coordinates through `coupled_use_direct_rc_surface`, so

$$r_c = R_{\text{surface}}\, r_c^S, \qquad r_c^S = [\, r_{c,t_1},\ r_{c,t_2},\ r_{c,n}\,]^\top,$$

with the tested components listed in table 4.1. The alternative offset-from-edge convention, in which the centre of compliance is placed relative to the tool feature projected onto the surface at the 20 mm clearance hand-off, is retained in the implementation for archived configuration files and was not used for the reported runs. With `coupled_pole_freeze_at_contact=1`, the references are those captured at the hand-off, so the active r_c is constant throughout set-up.

C.4 Null-Space and Operating Parameters

GREEN — sufficient. The selected mode, gains, probe settings, logging capacity, gripper state, and robot-side thresholds are stated explicitly.

The null-space settings are explained in section 4.4.5. The final two rows of Table C.4 are Franka collision/reflex thresholds configured through the libfranka collision-behaviour interface; they are neither commanded-torque saturation nor a software wrench limiter.

Table C.4: Null-space and operating parameters.

Parameter	Symbol / key	Value
Null-space mode	`nullspace_mode`	1 (projected damping only)
Null-space damping	d_{null}	2.0 N m s/rad
Sigma torque magnitude	k_σ	1.0 N m (inactive in mode 1)
Probe distance	α_{probe}	0.08 rad
Sigma difference deadband	δ_σ	10^{-6}
Relative SVD cutoff	ρ	10^{-4}
Recorded-data sampling / capacity	`log_every_n_cycles`, rows	every callback (1), 120000
Run duration / stop	`experiment_duration`	0 s: indefinite until `e+Enter`
Gripper grasp	width, speed, force	0.066 m, 0.050 m/s, 60 N
Tool-mount rotational play	physical observation	approximately $\pm 2^\circ$ about y_{EE}
Manual-guidance damping	`manual_guidance_damping`	0.5 N m s/rad
Joint-side collision threshold	acceleration/nominal	140 N m on each axis
Cartesian collision threshold	acceleration/nominal	240 N or N m on each channel